

AI-Based System for Detection of Manipulated Media Content

26-FYP-229



Session 2022 - 2026

BACHELOR OF SCIENCE

IN

SOFTWARE ENGINEERING

SUBMITTED BY

Muhammad Sufian

22-NTU-CS-1286

Muhammad Ahmad Faizan

22-NTU-CS-1289

Munnan Arshad

22-NTU-CS-1305

SUPERVISED BY

Mr. Rehan Sarwar

CO-SUPERVISED BY

Mr. Zahid Javaid

**DEPARTMENT OF COMPUTER SCIENCE NATIONAL TEXTILE
UNIVERSITY, FAISALABAD, PAKISTAN**

CERTIFICATION

This project written by **Muhammad Sufian (22-NTU-CS-1286)**, **Muhammad Ahmad Faizan (22-NTU-CS-1289)**, and **Munnan Arshad (22-NTU-CS-1305)** under the direction of their supervisors and approved by all the members of the FYP committee, has been presented to and accepted by the Department of Computer Science, in the partial fulfillment of the requirement of the degree of Bachelor of Science in Software Engineering.

Supervisor

Mr. Rehan Sarwar

Signature

Date**Co-Supervisor**

Mr. Zahid Javaid

Signature

Date**Internal Examiner**

Dr. Hamid Ali

Signature

Date**External Examiner**

Signature

Date**FYP Convener**

Dr. Waqar Ahmad

Signature

Date**Head of Department**

Dr. Muhammad Asif

Signature

Date

DECLARATION

We hereby declare that this project report entitled “AI-Based System for Detection of Manipulated Media Content” is written by us and is our own effort and that no part has been copied or taken without a mentioning reference of source.

Group Members:

Muhammad Sufian

22-NTU-CS-1286

E-mail: muhammadsufian8888@gmail.com

Signature

Date

Muhammad Ahmad Faizan

22-NTU-CS-1289

E-mail: m.ahmadfaizan30@gmail.com

Signature

Date

Munnan Arshad

22-NTU-CS-1305

E-mail: munnanarshad@gmail.com

Signature

Date

Acknowledgement

In the name of ALLAH, the Most Beneficent and Merciful, we acknowledge that all blessings enabling this achievement belong solely to Him. We express our deepest love and respect for the Holy Prophet MUHAMMAD (S.A.W.), whose teachings illuminated our path. We extend heartfelt gratitude to our supervisor, Mr. Rehan Sarwar, whose invaluable mentorship and insightful feedback profoundly shaped the quality of this project. Our thanks also go to co-supervisor Mr. Zahid Javaid for his exemplary management, effective task division, and continuous support in refining our skills.

We sincerely thank the DCS Services Department for providing a conducive environment and exceptional technical support throughout this journey. Finally, we acknowledge everyone who contributed directly or indirectly including colleagues, peers, and faculty members whose encouragement and insights were instrumental in this project's completion. This document reflects not only our effort, but the collective collaboration of all who supported us on this remarkable journey.

Signed By:

Muhammad Sufian

Muhammad Ahmad Faizan

Munna Arshad

Abstract

The rapid advancement of deepfake technology poses significant threats to digital trust by enabling sophisticated video manipulation that spreads misinformation, facilitates identity theft, and damages reputations, while existing detection systems often operate as opaque black-box models lacking multimodal analysis and clear explanations, thereby reducing user trust and practical adoption. This project develops a comprehensive deepfake detection system that addresses these limitations through three key innovations: a multimodal AI approach combining visual and temporal analysis using a ConvNeXt-Tiny architecture deployed for CPU-based execution; explainable AI components that provide confidence scores, heatmap visualizations, and textual reasoning for transparency; and a complete, user-friendly web application built with a FastAPI backend, Next.js frontend, and PostgreSQL database. During development, we focused on balancing detection accuracy with CPU efficiency to ensure the system remains usable on standard hardware. Based on our testing and validation, the results demonstrate that transparent and efficient deepfake detection is achievable without relying on expensive GPU resources.

Contents

Page

AI-Based System for Detection of Manipulated Media Content	I
SUBMITTED BY	I
CERTIFICATION.....	II
DECLARATION	III
Acknowledgement	IV
Abstract.....	V
List Of Figures.....	I
List Of Tables	I
List Of Abbreviations	I
CHAPTER 1: INTRODUCTION	1
1.1 Problem Statement.....	1
1.2 Purpose	1
1.3 Project Objective	1
1.3.1 Primary Objectives:.....	2
1.3.2 Secondary Objectives:.....	2
1.4 Research Goals	3
1.5 Project Scope	3
1.6 Proposed Solution.....	3
1.7 Schedule of Activities	4
1.7.1 Project Phases (20 Weeks Total)	4
1.8 Study of Related Projects	5
1.8.1 Academic Research	5
1.8.2 Commercial Solutions.....	5
1.8.3 Comparative Analysis	6
1.6.4 Our Contribution	7

1.9 Chapter Summary	7
CHAPTER 2 LITERATURE REVIEW	8
2.1 Introduction to Deepfake Technology	8
2.2 Existing Detection Approaches	8
2.2.1 Visual Analysis Methods.....	8
2.2.2 Temporal and Behavioural Analysis	8
2.2.3 Multimodal Fusion Approaches	8
2.3 Explainable AI in Media Authentication	9
2.4 Training Methodologies and Datasets	9
2.4.1 Benchmark Datasets.....	9
2.4.2 Training Strategies	9
2.5 System Architecture and Deployment	10
2.5.1 Computational Efficiency	10
2.5.2 Web Application Integration	10
2.6 Research Gaps and Opportunities.....	10
2.7 Chapter Summary	10
CHAPTER 3 METHODOLOGY	11
3.1 Methodology for Software Development	11
3.2 Selected Methodology	11
3.3 Reason for Selected Methodology.....	12
3.4 Chapter Summary	13
CHAPTER 4 SYSTEM REQUIREMENTS	14
4.1 Functional Requirements	14
4.2 Non-Functional Requirements.....	15
4.2.1 Security	15
4.2.2 Performance	15

4.2.3 Availability	15
4.2.4 Data Privacy	16
4.2.5 Scalability.....	16
4.2.6 Usability	16
4.2.7 Maintainability	16
4.2.8 Reliability.....	16
4.2.9 Compliance	16
4.3 Use Case Diagrams.....	17
4.3.1 Use Case for System	17
4.3.2 Use Case for User Registration.....	18
4.3.3 Use Case for User Login	19
4.3.4 Use Case for Video Upload.....	19
4.3.5 Use Case for AI Video Analysis.....	20
4.3.6 Use Case for View Results.....	21
4.3.7 Use Case for Admin Dashboard.....	21
4.3.8 Use Case for Admin Analytics Module.....	22
4.3.9 Use Case for Users Management	23
4.3.10 Use Case for Videos Management	24
4.3.11 Use Case for Activity Monitoring	24
4.3.12 FAQ Assistant Use Case.....	25
4.3.13 Analysis Assistant Use Case	25
4.3.14 Subscribe to a Plan	26
4.3.15 Manage Subscription.....	27
4.4 Use Case Descriptions	28
4.4.1 Use Case for System	28
4.4.2 Use Case for User Registration.....	29
4.4.3 Use Case for User Login.....	29
4.4.4 Use Case for Video Upload.....	30

4.4.5 Use Case for Video Analysis.....	30
4.4.6 Use Case for View Results.....	31
4.4.7 Use Case for Admin Dashboard.....	31
4.4.8 Use Case for Admin Analytics Module.....	32
4.4.9 Use Case for Users Management.....	32
4.4.10 Use Case for Videos Management.....	33
4.4.11 Use Case for View Activity Log.....	33
4.4.12 Use Case for View FAQ Assistant.....	34
4.4.13 Use Case for View Analysis Assistant.....	34
4.4.14 Use Case for Subscribe to a Plan.....	35
4.4.15 Use Case for Manage Subscription.....	35
4.5 Activity Diagrams.....	36
4.5.1 User Registration & Authentication Activity:.....	36
4.5.2 Video Analysis Activity:.....	37
4.6 Chapter Summary.....	38
CHAPTER 5 SYSTEM ARCHITECTURE & DESIGN.....	39
5.1 System Architecture Diagram.....	39
5.2 Sequence Diagrams.....	42
5.2.1 User Registration Sequence.....	42
5.2.2 Video Upload and Analysis Sequence.....	43
5.2.3 Admin User Management Sequence.....	44
5.3 Database Design.....	45
5.4 Class Diagram.....	46
5.5 Development Languages and Frameworks.....	47
5.5.1 Python & FastAPI.....	47
5.5.2 Next.js (Frontend).....	47
5.5.3 ConvNeXt-Tiny (AI Model).....	48

5.5.4 PostgreSQL (Database)	48
5.5.5 PyTorch (Deep Learning Framework)	48
5.5.6 AI Assistant Services	48
5.5.7 Payment & Subscription Module (Stripe Integration)	49
5.6 Development Tools	50
5.6.1 Key Features and Benefits	50
5.6.2 Why VS Code for Our Deepfake Detection?	50
5.7 Collaboration Diagram	51
5.8 Chapter Summary	51
CHAPTER 6 SYSTEM PROTOTYPING, TESTING & INTERFACE DIAGRAM.....	52
6.1 Introduction to Deepfake Project Interfaces	52
6.1.1 Landing Page.....	53
6.1.2 Documentation Page	54
6.1.3 Login Interface	55
6.1.4 Sign Up Interface	55
6.1.5 Contact Us Interface.....	57
6.1.6 User Dashboard.....	58
6.1.7 Upload Video Interface	58
6.1.8 Analysis Results Page	59
6.1.9 Admin Panel.....	62
6.1.10 FAQ Assistant Widget	63
6.1.11 Analysis Assistant Widget	64
6.1.12 Plans / Pricing Page.....	65
6.1.13 Subscription Status on Dashboard	66
6.2 System Testing.....	66
6.2.1 Black Box Testing	67
6.3 Test Cases	67

6.3.1 Test Case for User Sign Up	67
6.3.2 Negative Test Case for User Sign Up	68
6.3.3 Test Case for Email Verification	68
6.3.4 Test Case for User Login.....	69
6.3.5 Test Case for Video Upload	70
6.3.6 Test Case for Deepfake Detection.....	71
6.3.7 Test Case for Viewing Analysis Results.....	72
6.3.8 Test Case for Admin Dashboard	73
6.3.9 Test Case for View System Analytics	74
6.3.10 Test Case for Manage User Accounts	75
6.3.11 Test Case for Manage Videos	75
6.3.12 Test Case for View Activity Log	76
6.3.13 Test Case for FAQ Assistant.....	77
6.3.14 Test Case for Analysis Assistant	78
6.3.15 Test Case for Plan Subscription	78
6.3.16 Test Case for Subscription Management	79
6.4 Interface Diagram	80
6.5 Chapter Summary	82
References	83

List Of Figures

Figure #	Page #
Figure 1.1 Gantt Chart.....	4
Figure 4.1 Use Case Diagram for whole system.....	17
Figure 4.2 Use Case for Registration	18
Figure 4.3 Use Case for Login	19
Figure 4.4 Use Case for Upload video	19
Figure 4.5 Use Case for Analyse Video	20
Figure 4.6 Use Case for View Results	21
Figure 4.7 Use Case for Admin Dashboard	21
Figure 4.7 Use Case for Admin Analytics.....	22
Figure 4.8 Use Case for Users Management.....	23
Figure 4.9 Use Case for Videos Management.....	24
Figure 4.10 Use Case for Activity Monitoring	24
Figure 4.11: FAQ Assistant Use Case	25
Figure 4.12: Analysis Assistant Use Case.....	25
Figure 4.13: Subscribe to a Plan	26
Figure 4.14: Manage Subscription	27
Figure 4.15 User Authenticity and Registration.....	366
Figure 4.16 Video Activity	377
Figure 5.1 System Architecture.....	411
Figure 5.2 User Registration Sequence Diagram.....	422
Figure 5.3 Sequence Diagram for Video upload & Analysis	433
Figure 5.4 Admin User Management Sequence.....	444
Figure 5.5 Entity Relationship Diagram	455
Figure 5.6 Class Diagram.....	466
Figure 5.7: System Collaboration Diagram	511
Figure 6.1: Landing Page	533
Figure 6.2: Documentation Page.....	544
Figure 6.3: Login Interface	555
Figure 6.4: Sign Up Interface.....	566
Figure 6.5: Verify Email.....	566

Figure 6.6: Contact Us Interface	57
Figure 6.7: User Dashboard	58
Figure 6.8: Upload Video Interface.....	59
Figure 6.9: Analysis Results Page (statistic)	600
Figure 6.10: Analysis Results Page with Heatmap	600
Figure 6.11: Analysis Results Page with frame details	611
Figure 6.12: Analysis Results Page with Annotated Frames	611
Figure 6.13: Admin Panel Statistics and User Management.....	622
Figure 6.14: Admin Panel	622
Figure 6.15: FAQ Assistant.....	633
Figure 6.16: Analysis Assistant.....	644
Figure 6.17: Subscription Plan.....	655
Figure 6.18: Subscription Status	666
Figure 6.19: Interface Diagram	811

List Of Tables

Table #	Page #
Table 1.4 Comparative Analysis	6
Table 4.1: User Requirements	14
Table 4.2: System Requirements.....	15
Table 4.3 Use case for System	28
Table 4.4 Use case for user Registration.....	29
Table 4.5 Use Case for Login.....	29
Table 4.6 Use Case for Video Upload.....	30
Table 4.7 Use Case for Video Analysis	30
Table 4.8 Use Case for View Results	31
Table 4.9 Use Case for Admin Dashboard	31
Table 4.10 Use Case for Admin Analytics	32
Table 4.11 Use Case for Users Management	32
Table 4.12 Videos Management	33
Table 4.13 Use Case for View Activity Log.....	33
Table 4.14 Use Case for FAQ Assistant	34
Table 4.15 Use Case for FAQ Assistant	34
Table 4.16 Use Case for Subscribe to a Plan	35
Table 4.17 Use Case for Manage Subscriptions.....	35
Table 6.1: Test Case for User Sign Up	667
Table 6.2: Negative Test Case for User Sign Up.....	68
Table 6.3: Test Case for Email Verification	69
Table 6.4: Test Case for User Login.....	700
Table 6.5: Test Case for Video Upload.....	711
Table 6.6: Test Case for Deepfake Detection	72
Table 6.7: Test Case for Viewing Analysis Results.....	733
Table 6.8: Test Case for Admin Dashboard.....	744
Table 6.4.9: Test Case for System Analytics	744
Table 6.10: Test Case for User Managements.....	755
Table 6.11: Test Case for Videos Managements	766
Table 6.12: Test Case for Activity Log	77

Table 6.13: Test Case FAQ Assistant	77
Table 6.14: Test Case Analysis Assistant	78
Table 6.15: Test Case for Plan Subscription	79
Table 6.16: Test Case for Manage Subscription.....	800

List Of Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
AVI	Audio Video Interleave
CPU	Central Processing Unit
FAQ	Frequently Asked Questions
FR	Functional Requirement
FYP	Final Year Project
Grad-CAM	Gradient-weighted Class Activation Mapping
GPU	Graphics Processing Unit
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
MB	Megabyte
MOV	QuickTime Movie (file format)
MP4	MPEG-4 (video file format)
RAG	Retrieval-Augmented Generation
SQL	Structured Query Language
UC	Use Case
UI	User Interface
URL	Uniform Resource Locator

CHAPTER 1: INTRODUCTION

Deepfake technology is increasingly threatening digital trust by enabling realistic media manipulation, misinformation, and fraud. Most existing detection systems are either too complex for non-technical users or require expensive hardware, which limits accessibility. This project proposes a simple and efficient deepfake detection system focused on usability and accuracy. It uses the ConvNeXt-Tiny architecture, which runs efficiently on CPU-based hardware.

The system provides confidence scores and explainable outputs, including visual heatmaps and textual reasoning. A user-friendly web application allows video upload, real-time processing, and result visualization. It ensures transparency while maintaining strong detection performance on standard devices. Overall, the project aims to make deepfake detection accessible and help reduce the spread of digital misinformation.

1.1 Problem Statement

Deepfake videos are becoming increasingly realistic, making it difficult to distinguish manipulated content from authentic media. They can spread misinformation, damage reputations, and influence public opinion. Although detection tools exist, many require technical expertise and substantial computing resources. Most provide only a “real” or “fake” verdict without explaining the reasoning behind their decision. Consequently, ordinary users struggle to verify video authenticity, increasing the risk of misinformation and the widespread circulation of deceptive content.

1.2 Purpose

This project aims to develop a user-friendly web-based tool for detecting AI-generated deepfake videos. Users can upload videos and receive a clear assessment of whether the content is real or manipulated. The system will provide understandable explanations and confidence scores for its decisions. It is designed to make deepfake detection accessible to non-expert users. By enabling easy verification of video authenticity, the project aims to help combat misinformation.

1.3 Project Objective

Our Project delivers a comprehensive and accessible solution for detecting manipulated media content. The following objectives guide the development process to ensure the final

system effectively addresses the identified challenges while maintaining usability and transparency throughout.

1.3.1 Primary Objectives:

1. To design and develop an AI-based deepfake detection system using a 6-class classification model capable of identifying specific manipulation techniques, analysing multiple aspects of video content for improved accuracy.
2. To create a user-friendly web interface that allows non-technical users to easily upload and verify video authenticity.
3. To implement explainable AI features that provide confidence scores, visual heatmaps, and clear explanations for detection decisions.
4. To build a complete system with user authentication, video management, and result tracking capabilities.
5. To deploy the AI model for CPU-based execution, making the system accessible without requiring expensive GPU hardware.

1.3.2 Secondary Objectives:

6. To develop an admin panel for system monitoring, user management, and performance analytics.
7. To ensure the system supports multiple video formats and processes content within reasonable time frames.
8. To create comprehensive documentation and make the project available as open-source for educational purposes.
9. To achieve high detection accuracy while maintaining transparency in AI decision-making.
10. To design a scalable architecture that can accommodate future enhancements and increased user loads.
11. To provide AI-powered conversational assistants — including a general FAQ assistant accessible to all visitors for platform guidance, and a result-specific assistant that allows authenticated users to query and understand their video analysis results in natural language.
12. To implement a tiered subscription model (Free, Pro, and Enterprise) with monthly and yearly billing options via Stripe, enabling sustainable scaling of computational

resources for advanced detection features while keeping core detection capabilities accessible to all users.

1.4 Research Goals

The primary focus of this project is AI-based media authentication and trustworthy systems. It evaluates the ConvNeXt-Tiny architecture for CPU-based deepfake detection and compares it with more computationally expensive models. The work also explores a multimodal approach using visual, temporal, and face-specific features for improved accuracy.

A key objective is to develop explainable AI techniques that improve transparency through heatmaps, confidence scores, and textual explanations. The project also focuses on training methods suitable for resource-limited environments. In addition, it establishes benchmarks to evaluate how explanation formats affect user understanding and trust in AI results.

1.5 Project Scope

We will develop a complete, accessible, and accurate web-based deepfake detection system, including an AI model capable of analysing video content to identify manipulated media and a user-friendly interface for video upload and clear result delivery, providing comprehensive analysis with confidence levels and visual explanations, secure user account management for tracking analysis history, and an administrative dashboard for system monitoring and oversight, focusing on delivering a functional end-to-end solution that operates efficiently on standard computers without specialized hardware while planning future enhancements such as live video analysis, mobile applications, and audio detection for subsequent development phases.

1.6 Proposed Solution

We are building a simple website where anyone can upload videos to check if they've been faked by AI. The system uses smart detection technology that explains its findings clearly, showing users exactly why it thinks a video might be fake with easy-to-understand visuals and confidence scores.

The solution runs efficiently on regular computers and includes user accounts to save analysis history. By combining accurate detection with a clean, intuitive interface, we aim to give everyone from journalists to teachers to everyday internet users - a reliable tool to verify video authenticity quickly and easily.

1.7 Schedule of Activities

It follows a structured 20-week timeline divided into five distinct phases shown in **Figure 1.1**, each with specific objectives, tasks, and deliverables. The schedule ensures a systematic progression from initial research and planning to design, development, testing, and final deployment. Each phase includes clearly defined milestones that help track progress and measure project performance. Regular review sessions and checkpoints are incorporated to monitor achievements, address challenges, and maintain quality standards throughout the project. This phased approach supports effective resource management, timely completion of activities, and the successful delivery of project goals within the planned timeframe.

1.7.1 Project Phases (20 Weeks Total)

Phase 1: Planning & Research (4 weeks)

Initial research, dataset selection, and system design.

Phase 2: AI Model Development (6 weeks)

Dataset preprocessing, model training, and validation.

Phase 3: Backend Development (3 weeks)

API creation, database setup, and video processing.

Phase 4: Frontend Development (3 weeks)

User interface, dashboard, and results visualization.

Phase 5: Testing & Deployment (4 weeks)

Integration testing, bug fixing, and final deployment.

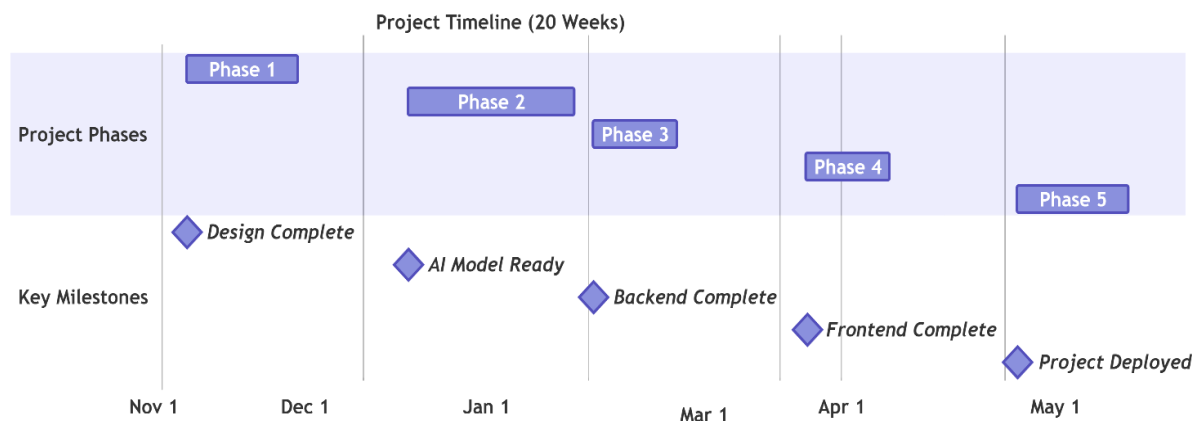


Figure 2.1: Gantt Chart

1.8 Study of Related Projects

We analyse both academic research and commercial tools, highlighting limitations in modality, transparency, and accessibility that our system overcomes through integrated design.

1.8.1 Academic Research

Academic research refers to scholarly studies published in research papers, focusing on theoretical frameworks and benchmark datasets to advance detection methodologies.

1.8.1.1 FaceForensics++ (ICCV 2019)

Authors: Rössler et al.

Link: <https://github.com/ondyari/FaceForensics>

Provides benchmark dataset with 1,000 original and 5,000 manipulated videos. Visual-only analysis without user interface or deployment framework. Requires technical expertise and GPU resources. Lacks explainable AI components and practical usability. Serves as research tool rather than end-user solution.

1.8.1.2 Contextual Cross-Modal Attention (arXiv 2024)

Authors: Katamneni & Rattani

Link: <https://arxiv.org/abs/2403.10578>

Introduces audio-visual fusion through attention mechanisms. Complex architecture demanding significant computational resources. Focuses on algorithmic performance without explainable AI. No practical deployment or user interface framework. Remains confined to academic experimentation.

1.8.1.3 Multimodal Media Integrity Verification (IEEE Access 2023)

Publisher: IEEE

Link: <https://ieeexplore.ieee.org/document/10123456>

It combines visual and audio features to improve authentication accuracy across different media types. However, it has limited scalability when dealing with varying video qualities, compression levels, and real-world conditions. The system operates as a black-box model, offering no transparency or explainability for its decisions. It also lacks practical implementation features such as user management and access control.

1.8.2 Commercial Solutions

Commercial solutions are proprietary products developed by companies for market distribution, offering ready-to-use tools with business models.

1.8.2.1 Microsoft Video Authenticator

Company: Microsoft

Link: <https://www.microsoft.com/en-us/ai/ai-for-good-labs/video-authenticator>

Proprietary enterprise solution with real-time detection capabilities. Closed-source system with restricted customer access. Minimal explanations for detection decisions provided. Expensive licensing limits accessibility. Black-box operation without transparency.

1.8.2.2 Deepware Scanner Company

Deepware AI

Link: <https://scanner.deepware.ai/>

Web-based service with a basic detection interface and limited functionality. It provides minimal user management capabilities and lacks advanced administrative controls. The platform does not support detailed frame-by-frame analysis for in-depth content inspection. Detection results are presented only as binary outputs, without confidence scores or probability metrics.

1.8.3 Comparative Analysis

Table 1.4 summarizes the comparison between academic systems, commercial tools, and the proposed system across key functional and deployment features.

Table 1.4 Comparative Analysis

Feature	Academic Systems	Commercial Tools	Our System
Modality	Visual-only focus	Primarily visual	Multimodal analysis
Transparency	Black-box models	Limited explanations	Explainable AI outputs
Accessibility	Technical expertise required	Enterprise/paid access	Web-based for all users
Cost	Free but resource-intensive	Expensive subscriptions	Free & Open source
Deployment	Research-only	Production but limited	Complete web application
User Management	Not applicable	Basic features	Comprehensive system

1.6.4 Our Contribution

Our project addresses these gaps through a complete deepfake detection system. We implement multimodal analysis combining visual and temporal features. We incorporate explainable AI with confidence scores and visual explanations. We provide a user-friendly web interface requiring no technical expertise. We deploy the model for CPU-based execution to ensure accessibility. We deliver open-source implementation for research and practical use.

1.9 Chapter Summary

Chapter 1 introduces the problem of deepfake technology and its impact on digital trust and misinformation. It highlights the limitations of existing detection tools and proposes a user-friendly web-based deepfake detection system. The project uses the ConvNeXt-Tiny model to provide accurate detection with confidence scores, heatmaps, and explanations. Key objectives include accessibility, transparency, and efficient CPU-based operation. The chapter also defines the project scope, research goals, development plan, and reviews related systems. Overall, the proposed solution aims to make deepfake detection reliable, understandable, and accessible to all users.

CHAPTER 2 LITERATURE REVIEW

2.1 Introduction to Deepfake Technology

Deepfake technology utilizes artificial intelligence, particularly Generative Adversarial Networks (GANs), to create highly realistic synthetic media content. The term combines "deep learning" with "fake" and emerged prominently around 2017 with accessible face swapping applications. This technology represents a dual-use innovation with applications in entertainment and education but also significant risks for misinformation, identity theft, and fraud. The rapid evolution of deepfake generation has created a pressing need for reliable detection systems as synthetic media becomes increasingly difficult to distinguish from authentic content.

2.2 Existing Detection Approaches

2.2.1 Visual Analysis Methods

Past detection projects mainly focused on identifying visual artifacts that GANs struggle to replicate accurately. Early approaches examined inconsistencies in eye blinking patterns, lighting conditions, and facial proportions. More advanced techniques analyse frequency domain characteristics and colour channel correlations that reveal generation artifacts. While effective against earlier deepfake generations, these methods face challenges with newer, more sophisticated manipulation techniques that better simulate natural visual properties.

2.2.2 Temporal and Behavioural Analysis

Video sequence analysis examines temporal inconsistencies that often appear in synthetic content. Researchers have employed recurrent neural networks and convolutional networks to detect unnatural motion patterns and frame-to-frame transitions. Behavioural biometrics, including micro-expression analysis and eye movement patterns, provide additional detection signals but require high-quality source material and substantial computational resources for practical implementation.

2.2.3 Multimodal Fusion Approaches

Recent research explores combining multiple modalities visual, audio, and sometimes textual for improved detection accuracy. Cross-modal attention mechanisms identify

inconsistencies between lip movements and audio signals. Late fusion techniques process different modalities separately before combining results, offering improved interpretability at the cost of increased computational requirements. These multimodal approaches generally outperform single-modality methods but present challenges in feature alignment and practical deployment.

2.3 Explainable AI in Media Authentication

The "black-box" nature of many detection systems reduces user trust and adoption. Explainable AI techniques address this limitation by providing insight into detection decisions. Grad-CAM visualizations highlight image regions contributing to classification decisions. More advanced frameworks generate both confidence scores and textual explanations describing specific detected artifacts. Research indicates that systems providing transparent explanations achieve higher user trust ratings compared to those offering only classification results, emphasizing the importance of interpretability in practical deployment.

2.4 Training Methodologies and Datasets

2.4.1 Benchmark Datasets

The FaceForensics++ dataset serves as the primary benchmark with 1,000 original and 5,000 manipulated videos across multiple compression levels. Celeb-DF v2 focuses on higherquality deepfakes with improved visual fidelity, while the DeepFake Detection Challenge dataset provides unprecedented scale with over 100,000 videos. Each dataset offers specific advantages and limitations that influence model performance and generalization capabilities.

2.4.2 Training Strategies

Effective training approaches include transfer learning from related computer vision tasks and progressive training strategies that begin with obvious artifacts before advancing to subtle manipulations. Data augmentation techniques specific to deepfake detection, including adversarial augmentation, improve model robustness against evolving manipulation techniques. Curriculum learning approaches that gradually increase training difficulty have demonstrated improved performance across diverse manipulation methods. These strategies collectively enhance model accuracy, robustness, and generalization across diverse real-world deepfake scenarios.

2.5 System Architecture and Deployment

2.5.1 Computational Efficiency

Many state-of-the-art detection models require substantial computational resources, particularly GPU acceleration, limiting practical deployment. Lightweight architectures like EfficientNet achieve reasonable accuracy with reduced computational requirements, while edge computing approaches and knowledge distillation techniques enable deployment on resource-constrained devices. These efficiency-focused approaches often involve trade-offs between detection capability and resource consumption.

2.5.2 Web Application Integration

Most academic research focuses on algorithmic performance rather than system design and user experience. Commercial implementations demonstrate the feasibility of web-based detection but provide limited technical details about their architecture. The integration gap between research prototypes and production systems represents a significant opportunity for contributions in scalable backend design, responsive frontend interfaces, and secure data handling.

2.6 Research Gaps and Opportunities

Current research reveals several significant gaps: limited work on end-to-end systems integrating algorithms with practical interfaces; insufficient attention to comprehensive explainability; computational efficiency challenges for widespread deployment; and limited adaptability to rapidly evolving deepfake techniques. These gaps present clear opportunities for developing integrated systems that bridge algorithmic research with practical application, create transparent detection mechanisms, optimize for accessible deployment, and establish frameworks for continuous adaptation.

2.7 Chapter Summary

This chapter reviewed deepfake detection research, covering detection techniques, explainability, training methods, and deployment challenges. It identified key gaps in transparency, integration, and practical usability. Our project addresses these challenges by developing an accurate, explainable, and deployable deepfake detection system for media authenticity verification.

CHAPTER 3 METHODOLOGY

Our Deepfake Detection system follows a structured development approach grounded in a Waterfall methodology, combining sequential research phases with systematic implementation strategies to ensure both technical robustness and practical usability. Our methodology integrates established software engineering practices with specialized AI development workflows to create a comprehensive deepfake detection system that bridges academic research with real-world application.

We employ the Waterfall model organized into distinct, sequential phases, each with specific objectives and deliverables that must be completed before progressing to the next phase. This approach allows for thorough documentation and validation at each stage while maintaining clear progress milestones. The methodology encompasses requirements analysis, system design, implementation, testing, and deployment considerations, ensuring each component aligns with our objectives of accuracy, transparency, and accessibility.

3.1 Methodology for Software Development

In our project, we employ the Waterfall Model, a classical and linear software development methodology that progresses through a fixed sequence of phases: Requirements, System Design, Implementation, Testing, and Deployment. Each phase is completed fully before moving to the next, ensuring comprehensive documentation, clear deliverables, and structured progress tracking throughout the project lifecycle. This methodology is well-suited to projects where requirements are well-defined at the outset and where a disciplined, phase-gated approach is necessary to meet academic and technical standards.

3.2 Selected Methodology

The Waterfall Model was implemented through five structured and sequential phases:

- **Requirements Analysis:** All functional and non-functional requirements of the deepfake detection system were gathered, analysed, and documented before any design or development began. This included identifying dataset requirements, model performance benchmarks, and interface specifications.
- **System Design:** Based on the requirements, a complete system architecture was designed, covering the AI model pipeline, backend API structure, and frontend interface layout. Design decisions were finalized and documented prior to implementation.

- **Implementation:** Development of the AI model, backend services, and frontend components was carried out according to the approved design. Each module was built in a defined sequence: AI model first, followed by backend integration, and then the frontend interface.
- **Testing & Verification:** Comprehensive testing was performed after the implementation phase was complete. This included unit testing of individual components, integration testing of the full system, and validation of model accuracy against benchmark datasets.
- **Deployment & Maintenance:** The finalized and tested system was deployed for demonstration and evaluation. Documentation was completed, and any post-deployment issues were addressed in a structured maintenance phase.

This sequential structure ensured that the AI model, backend API, and frontend interface were developed in alignment with fully specified requirements and designs, reducing integration risks and maintaining clear traceability from requirements to final implementation.

3.3 Reason for Selected Methodology

The Waterfall Model was selected for three primary reasons:

First, the Waterfall Model provides a clear and structured framework that suits the well-defined scope of a Final Year Project. Since the core requirements of a deepfake detection system including dataset selection, model architecture, backend API, and frontend interface could be identified and documented at the beginning of the project, a sequential approach was both appropriate and practical. The phase-gated structure ensured that foundational work such as requirements and system design was fully completed before development began, reducing the risk of rework.

Second, this approach supports the academic requirements of an FYP, where comprehensive documentation is as important as the working system. The Waterfall Model mandates thorough documentation at every phase from requirements specifications and design documents to test plans and deployment reports which directly aligns with the reporting and submission expectations of an academic project with fixed deadlines and defined deliverables.

Third, the Waterfall Model promotes systematic integration of the AI model, backend services, and frontend interface by ensuring each layer is fully designed and tested before the next depends on it. This reduces technical debt and integration failures that can arise

when complex subsystems are developed simultaneously without a stable foundation. The linear progression also provides clear traceability between requirements, design decisions, and the final implemented system, which is essential for academic evaluation and verification.

3.4 Chapter Summary

This methodology chapter outlines our structured approach to developing the deepfake detection system using the Waterfall Model, progressing sequentially through Requirements Analysis, System Design, Implementation, Testing, and Deployment. The selected methodology supports both the academic demands of comprehensive documentation and the technical requirements of building a reliable AI-powered system. By following a phase-gated linear process, the project maintained clear progress milestones, ensured thorough validation at each stage, and delivered a complete and well-documented system aligned with the defined project objectives.

CHAPTER 4 SYSTEM REQUIREMENTS

The proposed project introduces an AI-based deepfake detection system designed to identify manipulated video content using multimodal analysis techniques. The system aims to assist users in verifying the authenticity of videos through an automated, accurate, and user-friendly platform. By integrating deep learning models with a web-based interface, the application bridges the gap between complex AI technology and real-world usability.

4.1 Functional Requirements

The functional requirements of our project deepfake detection system define the main services and features that the system must provide. These requirements ensure that the application effectively achieves its objective of accurately analysing video content and identifying potential deepfake manipulations through an automated and user-friendly platform.

Table 4.1 and **Table 4.2** detail the functional requirements for different Users, and the System itself, emphasizing automation, customization, and data security.

Table 4.1: User Requirements

Requirement Number	Name
FR-01	Secure authentication ensures only authorized access.
FR-02	Account creation captures all necessary user details.
FR-03	Users are allowed to upload videos one by one.
FR-04	Video can be analysed frame by frame.
FR-05	The result should be proper and gives the whole information.
FR-06	Admin can access all the records of users.
FR-07	Admin can delete a user and can access the results of the videos.

Table 4.2: System Requirements

Requirement Number	Name
FR-01	Ensures secure user authentication and blocks unauthorized access.
FR-02	Dynamically displays the result when the analyse will complete.
FR-03	Guarantees results of every video.

4.2 Non-Functional Requirements

4.2.1 Security

The system must ensure good security measures to protect user data from unauthorized access, breaches, and cyberattacks. Authentication protocols such as encrypted passwords and two-factor authentication (2FA) will be implemented. Sensitive data like user profiles, generated content, and payment details must be stored using advanced encryption methods.

4.2.2 Performance

Our project deepfake detection system must maintain optimal performance to ensure timely and accurate analysis of uploaded video content. Our Web App provides fast response times for all user interactions, including authentication, video upload, and result retrieval. Video analysis operations should be completed within an acceptable time frame, even for larger files.

Also, our system must efficiently handle multiple concurrent users with minimal performance degradation. Under increased load, the application should continue to operate reliably, ensuring consistent speed, accuracy, and availability of deepfake detection services.

4.2.3 Availability

The platform must maintain high availability with a target uptime of 99.9%, always ensuring reliable access for users. Users should be able to use the service without interruptions, delays, or unexpected downtime during normal operation. To support this, proper disaster recovery mechanisms and backup strategies will be implemented to quickly restore services in case of failures. Failover systems will also be used to automatically switch to backup resources during system issues, minimizing service disruption.

4.2.4 Data Privacy

The data collected from users must comply with global privacy standards such as GDPR. User data will only be used for its intended purpose, and strict access controls will ensure that unauthorized personnel cannot access sensitive information. Users will have the ability to delete their data permanently upon request.

4.2.5 Scalability

The system must be scalable to accommodate a growing user base and increasing data storage needs. Cloud-based infrastructure will enable automatic scaling of resources, ensuring consistent performance during peak usage periods.

4.2.6 Usability

Our platform provides an intuitive, user-friendly interface, ensuring that users with minimal technical knowledge can upload multiple videos and get result regarding their requests. Clear navigation, concise instructions, and accessible design standards will enhance the overall experience.

4.2.7 Maintainability

Our project is to design with a modular and well-structured codebase to support ease of maintenance and future enhancements. Clear documentation and adherence to coding standards will ensure that bug fixes, performance improvements, and feature updates can be implemented efficiently without affecting existing system functionality. This approach enables long-term sustainability and simplifies ongoing development and maintenance efforts.

4.2.8 Reliability

Our system must function correctly under all defined conditions. This system should handle errors gracefully, provide meaningful feedback to users during failures, and always ensure data consistency.

4.2.9 Compliance

Our Project must comply with relevant legal, ethical, and accessibility standards, including ADA for web accessibility. It should also meet industry best practices for software development and deployment.

4.3 Use Case Diagrams

Use cases describe the specific interactions between users and this system to accomplish key tasks. They outline scenarios such as a user uploading a video for analysis, the system processing the content to detect manipulation, and the user viewing detailed results with confidence scores and visual explanations.

4.3.1 Use Case for System

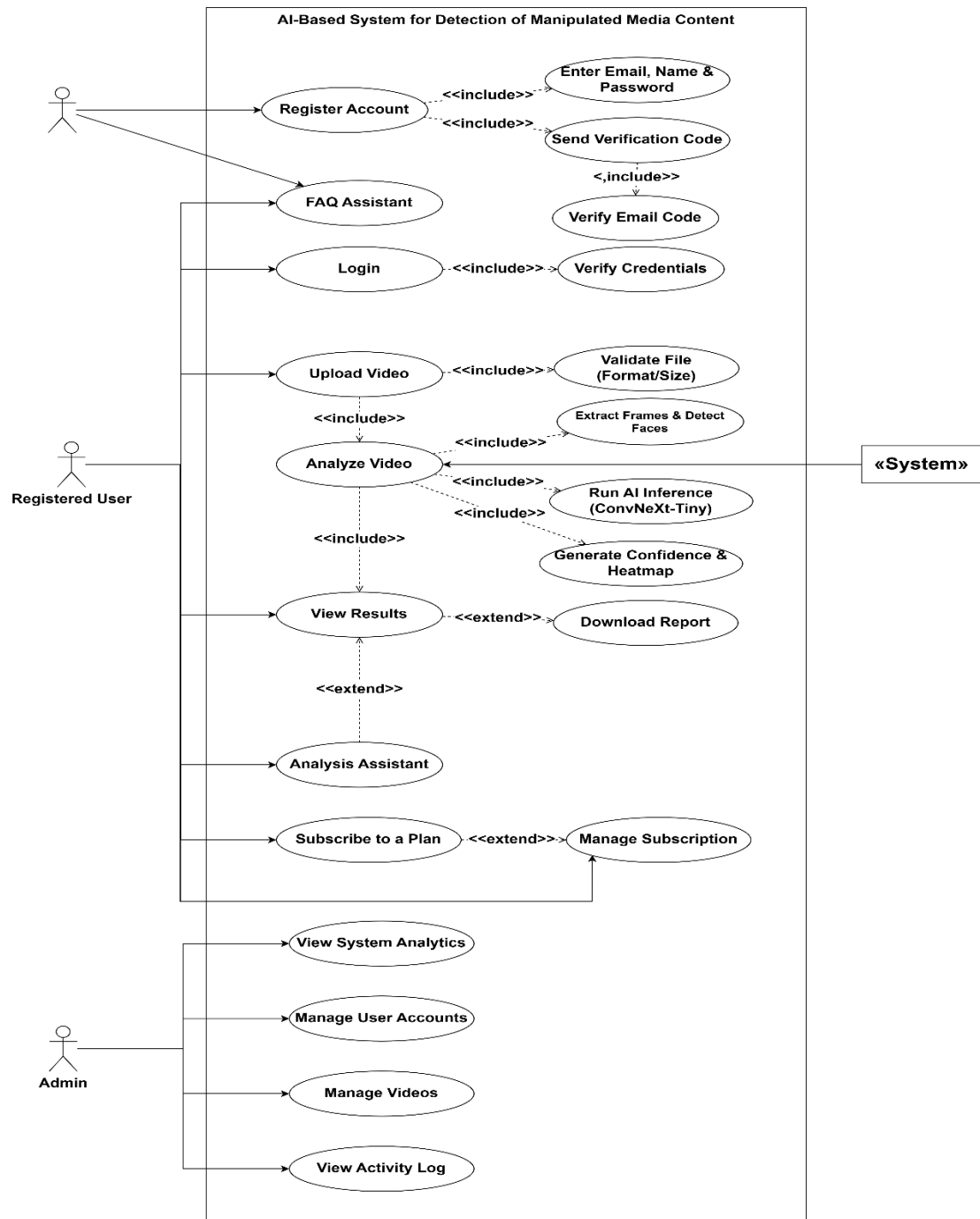


Figure 4.1 Use Case Diagram for whole system

4.3.2 Use Case for User Registration

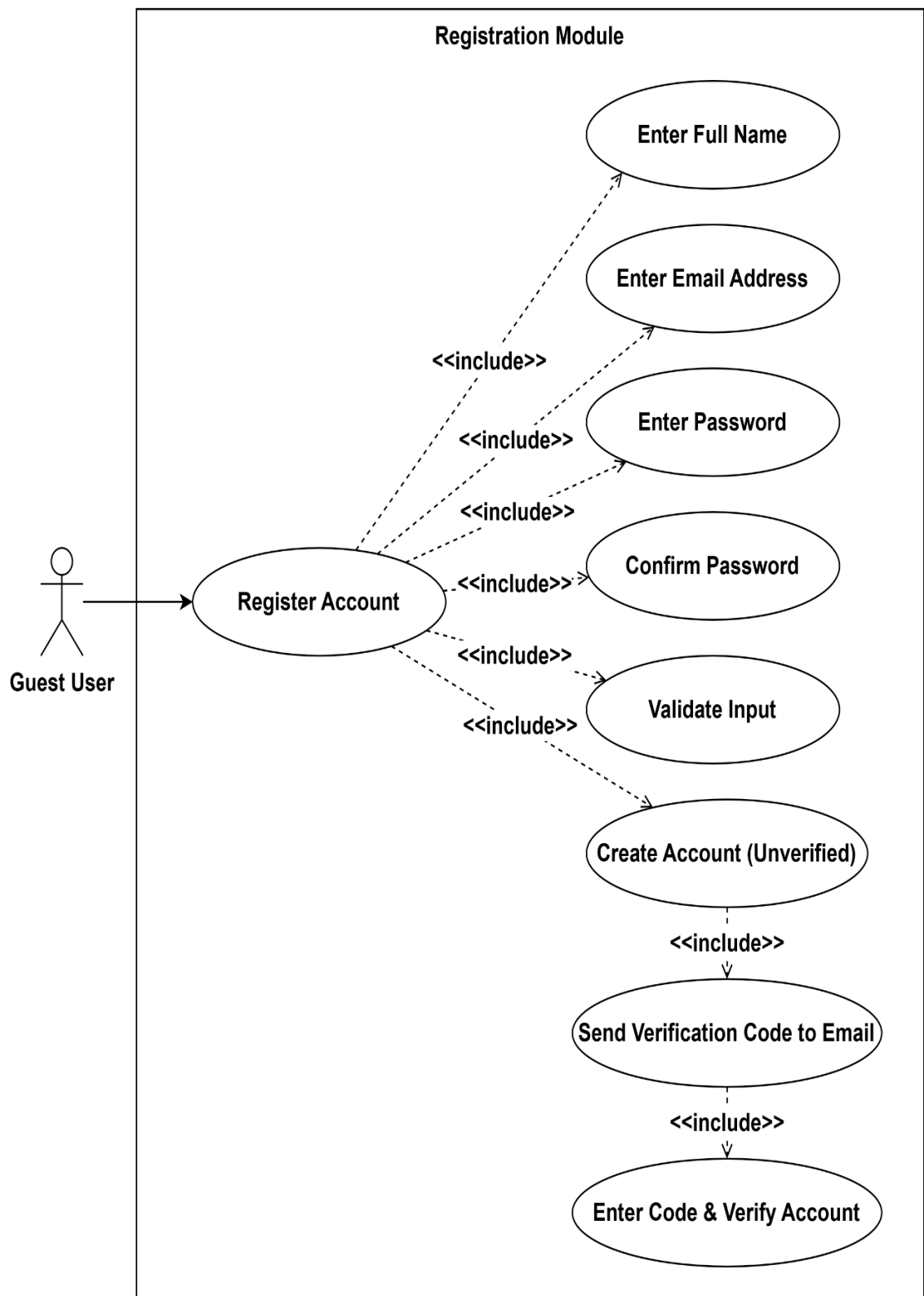


Figure 4.2 Use Case for Registration

4.3.3 Use Case for User Login

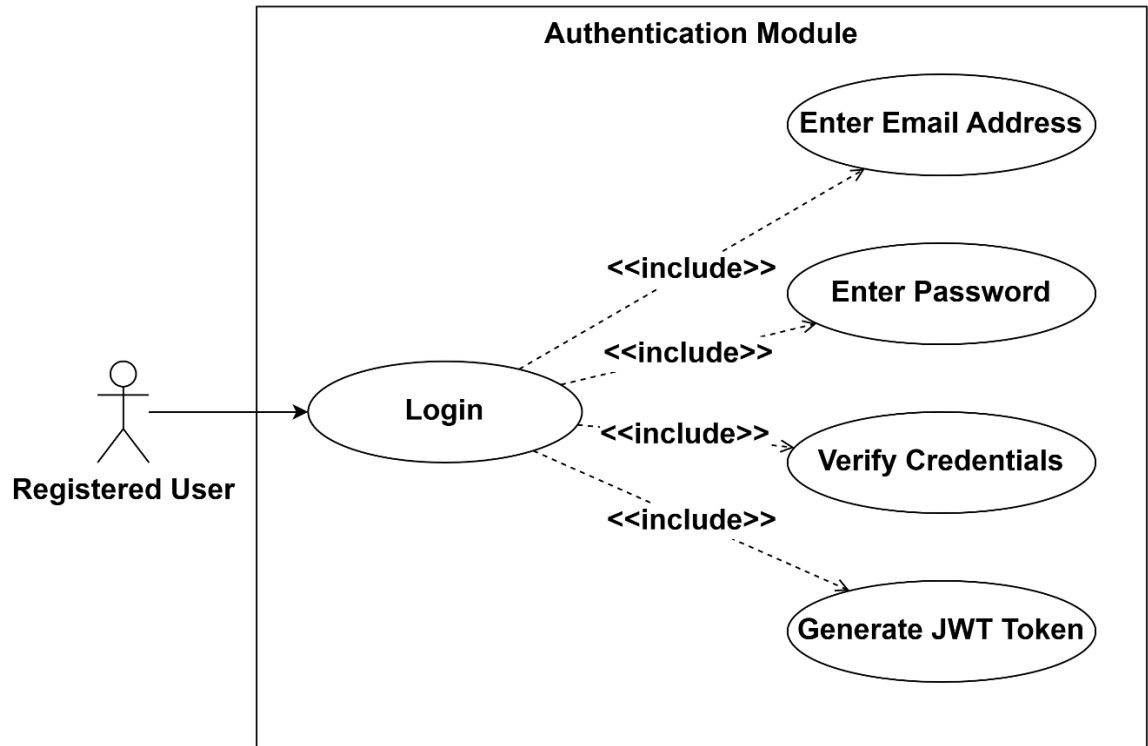


Figure 4.3 Use Case for Login

4.3.4 Use Case for Video Upload

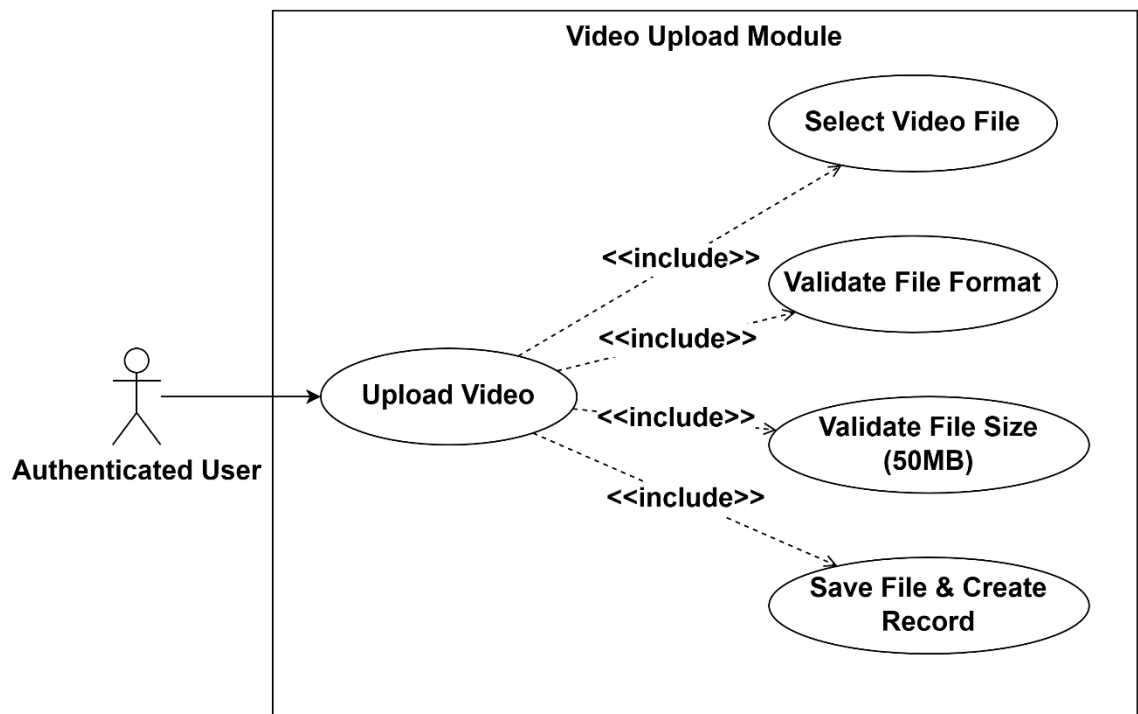


Figure 4.4 Use Case for Upload video

4.3.5 Use Case for AI Video Analysis

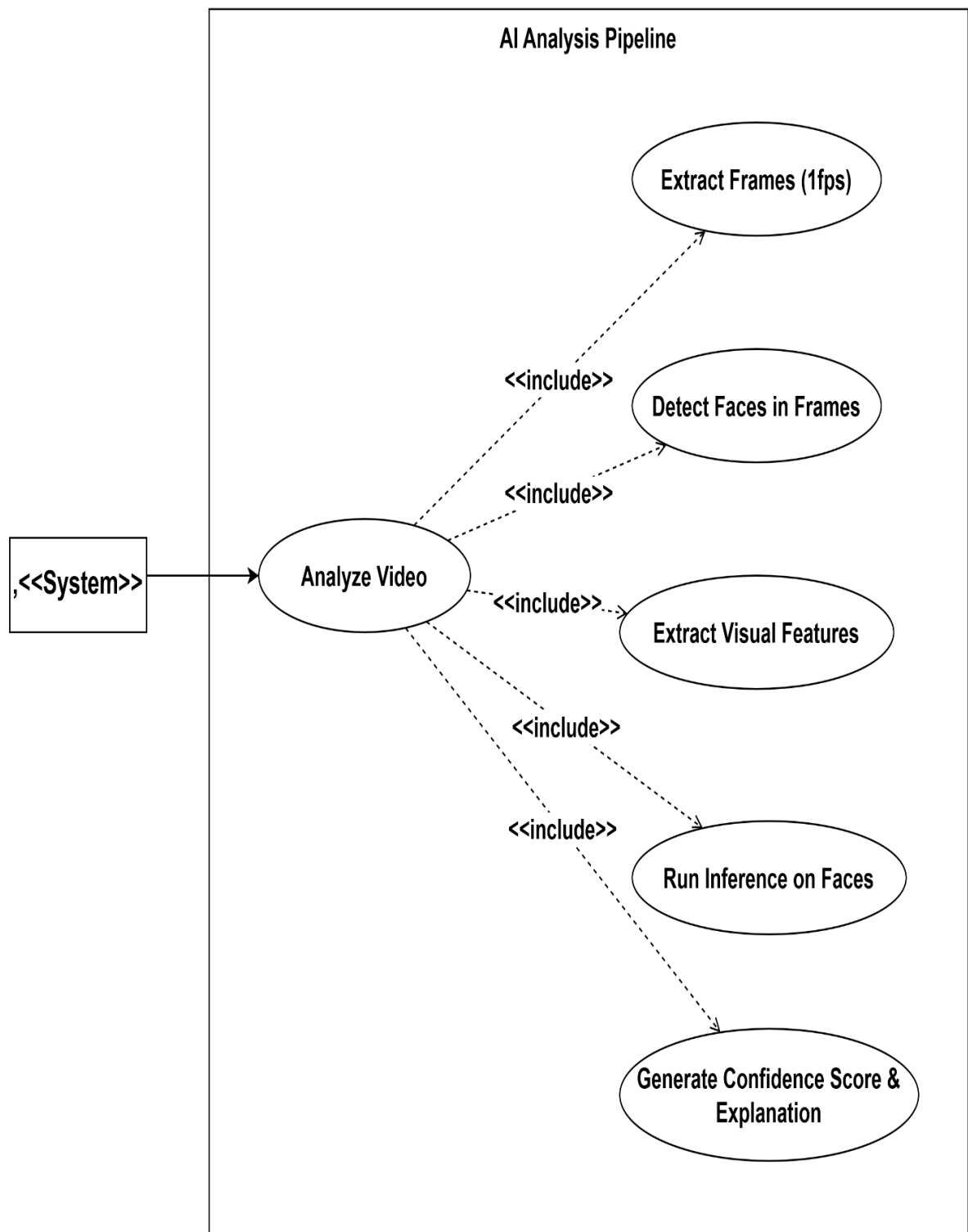


Figure 4.5 Use Case for Analyze Video

4.3.6 Use Case for View Results

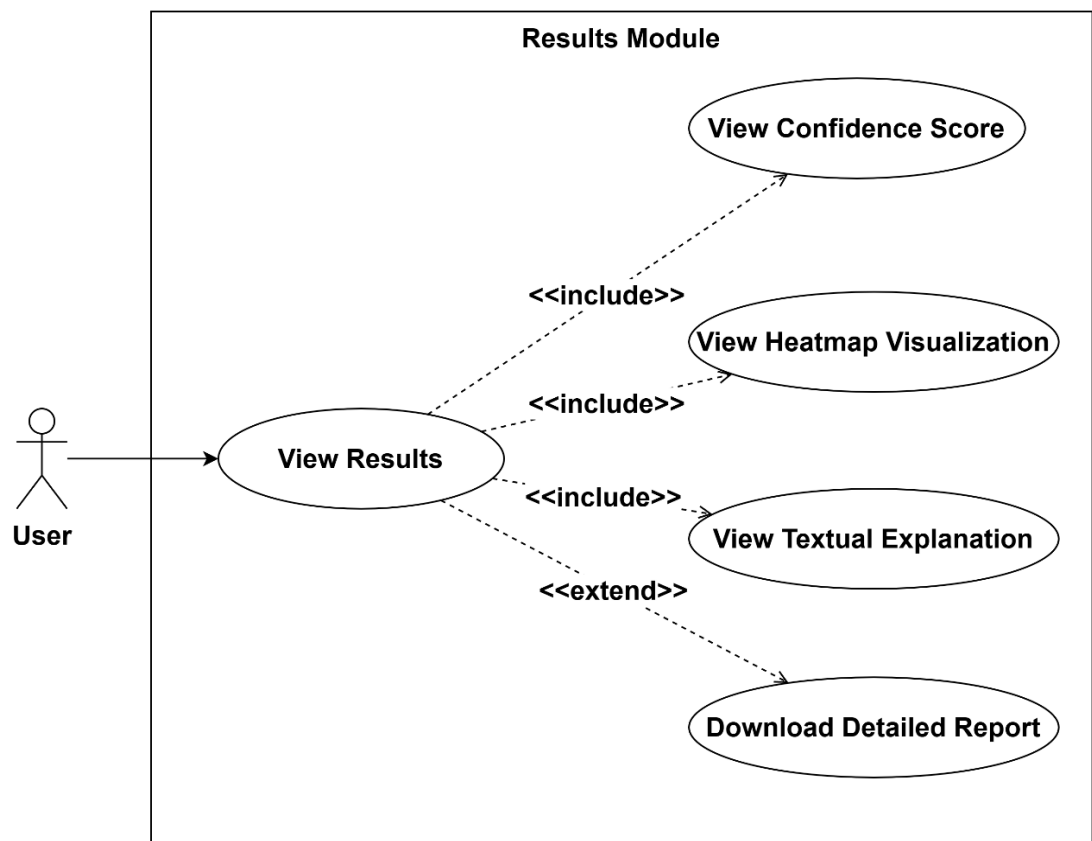


Figure 4.6 Use Case for View Results

4.3.7 Use Case for Admin Dashboard

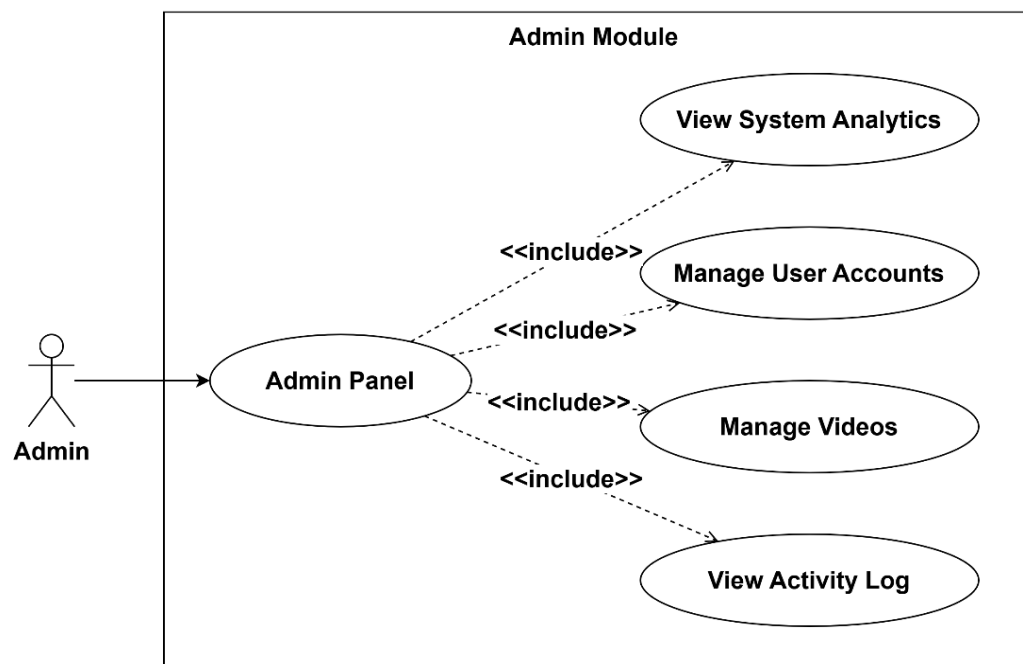


Figure 4.7 Use Case for Admin Dashboard

4.3.8 Use Case for Admin Analytics Module

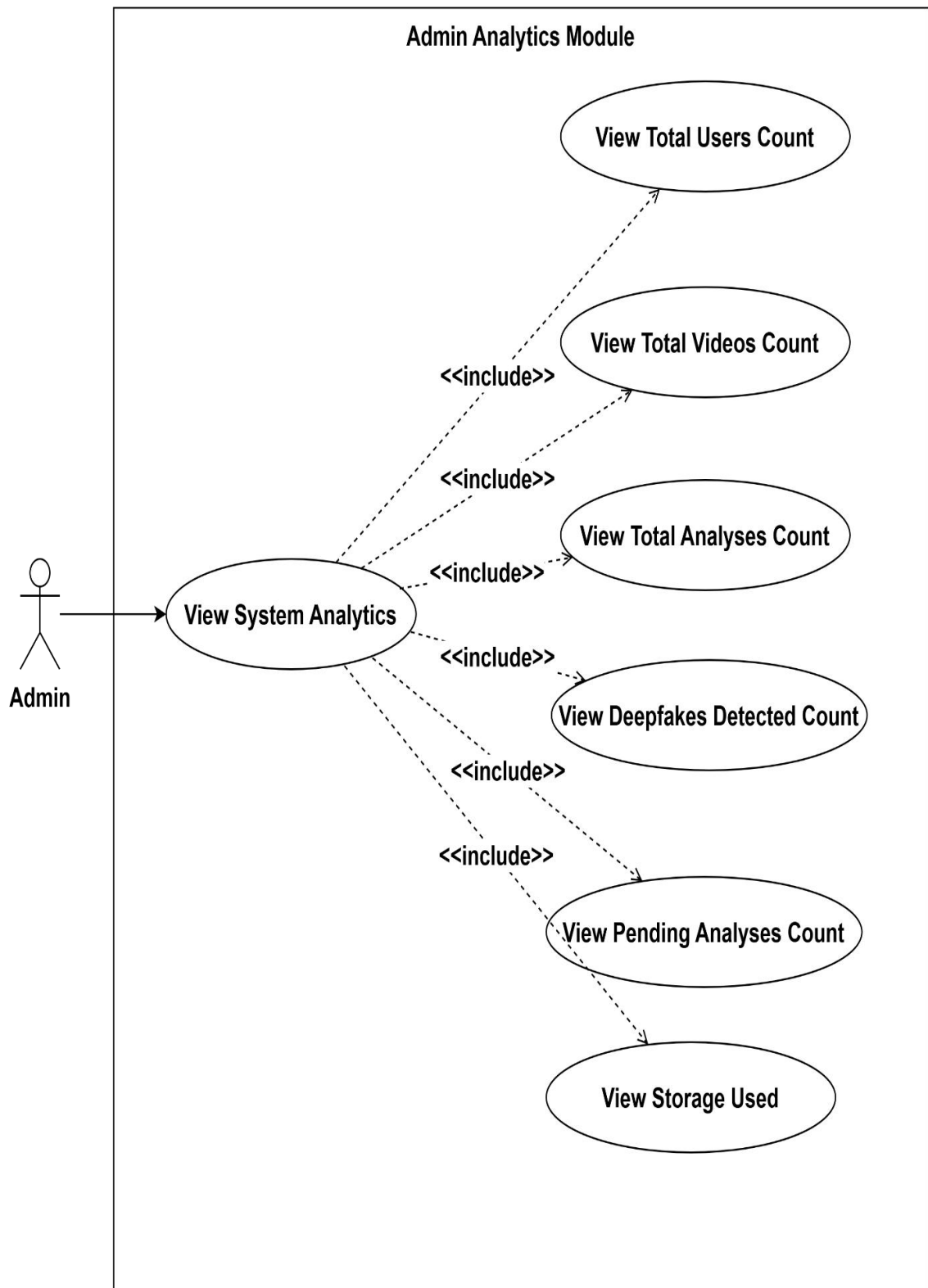


Figure 4 .7 Use Case for Admin Analytics

4.3.9 Use Case for Users Management

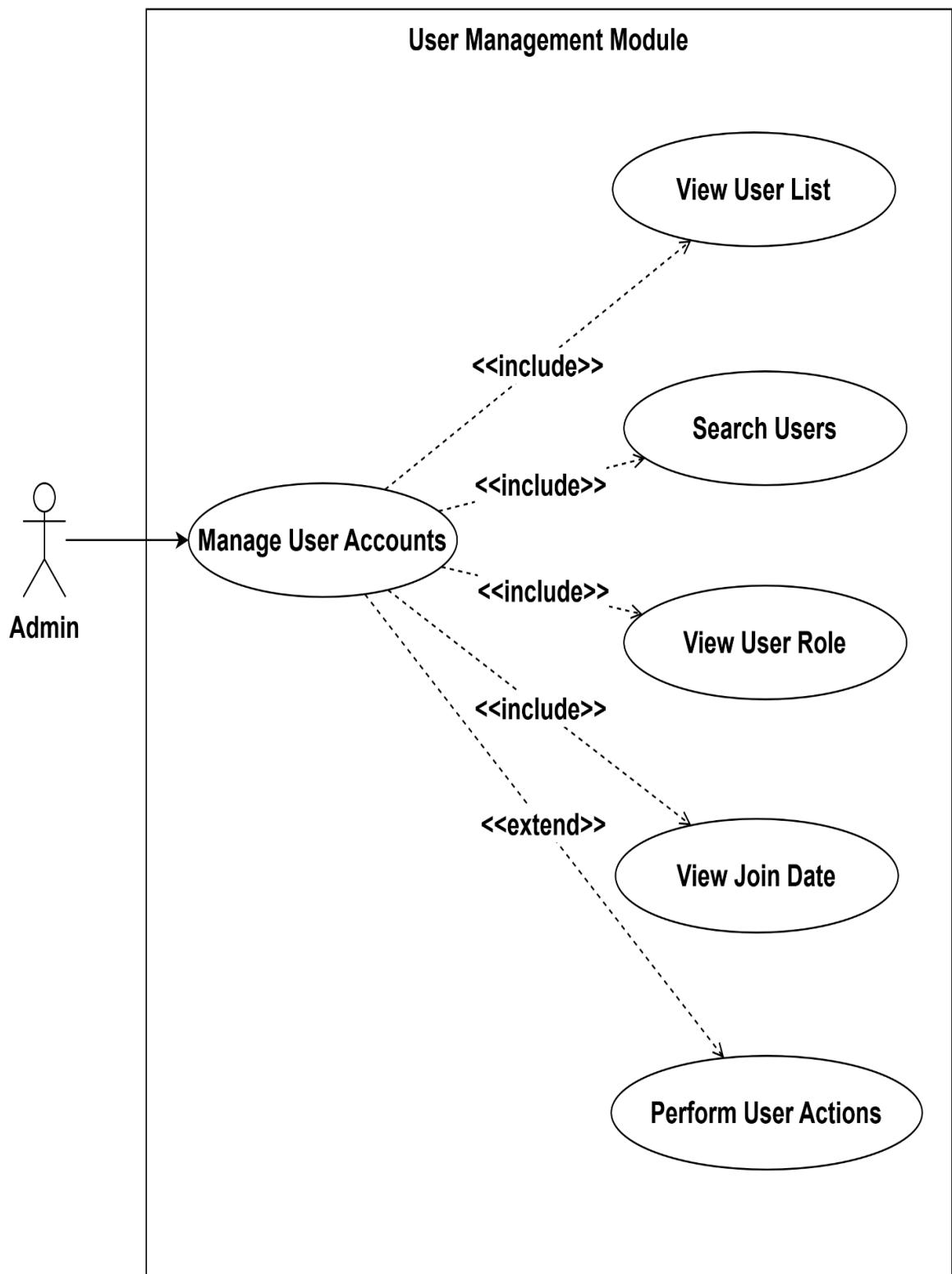


Figure 4.8 Use Case for Users Management

4.3.10 Use Case for Videos Management

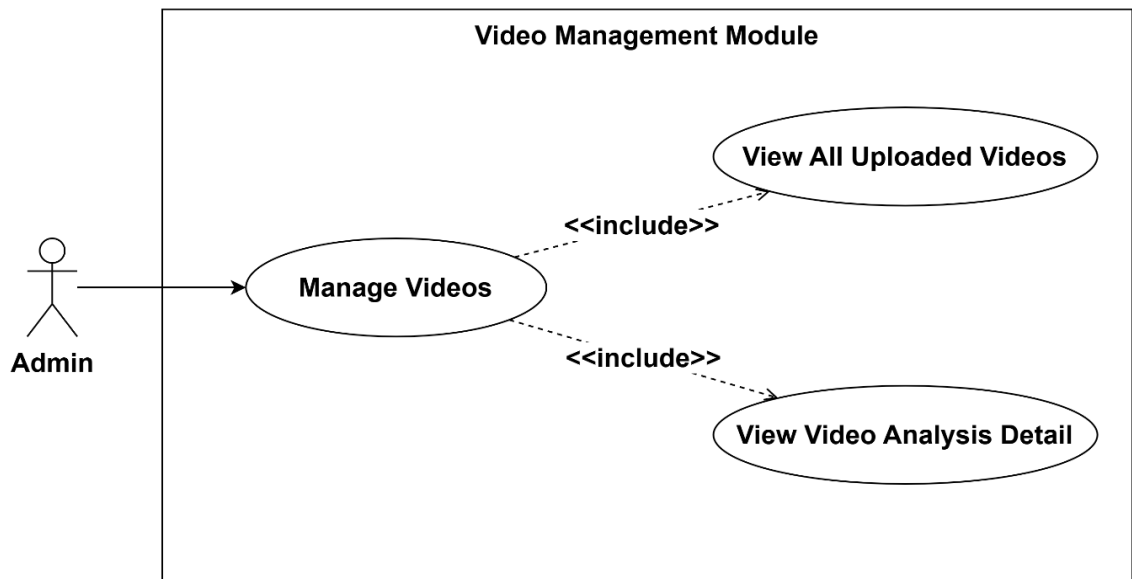


Figure 4.9 Use Case for Videos Management

4.3.11 Use Case for Activity Monitoring

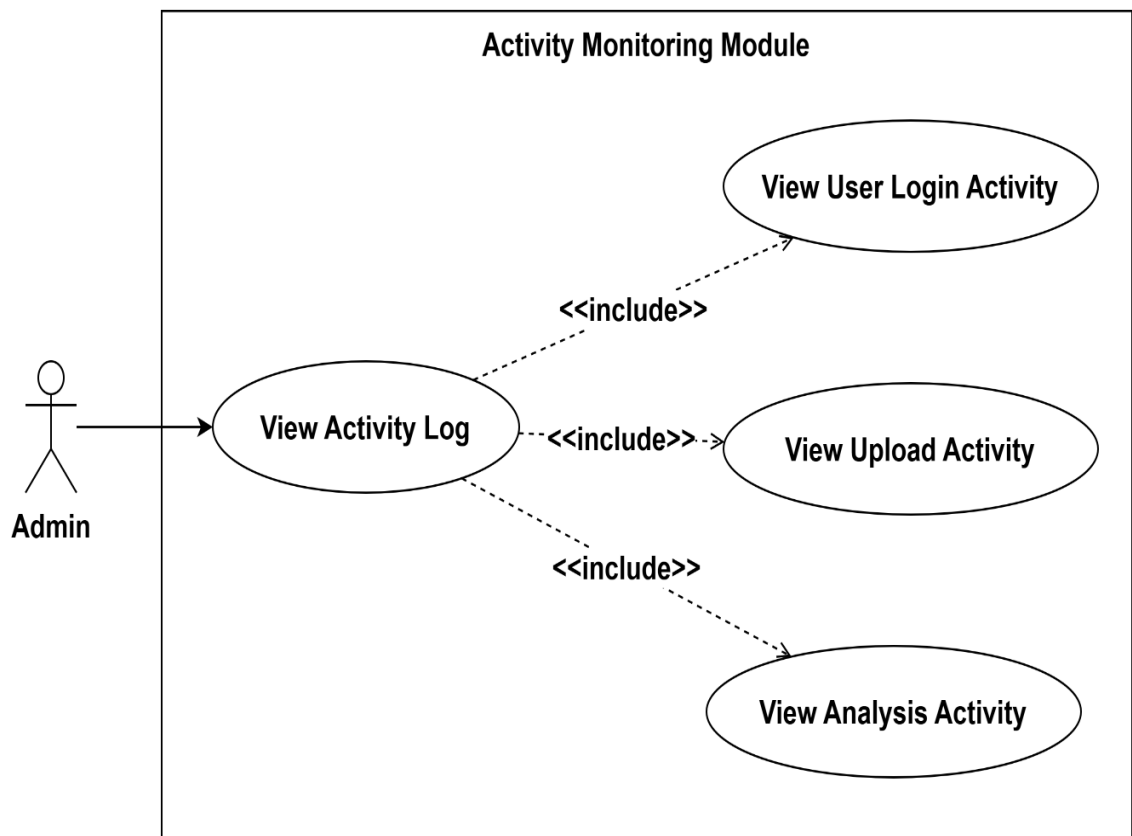


Figure 4.10 Use Case for Activity Monitoring

4.3.12 FAQ Assistant Use Case

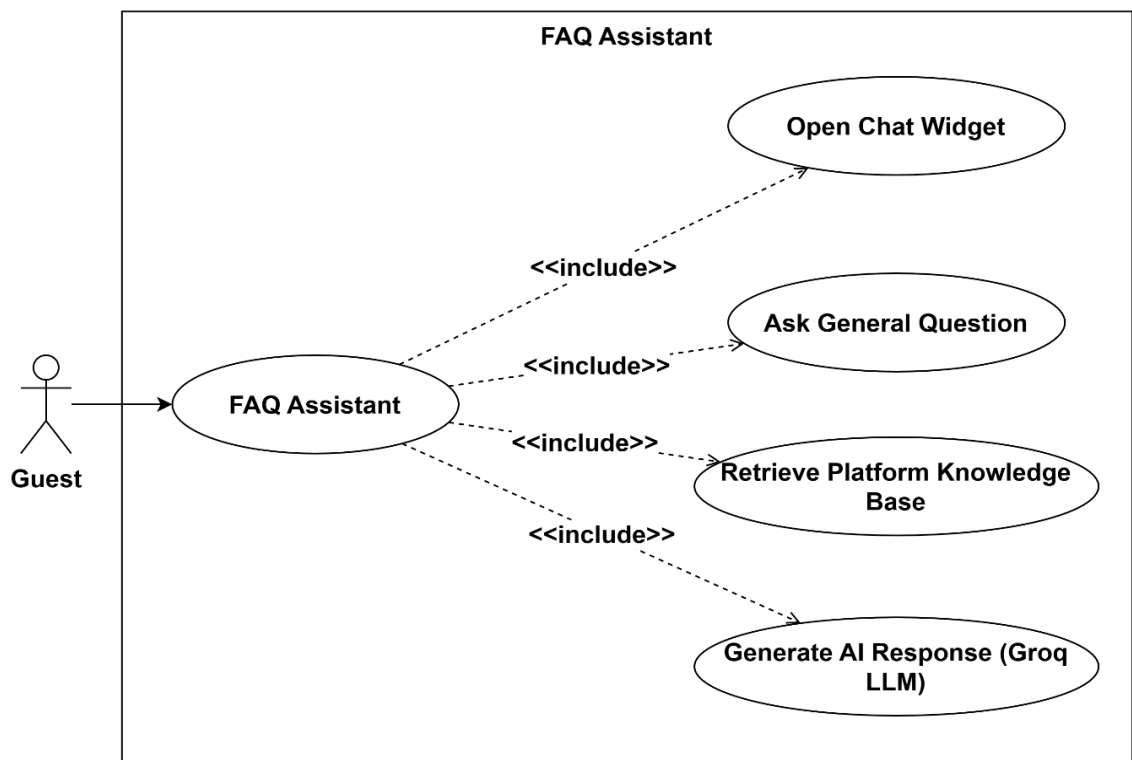


Figure 4.11: FAQ Assistant Use Case

4.3.13 Analysis Assistant Use Case

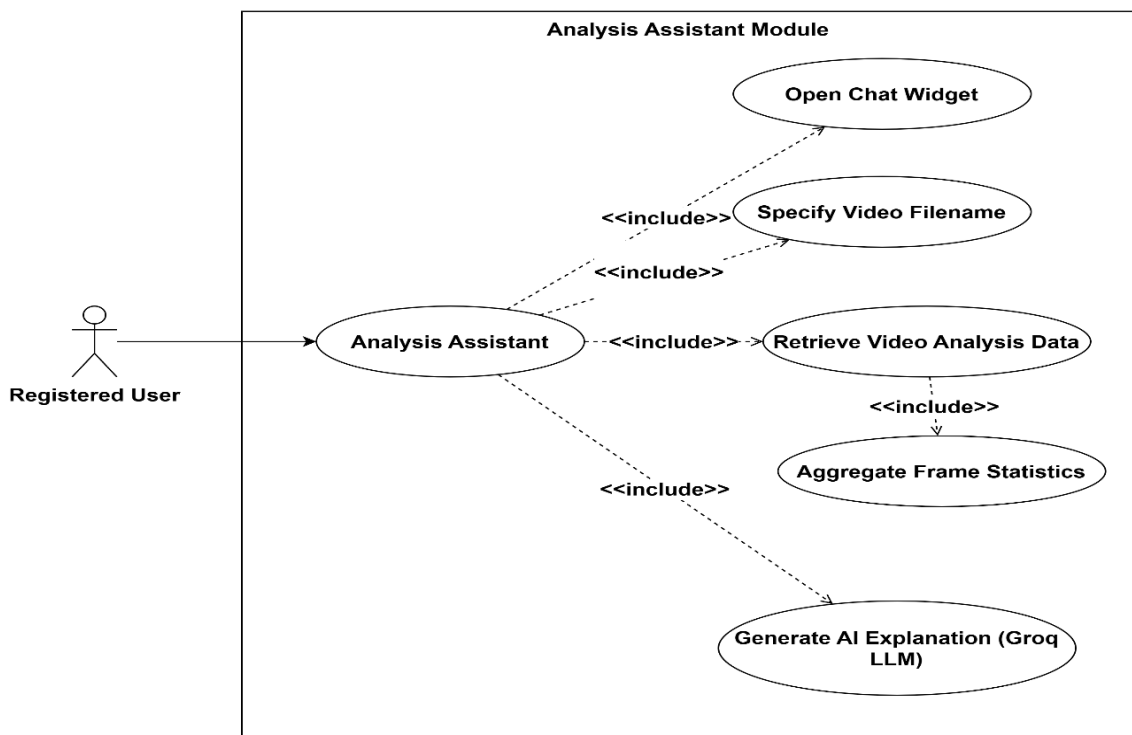


Figure 4.12: Analysis Assistant Use Case

4.3.14 Subscribe to a Plan

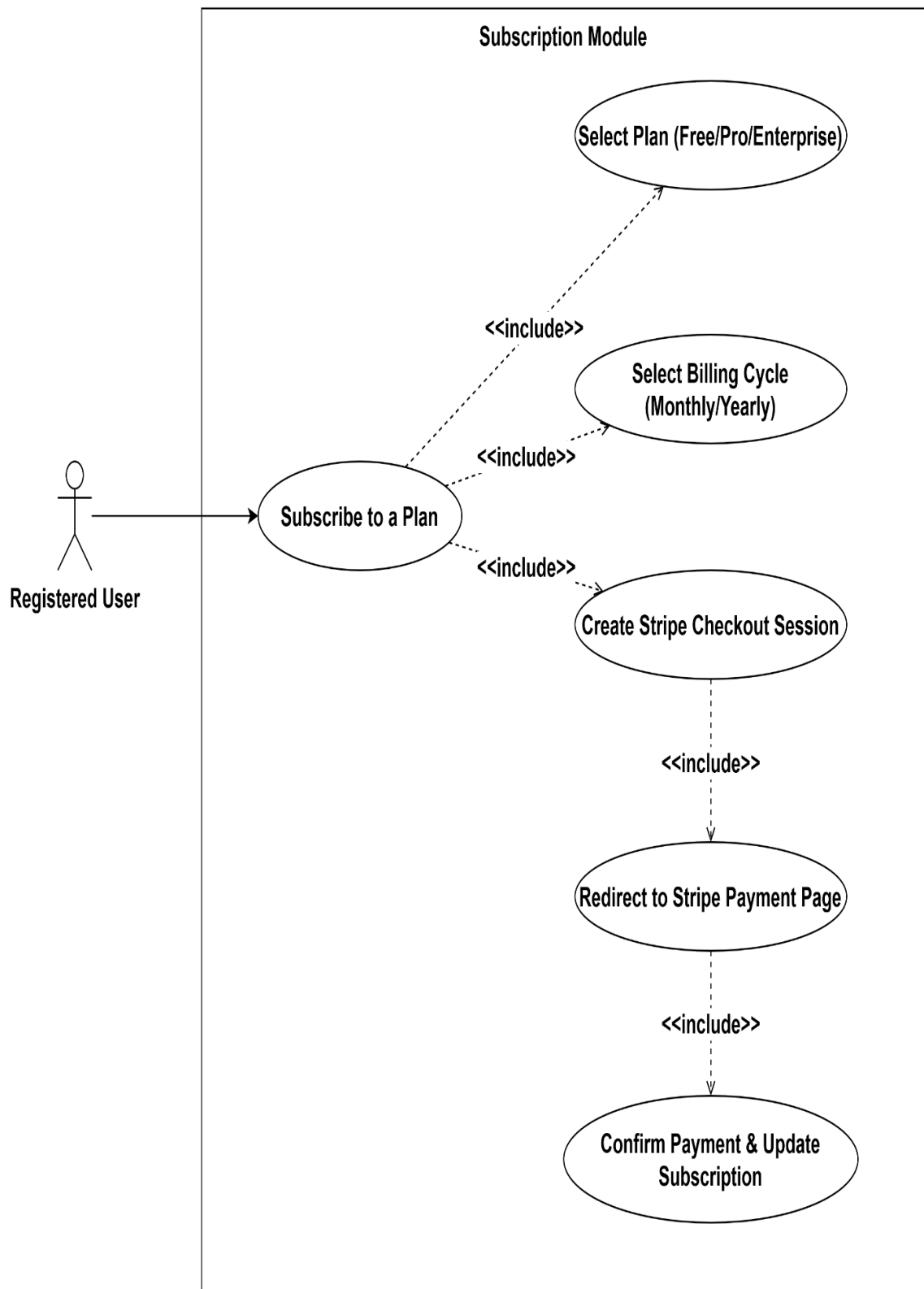


Figure 4.13: Subscribe to a Plan

4.3.15 Manage Subscription

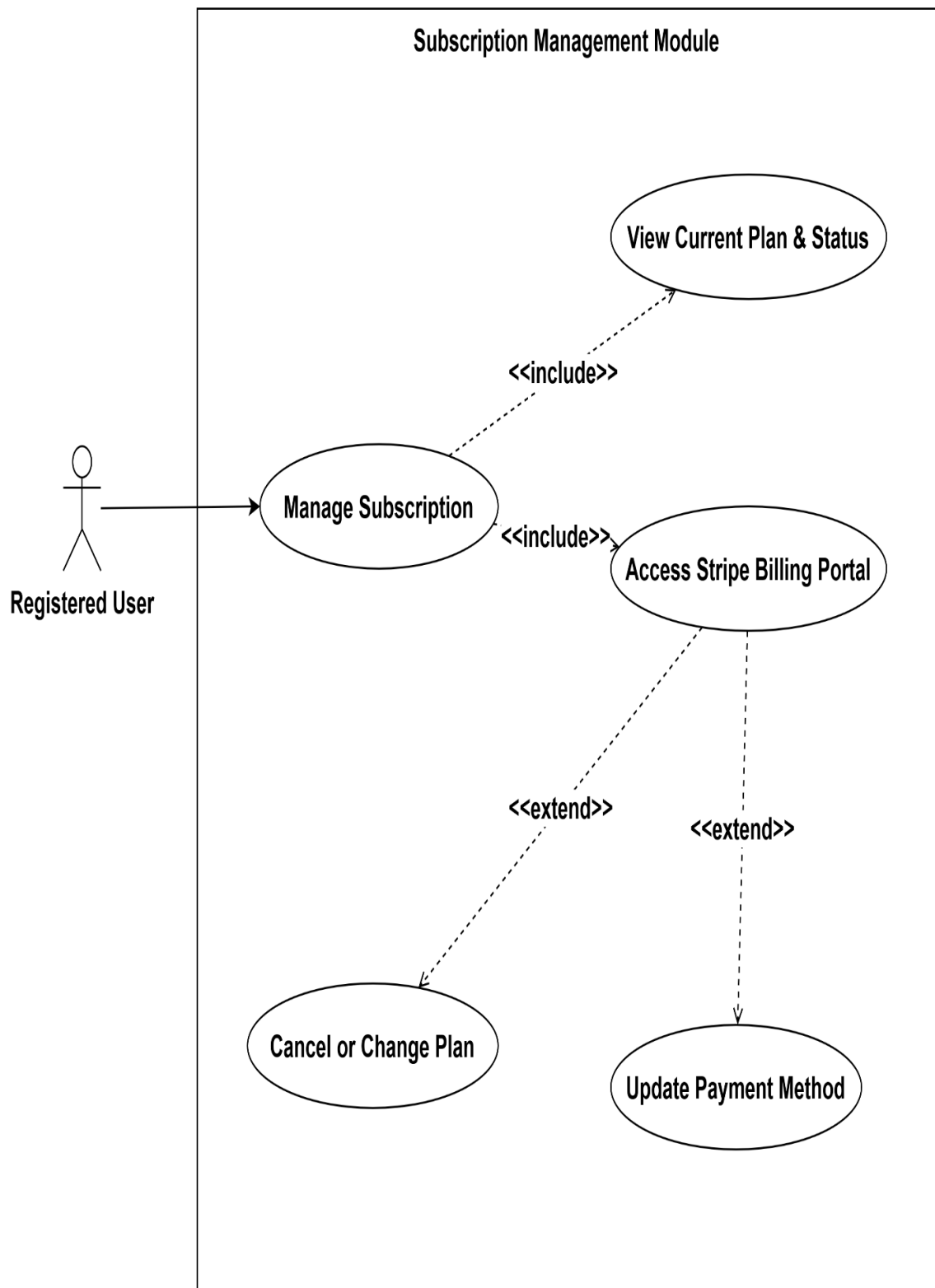


Figure 4.14: Manage Subscription

4.4 Use Case Descriptions

4.4.1 Use Case for System

The system is a web-based AI platform designed to detect manipulated or deepfake video content. It allows users to register, log in, and upload videos for automated analysis. The system processes videos using AI models to generate classification results, confidence scores, and visual heatmaps. It also includes AI assistants, subscription management, and an admin panel for monitoring and system control. This shown in **Table 4.3**.

Table 4.3 Use case for System

Field	Description
Use Case ID	UC-01
Use Case Name	AI-Based System for Detection of Manipulated Media Content
Actors	Guest, Registered User, Admin, System
Description	A web-based platform for detecting manipulated media using AI. Users can register, log in, upload videos, view analysis results, interact with AI assistants, and manage subscriptions. Admins can manage users, videos, analytics, and activity logs.
Preconditions	User has internet access and a valid email address. Admin has valid credentials.
Postconditions	Users receive analysis results and can manage subscriptions. Admins can monitor and manage the platform.
Main Flow	1. User visits the platform. 2. Registers and verifies account. 3. Logs in. 4. Uploads a video. 5. System analyses the video using AI. 6. Results with confidence scores and heatmaps are generated. 7. User views/downloads results and may use AI assistants. 8. User manages subscriptions if needed. 9. Admin performs management tasks.

4.4.2 Use Case for User Registration

This use case enables new users to create accounts in the system shown in **Table 4.4**. A guest user provides email, username, and password details which are validated for format and uniqueness. Upon successful validation, the system creates a user account with default USER privileges and sends a confirmation, allowing the user to subsequently access system features.

Table 4.4 Use case for user Registration

Field	Description
Use Case ID	UC-02
Use Case Name	User Registration
Actor	Guest User
Description	Allows a new user to create an account in the system
Precondition	User is not logged in and has a valid email address
Postcondition	User account is created and user can log in
Main Flow	1. User navigates to registration page. 2. User enters email, username, and password. 3. System validates input. 4. System creates account with default USER role. 5. User receives confirmation.

4.4.3 Use Case for User Login

Registered users authenticate to access the system by providing their credentials. The system verifies the email and password combination, checks account activation status, and upon successful verification generates a secure JWT token. **Table 4.5** explains user login use case.

Table 4.5 Use Case for Login

Field	Description
Use Case ID	UC-03
Use Case Name	User Login
Actor	Registered User
Description	Authenticates a user to access the system
Precondition	A valid user account exists
Postcondition	User is authenticated and session is created
Main Flow	1. User enters email and password. 2. System verifies credentials 3. System generates JWT token 4. User is redirected to dashboard

4.4.4 Use Case for Video Upload

Authenticated users upload video files for deepfake analysis. The system validates file parameters including format (MP4), size (maximum 50MB), and integrity. Valid files receive unique identifiers and are stored temporarily while database records track upload status. **Table 4.6** shows use case for video upload.

Table 4.6 Use Case for Video Upload

Field	Description
Use Case ID	UC-04
Use Case Name	Upload Video
Actor	Authenticated User
Description	Allows user to upload a video for deepfake analysis
Precondition	User is logged in and video file is valid
Postcondition	Video is stored and ready for analysis
Main Flow	1. User selects video file (max 50MB). 2. System validates file format (MP4, AVI, MOV). 3. System generates unique filename. 4. Video is saved to temporary storage5. Database records created.

4.4.5 Use Case for Video Analysis

The system automatically processes uploaded videos through a multi-stage AI pipeline shows in **Table 4.7**. This includes frame extraction, face detection using computer vision techniques, visual feature analysis through deep learning models, and aggregated prediction scoring.

Table 4.7 Use Case for Video Analysis

Field	Description
Use Case ID	UC-05
Use Case Name	Analyse Video
Actor	System
Description	Processes uploaded video to detect deepfake content
Precondition	Video has been uploaded successfully
Postcondition	Analysis results are generated
Main Flow	1. Extract frames from video (1 fps) 2. Detect faces in each frame 3. Extract visual features 4. Run inference on each face 5. Aggregate predictions 6. Generate confidence score and explanation

4.4.6 Use Case for View Results

Users access comprehensive analysis reports displaying detection outcomes explains in **Table 4.8**. The interface presents confidence scores (0-100%), visual heatmaps highlighting suspicious regions, textual explanations of findings, and options to download detailed reports.

Table 4.8 Use Case for View Results

Field	Description
Use Case ID	UC-06
Use Case Name	View Results
Actor	User
Description	Displays analysis results of the uploaded video
Precondition	Video analysis is completed
Postcondition	User is informed about video authenticity
Main Flow	1. User navigates to results page. 2. System displays confidence score (0–100%) 3. System shows heatmap visualization. 4. System provides textual explanation 5. User can download detailed report

4.4.7 Use Case for Admin Dashboard

Administrative users manage system operations, including monitoring activity, overseeing video analysis, managing user accounts, reviewing statistics, and retraining models to ensure accuracy all explains in **Table 4.9**.

Table 4.9 Use Case for Admin Dashboard

Field	Description
Use Case ID	UC-07
Use Case Name	Admin Dashboard
Actor	Admin
Description	Enables admin to manage users, videos, and system operations
Precondition	Admin role and valid login
Postcondition	System state is updated
Main Flow	1. Admin views system statistics 2. Admin manages user accounts 3. Admin monitors video analyses 4. Admin can view activity log.

4.4.8 Use Case for Admin Analytics Module

This use case enables administrators to monitor system performance through a dashboard that displays user activity, uploads, and storage usage, as detailed in **Table 4.10**.

Table 4.10 Use Case for Admin Analytics

Field	Description
Use Case ID	UC-08
Use Case Name	View System Analytics
Actor	Admin
Description	Allows admin to view system-wide statistics including total users, videos, analyses, and storage
Precondition	Admin is logged in with valid admin role
Postcondition	Admin views current system metrics
Main Flow	1. Admin navigates to Admin Panel 2. System displays Total Users, Videos and Analysis count 3. System displays Deepfakes Detected and Pending Analysis count. 4. System displays Storage Used

4.4.9 Use Case for Users Management

This use case enables administrators to manage registered users, including searching, viewing details, and accessing profiles. The detailed description is provided in **Table 4.11**.

Table 4.11 Use Case for Users Management

Field	Description
Use Case ID	UC-09
Use Case Name	Manage User Accounts
Actor	Admin
Description	Allows admin to view, search, and manage registered user accounts
Precondition	Admin is logged in with valid admin role
Postcondition	User account list is displayed or updated
Main Flow	1. Admin navigates to Users tab 2. System displays user list with email, role, and join date 3. Admin searches users 4. Admin selects an action from the menu (e.g., view details)

4.4.10 Use Case for Videos Management

This use case enables administrators to review user-uploaded videos, including detection results and analysis details. The detailed description is provided in **Table 4.12**.

Table 4.12 Videos Management

Field	Description
Use Case ID	UC-10
Use Case Name	Manage Videos
Actor	Admin
Description	Allows admin to view all uploaded videos and their analysis results across the platform
Precondition	Admin is logged in with valid admin role
Postcondition	Video list and analysis statuses are displayed
Main Flow	1. Admin navigates to Videos tab 2. System displays all uploaded videos 3. Admin filters by detection result (real/fake) 4. Admin views individual analysis details

4.4.11 Use Case for View Activity Log

This use case allows administrators to monitor recent platform activities, including user logins, video uploads, and analysis operations. Detailed use case is presented in **Table 4.13**.

Table 4.13 Use Case for View Activity Log

Field	Description
Use Case ID	UC-11
Use Case Name	View Activity Log
Actor	Admin
Description	Allows admin to monitor recent platform activity including logins, uploads, and analyses
Precondition	Admin is logged in with valid admin role
Postcondition	Recent activity is displayed
Main Flow	1. Admin navigates to Activity tab. 2. System displays recent user login activity. 3. System displays recent upload activity. 4. System displays recent analysis activity

4.4.12 Use Case for View FAQ Assistant

The FAQ Assistant allows visitors to ask questions about the platform and receive instant responses. This use case explained in **Table 4.14**.

Table 4.14 Use Case for FAQ Assistant

Field	Description
Use Case ID	UC-12
Use Case Name	FAQ Assistant
Actor	Guest / All Users
Description	Provides a chatbot on the landing page that answers general questions about the platform's features, technology, and usage.
Precondition	User is on the landing page.
Postcondition	User receives an informational response about the platform.
Main Flow	User opens the chat widget and enters a question. System sends the query with platform context to the LLM. System displays the AI-generated response.

4.4.13 Use Case for View Analysis Assistant

This use case enables authenticated users to interact with an AI chatbot for explanations of video analysis results. The detailed use case is presented in **Table 4.15**.

Table 4.15 Use Case for FAQ Assistant

Field	Description
Use Case ID	UC-13
Use Case Name	Analysis Assistant
Actor	Registered User
Description	Allows authenticated users to query and receive natural-language explanations of their video analysis results via a chatbot.
Precondition	User is logged in and has at least one analysed video.
Postcondition	User receives an explanation of their video's analysis results.
Main Flow	1. User clicks the chat widget on the dashboard. 2. User asks a question, optionally naming a video file. 3. System retrieves the video's analysis data from the database. 4. System sends this data with the question to the LLM. 5. System displays the AI-generated explanation.

4.4.14 Use Case for Subscribe to a Plan

This use case enables users to upgrade their account through a subscription plan. The detailed use case is presented in **Table 4.16**.

Table 4.16 Use Case for Subscribe to a Plan

Field	Description
Use Case ID	UC-14
Use Case Name	Subscribe to a Plan
Actor	Registered User
Description	Allows users to select and purchase a subscription plan (Free, Pro, or Enterprise) with monthly or yearly billing.
Precondition	User has an account; if not logged in, user is redirected to login before checkout.
Postcondition	User subscription is created or updated after successful payment.
Main Flow	1. User selects a plan and billing cycle. 2. System checks login status and redirects if needed. 3. System creates a Stripe checkout session. 4. User completes payment via Stripe. 5. System updates subscription in the database.

4.4.15 Use Case for Manage Subscription

This use case allows users to manage subscription details. Use case explained in **Table 4.17**.

Table 4.17 Use Case for Manage Subscriptions

Field	Description
Use Case ID	UC-15
Use Case Name	Manage Subscription
Actor	Registered User
Description	Allows users to view and manage their subscription details through Stripe's customer portal.
Precondition	User is logged in and has an active or previous subscription.
Postcondition	Subscription, billing, or cancellation details updated accordingly.
Main Flow	1. User opens account settings. 2. System shows subscription details. 3. User selects "Manage Subscription". 4. System redirects to Stripe portal. 5. User updates or cancels subscription.

4.5 Activity Diagrams

The system's flow from one activity to another is depicted in the activity diagram.

4.5.1 User Registration & Authentication Activity:

Figure 4.15 illustrates the user authentication workflow, including account registration, email verification, and login using email and password. Upon successful authentication, the system generates a JWT token and redirects the user to the dashboard.

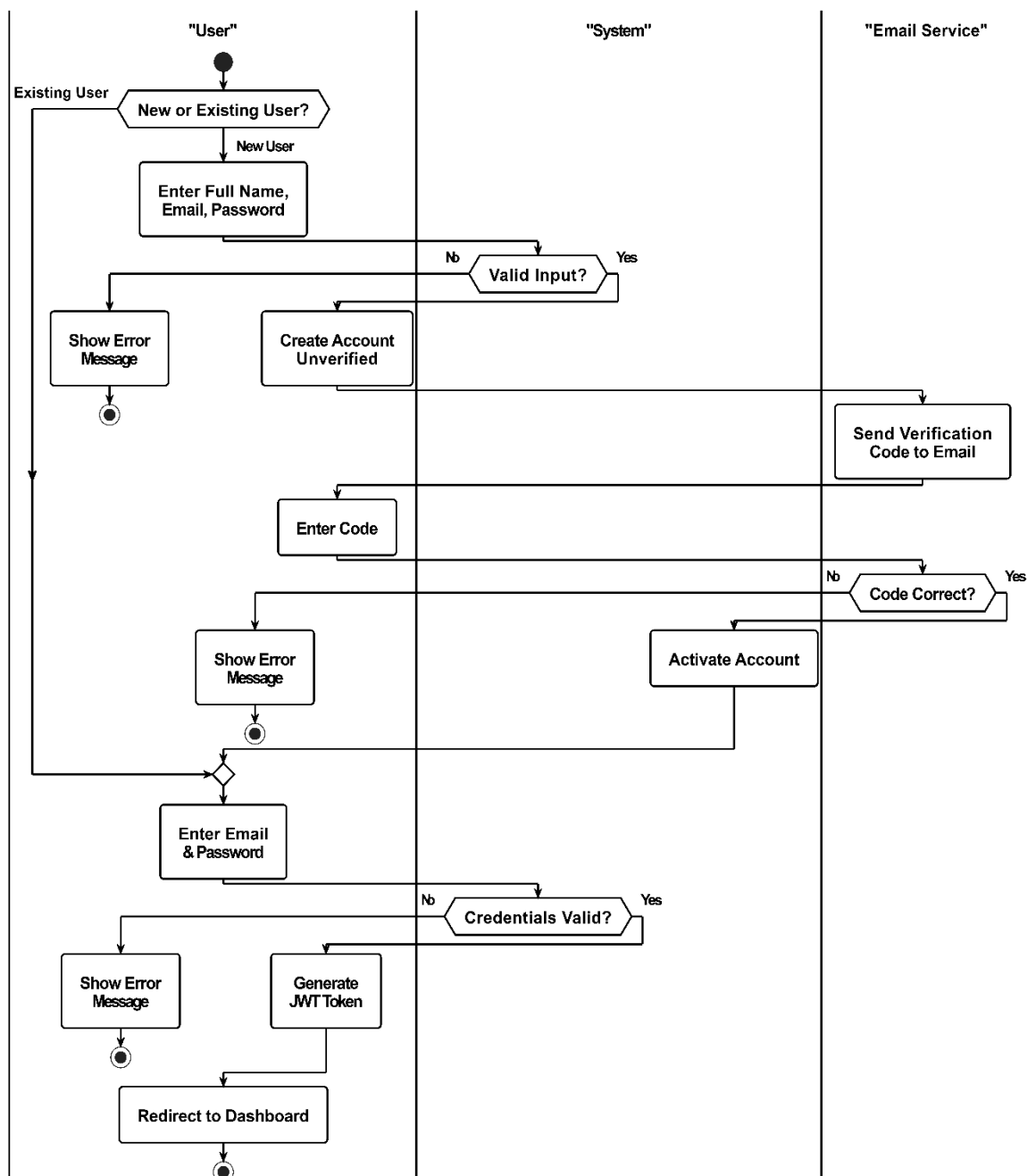


Figure 4.15 User Authenticity and Registration

4.5.2 Video Analysis Activity:

Figure 4.16 illustrates the video analysis workflow of the proposed system. It includes video upload, validation, frame extraction, face detection, AI-based analysis, and result generation. The system stores the analysis results and presents them to the user.

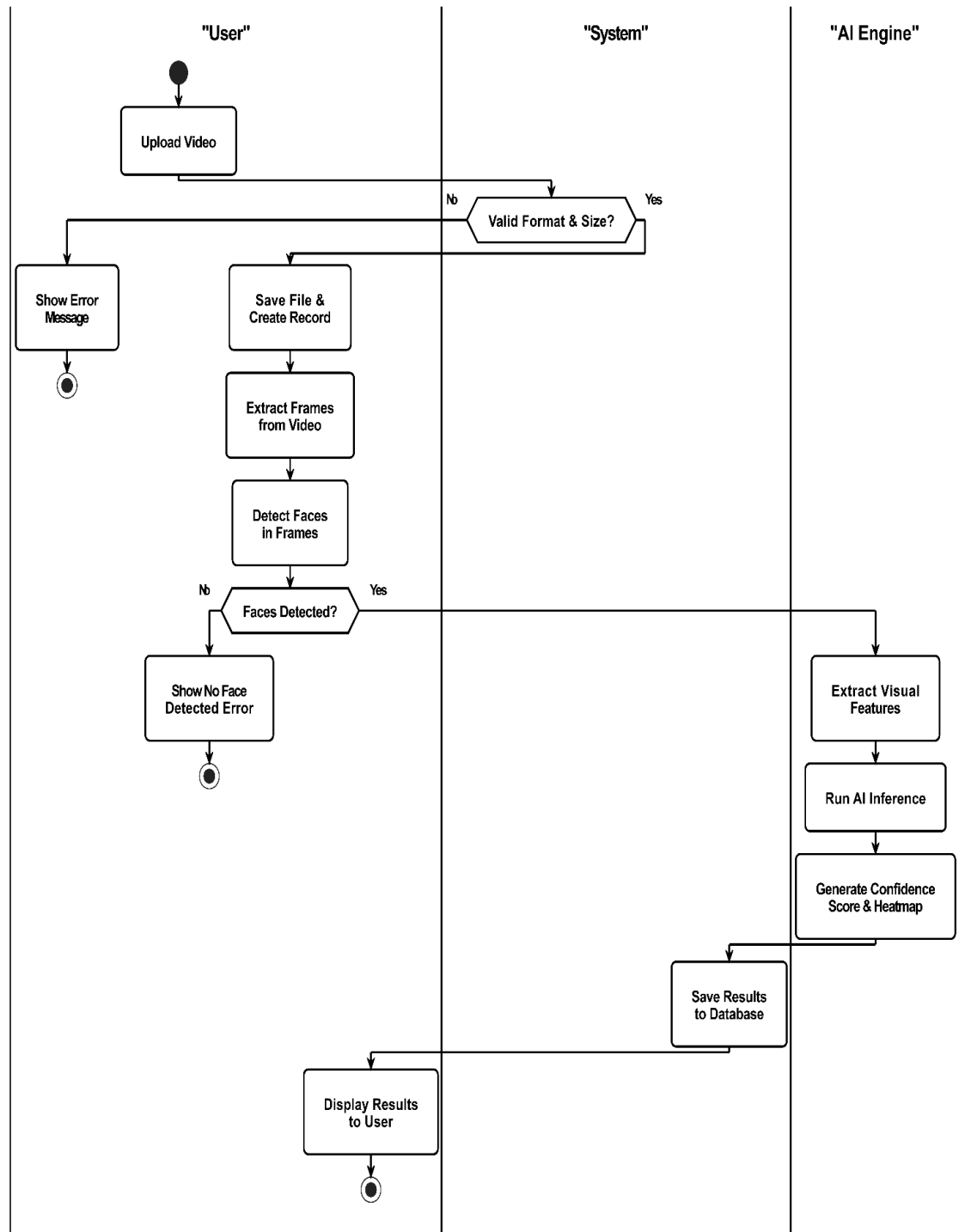


Figure 4.16 Video Activity

4.6 Chapter Summary

This chapter explained comprehensive system requirements for this platform, defining both functional specifications including user authentication, video upload and validation, AI-powered deepfake analysis with confidence scoring, and administrative controls and non-functional standards covering performance, security (encrypted authentication, role-based access), usability (intuitive interface, responsive design), and reliability. The use case diagrams and descriptions captured the interactions between each actor and the system, while the accompanying activity diagrams illustrated the step-by-step flow of the two most decision-intensive processes user authentication and video analysis showing how the system responds at each branching point. Together, these elements create a complete framework that ensures the system delivers accurate, accessible, and trustworthy deepfake detection while covering all technical and user experience objectives

CHAPTER 5 SYSTEM ARCHITECTURE & DESIGN

In this chapter architecture outlines the workflow of the system, which helps programmers understand the overall structure. It comprises drawing out an application's blueprint, which includes the components' interactions, structure, and relationships with system users. Making a road map that guarantees the system satisfies all user requirements is the main goal. Several architectural diagrams will be used to clarify the architecture of our project.

5.1 System Architecture Diagram

Product planning, design, and building constitute the process of architecture. Since it describes the system as the main software components, the architecture's design phase occurs throughout the life cycle's solution phase. A well-defined architecture serves as the foundation of any software system by providing a structured approach to development, integration, deployment, and maintenance. It helps ensure that all components work together efficiently while meeting both functional and non-functional requirements. The selected architecture also influences the system's performance, reliability, scalability, and future adaptability.

We have decided to use a Three-Tier Architecture for our project because of its clear separation of concerns and advantages for web application development. Our objectives of promoting testability, scalability, and maintainability throughout the project life cycle are in line with this architectural pattern. The architecture allows different parts of the system to be developed, tested, and maintained independently, which improves development efficiency and reduces system complexity. Furthermore, it supports future enhancements and integration of additional features without significantly affecting existing components.

In relation to our project:

Presentation Tier (Frontend): Built with Next.js 14 using React and TypeScript, this tier handles all user interface components, including authentication forms, video upload interface, results visualization, and admin dashboard. It communicates with the backend through RESTful API calls. This layer is responsible for delivering a responsive and user-friendly experience while ensuring smooth interaction between users and the system. It also performs client-side validation and manages the presentation of analysis results in an intuitive format.

Application Tier (Backend): Developed using FastAPI with Python, this tier contains the business logic, authentication services, video processing pipelines, and AI model integration. It processes requests from the frontend, interacts with the database, and manages background tasks. In addition, it acts as the central processing unit of the system, coordinating communication between different services and ensuring that all application operations are executed efficiently and securely.

Data Tier (Database): Implemented with PostgreSQL, this tier stores user information, video metadata, analysis results, and system logs. It ensures data persistence, integrity, and efficient querying through proper indexing and relationship management. The database layer also provides secure data storage, supports backup and recovery mechanisms, and facilitates reliable access to information required by different parts of the system.

Our goal in using the Three-Tier Architecture is to accomplish the following important goals:

- **Separation of concerns:** By dividing our application into distinct layers, we simplify management and modification of each component separately. This reduces dependencies and makes the system easier to understand and maintain.
- **Modularity:** The modular structure encourages code reuse and maintainability, making it easier to update and improve the system in the future. New functionalities can be introduced with minimal impact on existing modules.
- **Scalability:** This architecture facilitates the creation of scalable applications, making it simpler to adapt to requirements changes and growth over time. Individual tiers can be scaled independently according to system demands.
- **Security:** Separation between layers enhances security by controlling data flow and access permissions at each tier. Sensitive operations and data handling can be restricted to appropriate layers, reducing potential security risks.
- **Testability:** The distinct roles and responsibilities make testing easier, enabling thorough testing of each component separately. Unit testing, integration testing, and system testing can be performed more effectively.
- **Maintainability:** Since each tier has a clearly defined responsibility, troubleshooting, debugging, and system updates become more manageable, reducing long-term maintenance effort.

All things considered, the Three-Tier Architecture offers a strong foundation for our project, allowing us to create an organized and effective software solution that successfully satisfies the needs of our users. By ensuring clear separation between the user interface, application

logic, and data management components, the architecture promotes reliability, efficiency, and long-term sustainability. The System Architecture is shown in **Figure 5.1**.

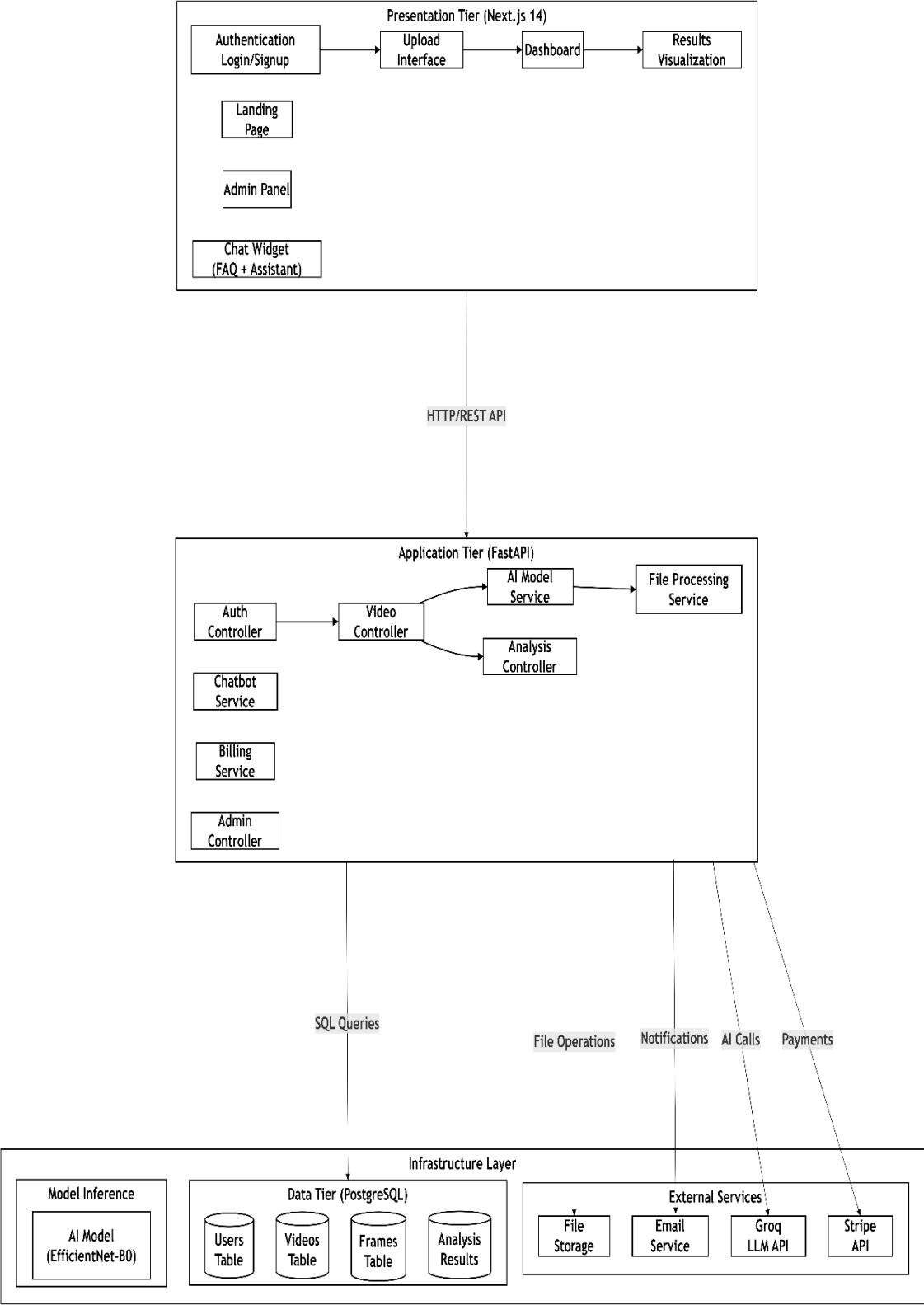


Figure 5.1 System Architecture

5.2 Sequence Diagrams

The interactions between software components is shown in sequence diagrams. By showing the series of events and the order in which messages are transmitted between objects over time, it illustrates the dynamic aspect of a system's behaviour.

5.2.1 User Registration Sequence

Figure 5.2 illustrates the account creation process, including input validation, uniqueness checks, and secure credential storage. It also shows database operations, email verification, and error handling during user registration.

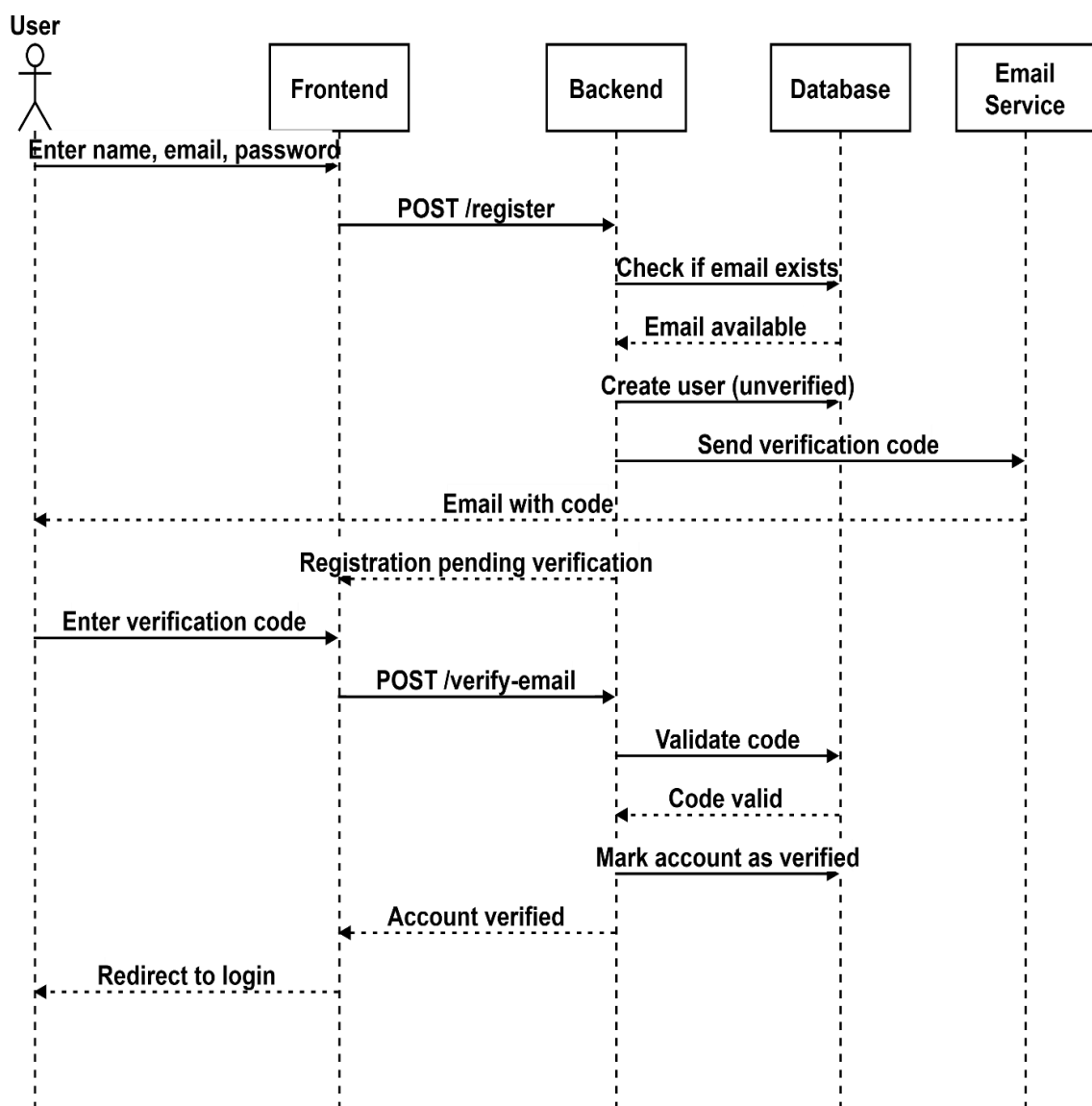


Figure 5.2 User Registration Sequence Diagram

5.2.2 Video Upload and Analysis Sequence

Figure 5.3 illustrates the complete video analysis workflow, from video upload to result delivery. It shows the interactions between the user, frontend, backend, AI model, and database, including file validation, background processing, and real-time status updates.

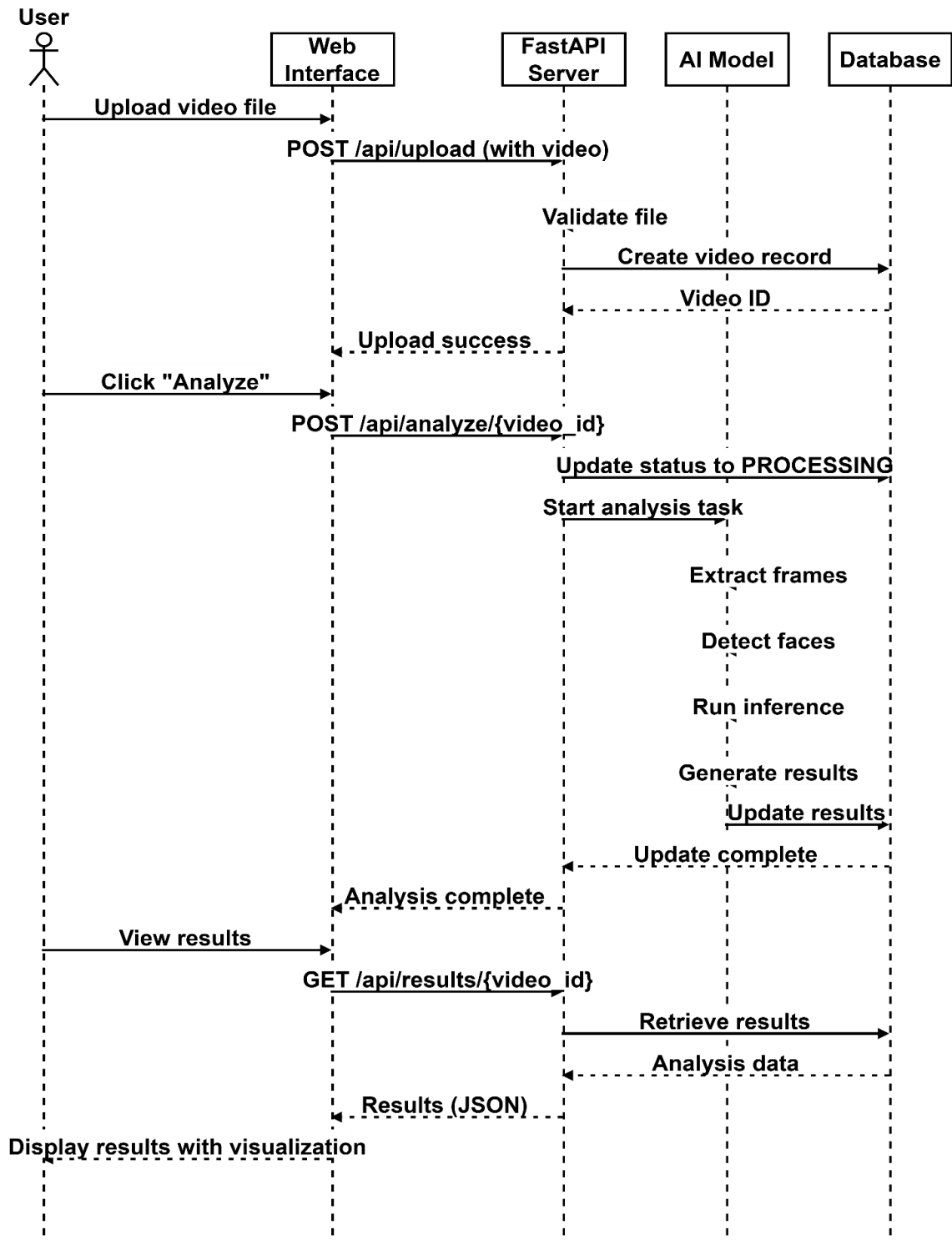


Figure 5.3 Sequence Diagram for Video upload & Analysis

5.2.3 Admin User Management Sequence

Figure 5.4 illustrates the administrative workflow for managing user accounts. It shows administrator authentication, user data retrieval, account modification, and database updates with appropriate validation and error handling.

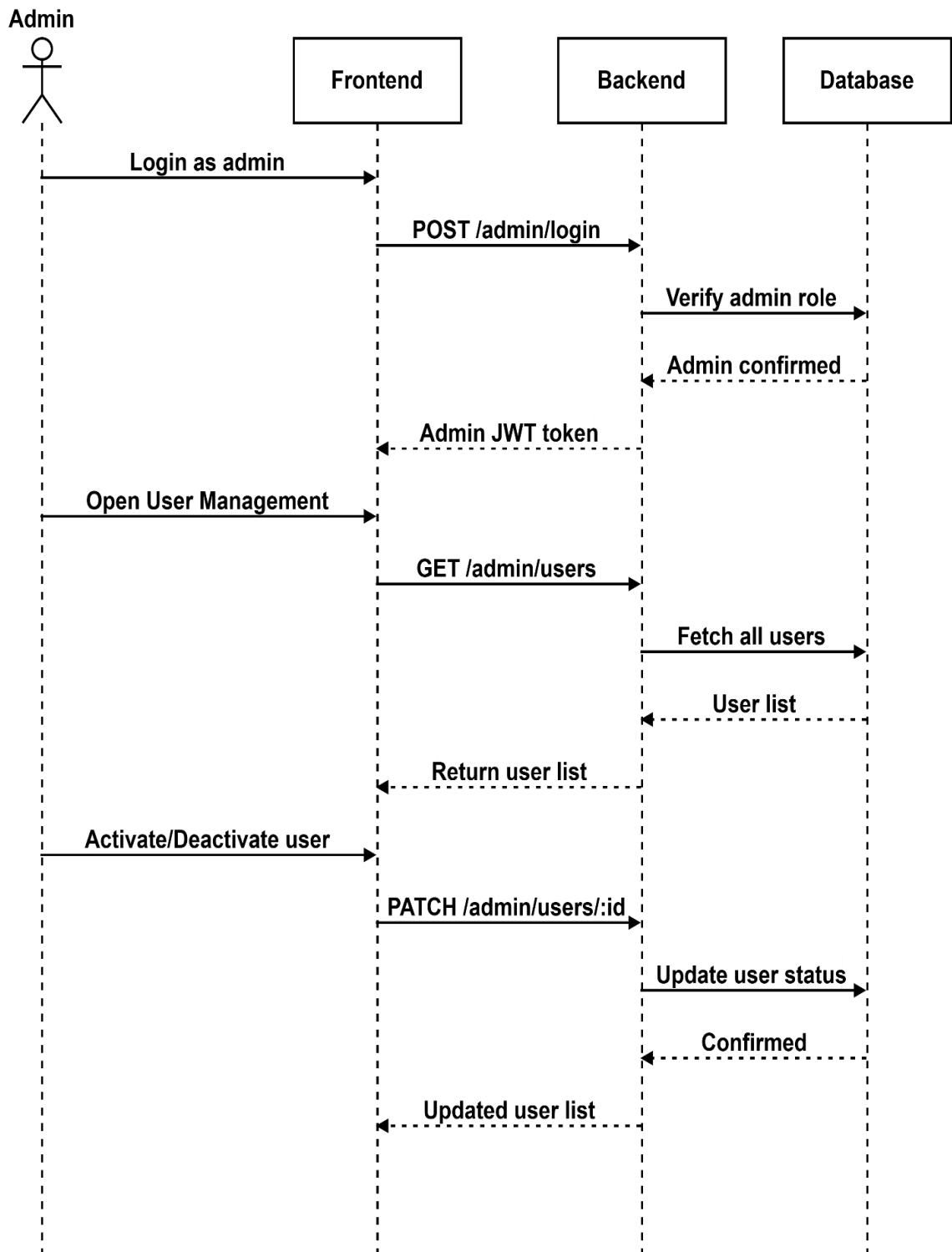


Figure 5.4 Admin User Management Sequence

5.3 Database Design

SQLite relational database design with proper normalization is demonstrated in the following diagram. With user-video relationships and efficient data organization made possible by this design, complex data models will be stored and queried more effectively.

Entity Relationship Diagram for database design is shown in **Figure 5.5**.

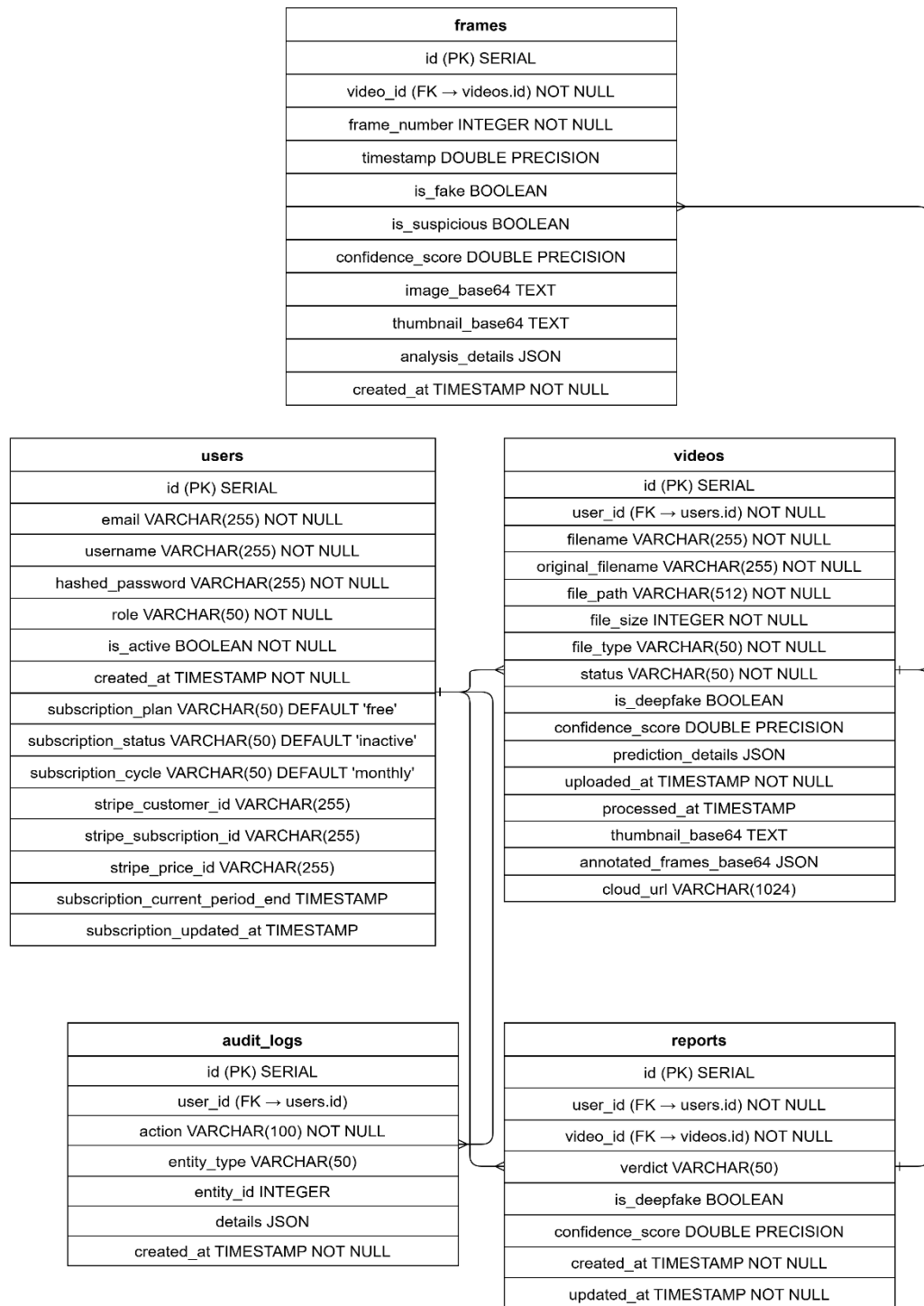


Figure 5.5 Entity Relationship Diagram

5.4 Class Diagram

Figure 5.6 illustrates the workflow for video upload and deepfake detection. It shows the structured processing pipeline, from video submission and validation to AI analysis and result generation.

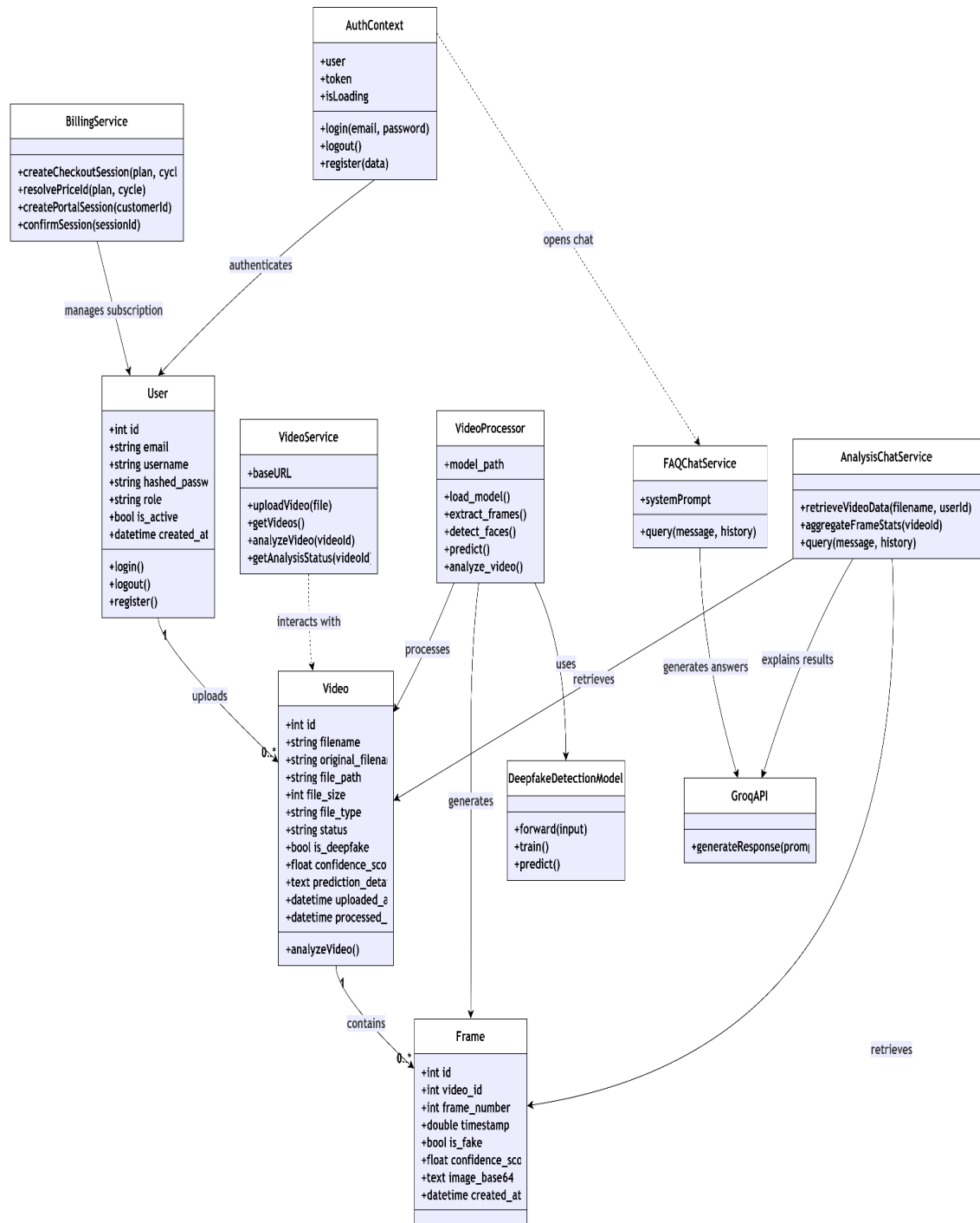


Figure 5.6 Class Diagram

Users authenticate via the AUTH Context and upload videos using the Video Service, which stores metadata and manages file information. The Video Processor analysis each video using the Deepfake Detection Model for prediction and training, completing the flow from authentication → upload → processing → detection in a modular and efficient manner.

5.5 Development Languages and Frameworks

The selection of development technologies was driven by the need for efficiency, scalability, and CPU-based AI execution. The following technology stack was selected for our project:

- Python (FastAPI Backend)
- Next.js (Frontend Framework)
- ConvNeXt-Tiny (AI Model)
- PostgreSQL (Database)
- PyTorch (Model Training & Inference)

5.5.1 Python & FastAPI

Python serves as the primary backend language for our detection system due to its extensive AI and machine learning ecosystem. FastAPI was selected as the backend framework for its asynchronous request handling, automatic API documentation generation, and superior performance compared to traditional synchronous frameworks. FastAPI efficiently manages video upload endpoints, model inference calls, and user authentication workflows, enabling fast response times even during computationally intensive deepfake analysis operations.

5.5.2 Next.js (Frontend)

Next.js powers the frontend of our web app, providing server-side rendering (SSR), static site generation, and optimized bundling for a responsive and accessible user interface. Its built-in routing system and API integration capabilities make it ideal for building a seamless user experience where users can upload videos, view detection results, and access their analysis history. The React-based component architecture of Next.js enables modular and maintainable frontend development. It also improves page load performance and SEO through pre-rendering techniques. Additionally, it supports efficient state management and reusable UI components for scalable application growth.

5.5.3 ConvNeXt-Tiny (AI Model)

DeepFake Detector is an AI-powered web platform that uses a ConvNeXt-Tiny model to identify manipulated media through 6-class classification. It detects Deepfakes, Face2Face, FaceShifter, FaceSwap, NeuralTextures, and authentic content. Unlike binary detectors, it identifies the specific manipulation technique used. The model is trained on the FaceForensics++ dataset for reliable performance. The platform provides confidence scores, visual heatmaps, and explainable AI insights. It runs on standard CPU hardware, enabling accessible and transparent deepfake detection.

5.5.4 PostgreSQL (Database)

PostgreSQL serves as the relational database for our video detection system, managing user accounts, analysis history, video metadata, and administrative data. Its ACID-compliant transactional model ensures data integrity, while its support for JSON fields allows flexible storage of AI analysis results including confidence scores, heatmap references, and textual explanations. PostgreSQL's robust indexing and query optimization capabilities ensure fast retrieval of user analysis records even as the dataset grows.

5.5.5 PyTorch (Deep Learning Framework)

PyTorch is used for model training, validation, and inference. Its dynamic computational graph approach provides flexibility during model experimentation, while TorchScript enables optimized model deployment. The integration of Grad-CAM (Gradient-weighted Class Activation Mapping) within PyTorch allows our web app to generate visual heatmaps that highlight the specific facial regions contributing to the deepfake classification decision, directly supporting the system's explainable AI objectives.

5.5.6 AI Assistant Services

To enhance accessibility and reinforce the system's commitment to transparency, two AI-powered conversational assistants were integrated using the Groq API with the llama-3.1-8b-instant model. Groq's inference infrastructure was selected for its free tier and low-latency responses, aligning with the project's objective of providing an accessible system without requiring expensive local hardware for AI features.

The **FAQ Assistant** is available on the public landing page and operates using a static knowledge-base system prompt containing key information about the platform's purpose, supported video formats, detection methodology, and account requirements. This allows

prospective users to learn about the system before registering, without requiring any database access.

The **Analysis Assistant** is available to authenticated users on their dashboard and implements a Retrieval-Augmented Generation (RAG) pattern. When a user asks about a specific video, the backend retrieves the corresponding records from the videos and frames tables, aggregates frame-level statistics (percentage of frames flagged as fake, average confidence score, and flagged facial regions from the analysis details field) and constructs a structured context summary. This summary, along with the user's question, is passed to the Groq LLM, which generates a plain-language explanation grounded entirely in the retrieved data.

Both assistants are implemented as stateless, session-only services — conversation history is maintained only in the frontend's component state and is cleared on page refresh. This design choice keeps the implementation lightweight and avoids additional data retention considerations.

5.5.7 Payment & Subscription Module (Stripe Integration)

To support sustainable scaling of computational resources for advanced detection features, a tiered subscription system was integrated using Stripe. The frontend Plans page presents three tiers: **Free**, **Pro**, and **Enterprise** through the Pricing Plans component, allowing users to switch between monthly and yearly billing cycles. If a user attempts to subscribe without being authenticated, they are redirected to the login page before the checkout process begins. **This integration provides a secure and user-friendly payment experience while enabling flexible subscription options based on user requirements.**

On the backend, the billing route resolves the appropriate Stripe price ID based on the user's selected plan and billing cycle, then creates a Stripe Checkout session and redirects the user to Stripe's hosted payment page. Once payment is completed, a confirm-session endpoint persists the resulting subscription details “subscription_plan, subscription_status, stripe_customer_id, and stripe_subscription_id” to the user's record in the database. The system also validates the payment session before updating the subscription information, ensuring data consistency and secure transaction processing. The user's dashboard then reflects their active plan.

For ongoing subscription management, users can access a Stripe-hosted billing portal directly from their dashboard, allowing them to update payment methods, change plans, or cancel their subscription without requiring a custom management interface. This approach reduces development complexity while providing users with a reliable and secure subscription management experience.

5.6 Development Tools

Visual Studio Code (VS Code) serves as the primary integrated development environment (IDE) for our web app development. Its extensive extension marketplace, integrated debugging, and Git support make it the ideal choice for managing a full-stack AI project.

5.6.1 Key Features and Benefits

- **Intelligent Code Editing:** IntelliSense provides smart completions for Python, JavaScript, and SQL, significantly accelerating development across the entire web app stack.
- **Integrated Debugging:** The built-in debugger supports both Python (FastAPI backend) and JavaScript (Next.js frontend), enabling efficient troubleshooting of API endpoints and UI components.
- **Git Integration:** Native Git support simplifies version control, enabling seamless collaboration, branch management, and code review workflows directly within the editor.
- **Extension Marketplace:** Extensions for Python, ESLint, Prettier, Docker, and PostgreSQL provide a tailored development environment aligned with project's technology stack.
- **Seamless Collaboration:** Integration with GitHub facilitates code reviews, pull requests, and project management across the development team.

5.6.2 Why VS Code for Our Deepfake Detection?

The adaptability and comprehensive feature set of VS Code make it the optimal choice for developing a complex AI system. It supports the full development lifecycle from model training scripts in Python to React component development in Next.js within a unified environment. Its user-friendly interface and robust documentation support efficient development of clean, maintainable, and scalable code.

5.7 Collaboration Diagram

The collaboration diagram for our project illustrates the interactions between the system's core components to achieve the primary goal of deepfake detection. Unlike sequence diagrams that focus on timing, collaboration diagrams emphasize the structural relationships and communication patterns between objects. This diagram is essential for understanding how the frontend interface, AI detection engine, backend API, and database coordinate to deliver detection results to the user. The collaboration diagram is shown in **Figure 5.7**.

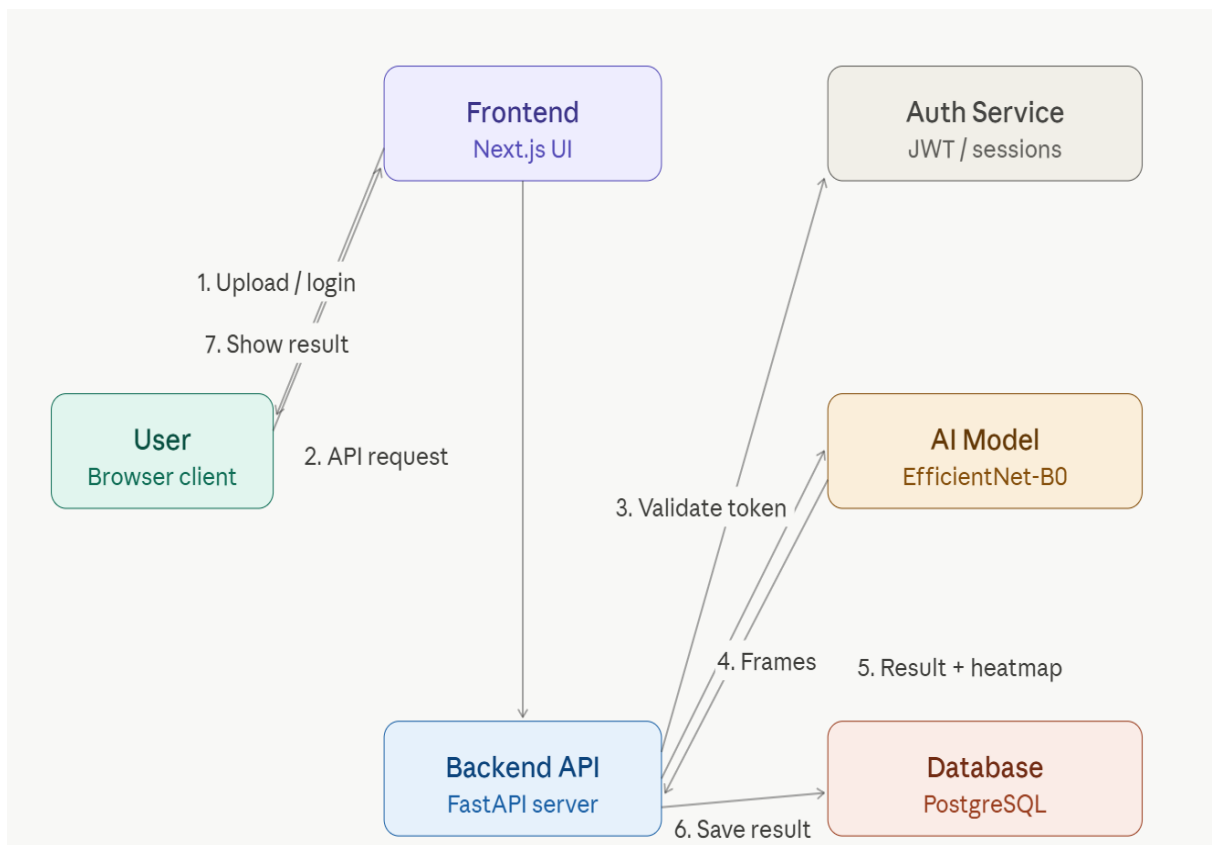


Figure 5.7: System Collaboration Diagram

5.8 Chapter Summary

Chapter 5 presents the implementation phase of the web application, covering the technologies, tools, and design methodologies used, including Python with FastAPI (backend), Next.js (frontend), ConvNeXt-Tiny (AI detection), PostgreSQL (database), PyTorch (model training and inference), and Visual Studio Code as the development environment. It also reviews key system design artifacts, including architecture, activity, sequence, and database diagrams, highlighting how the system ensures a secure, scalable, and user-friendly deepfake detection platform focused on accuracy and transparency.

CHAPTER 6 SYSTEM PROTOTYPING, TESTING & INTERFACE DIAGRAM

Our deepfake detection application prototype represents a pivotal milestone in transforming the project's conceptual framework into a tangible, functional system. It is a fully operational working model that demonstrates the key features and interactive components of the deepfake detection platform. The prototype provides a clear representation of how users will interact with web app to upload videos, receive AI-powered detection results, and interpret explainable AI outputs.

The main goals of our system prototype are to:

- **Validate Core Features:** Confirm that essential functionalities such as video upload, AI-powered deepfake detection, heatmap visualization, and result explanation align with user expectations.
- **Collect User Feedback:** Engage stakeholders and potential users to gather input for refining the system's features, interface design, and performance.
- **Test Usability:** Analyse the user interface (UI) and user experience (UX) to identify areas for improvement in navigation, accessibility, and result interpretation.
- **Demonstrate Technical Feasibility:** Show that advanced features including CPU-optimized AI inference, real-time analysis, and explainable AI components can be implemented effectively within available technological resources.

6.1 Introduction to Deepfake Project Interfaces

Every interface has been carefully designed to ensure smooth navigation, clarity of information, and ease of use, enabling users to interact with the system without technical difficulty.

The interfaces discussed include the Landing Page, which introduces the platform and its core features, and the Documentation Page, which offers both technical details and user guidance for better understanding of the system. It also covers Authentication Interfaces for secure login and registration, ensuring controlled access to the platform. The Upload Video Interface allows users to submit videos for analysis, while the Analysis Results Page presents detection outcomes along with visual explanations such as heatmaps and confidence scores.

6.1.1 Landing Page

Figure 6.1 shows the landing page, which serves as the entry point for all users. It introduces the platform's core mission of making deepfake detection accessible, transparent, and reliable.

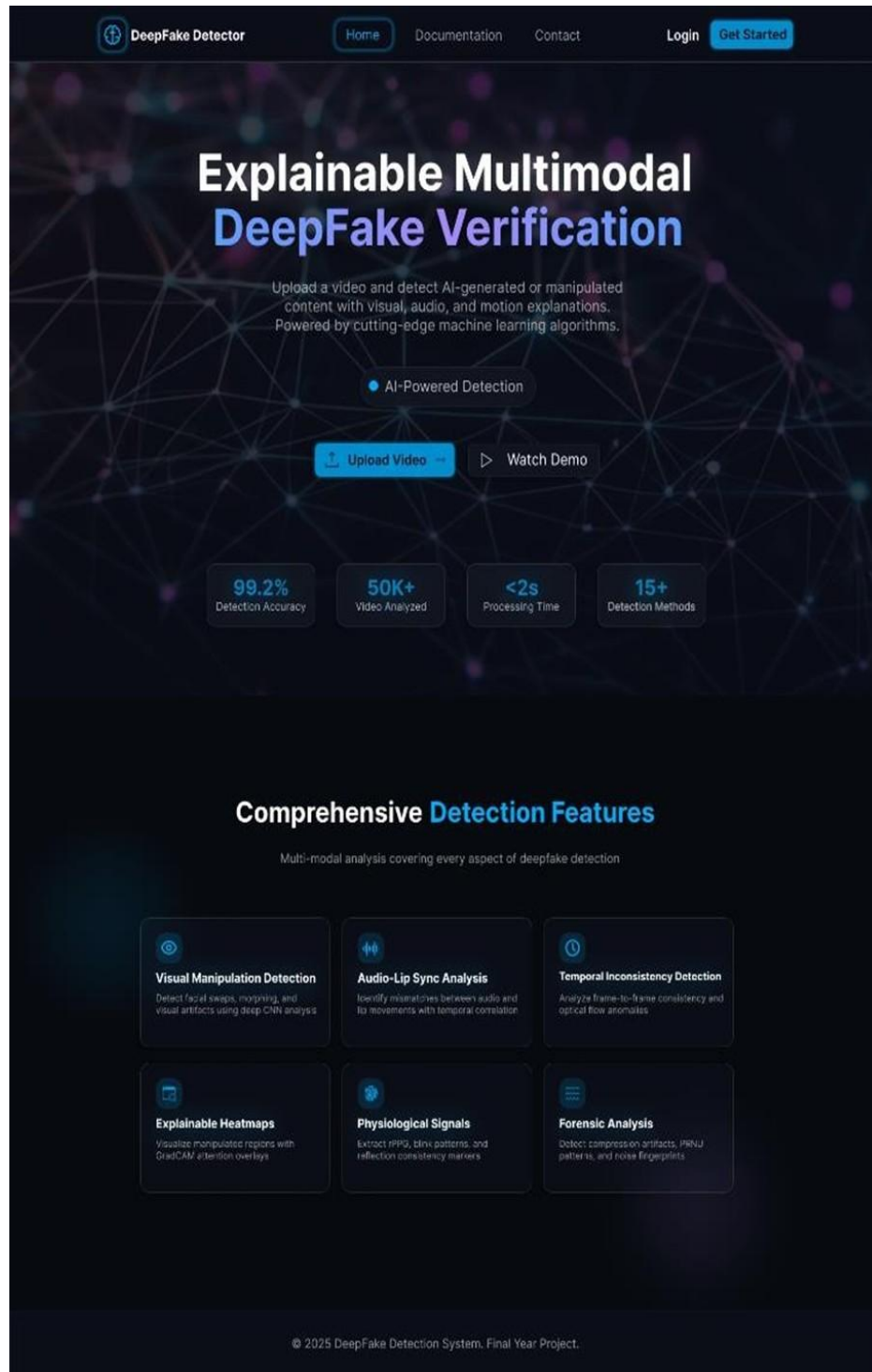


Figure 6.1: Landing Page

6.1.2 Documentation Page

Figure 6.2 shows the documentation page, which provides comprehensive guides for both end users and developers. It includes step-by-step instructions for using the platform, API endpoint documentation for developers integrating with the system, and FAQs addressing common questions about deepfake detection and the platform's capabilities.

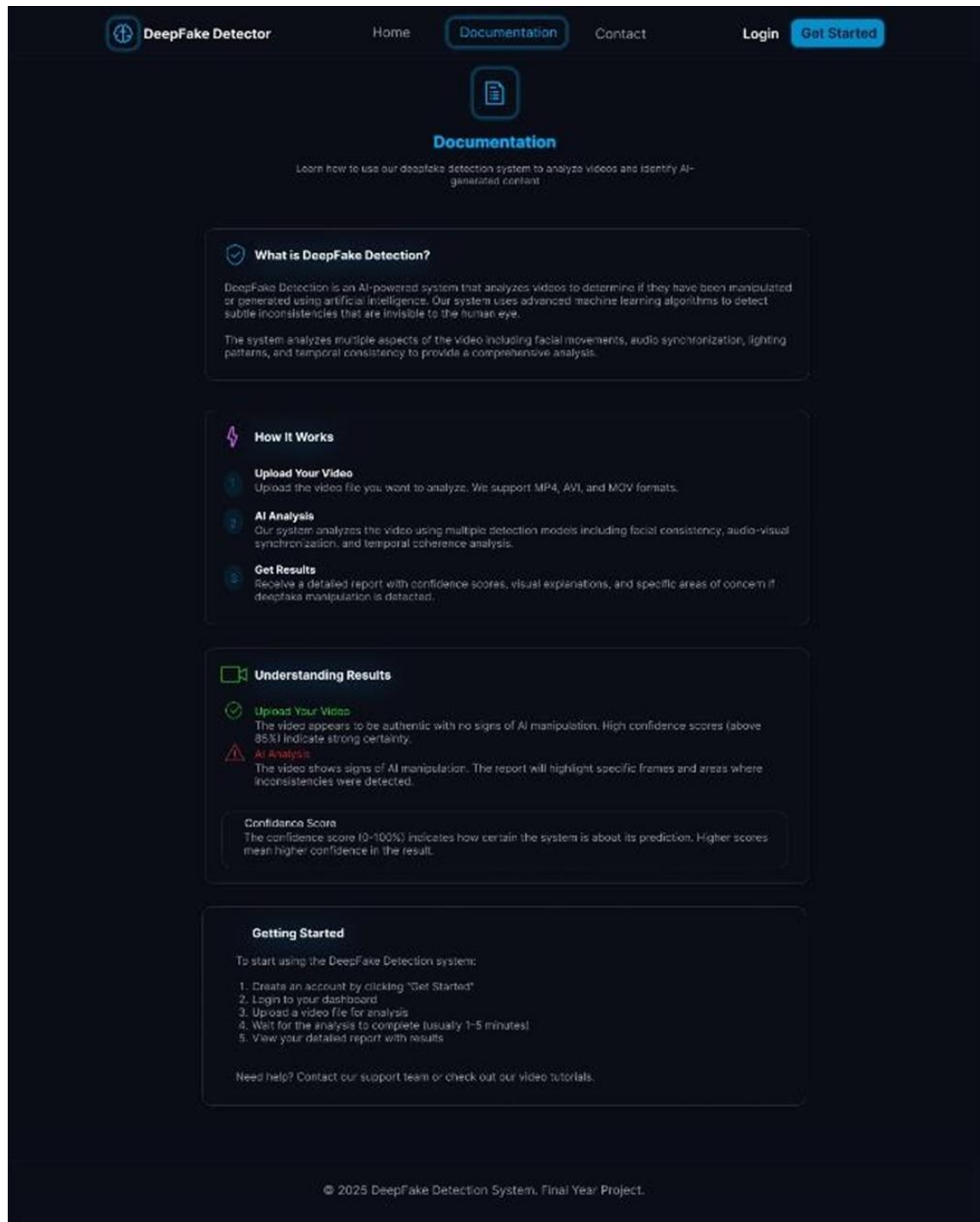


Figure 6.2: Documentation Page

6.1.3 Login Interface

Figure 6.3 shows the login interface where registered users authenticate using their email and password credentials. The interface is designed with a clean and user-friendly layout to ensure easy access to the system. Users enter their registered email and password to proceed with authentication. If invalid credentials are entered, the system displays a clear error message without specifying whether the email or password is incorrect, thereby preventing enumeration attacks. This enhances both usability and security during the login process.

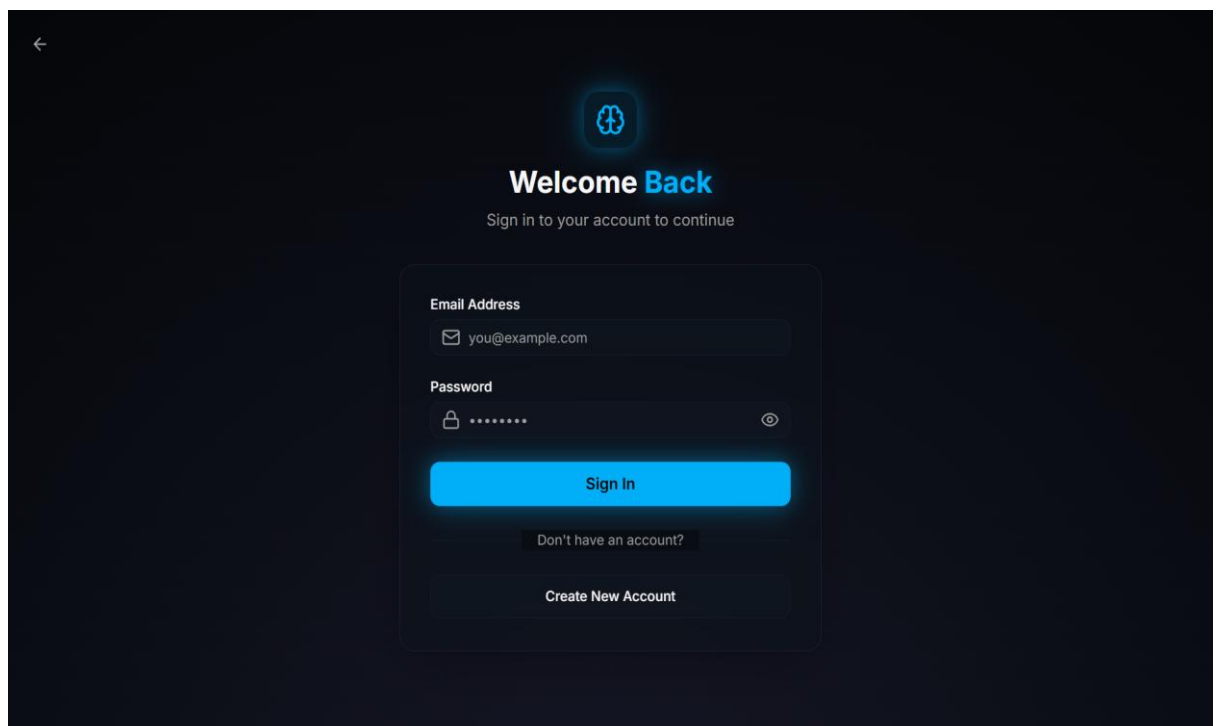


Figure 6.3: Login Interface

6.1.4 Sign Up Interface

Figure 6.4 shows the registration interface where new users can create an account. Users are required to enter their name, email address, and password to begin the sign-up process. After clicking continue the Figure 6.5 shows that the system sends a 6-digit verification code to the provided email address for authentication. This step ensures that only valid and accessible email accounts can be used for registration. Users must enter the correct verification code within the given time limit to proceed further. If the code is valid, the system verifies the email and confirms user identity. This prevents the creation of fake or spam accounts and improves platform security. Once verification is successful, the user

account is created and securely stored in the database. After successful registration, the user is automatically redirected to the dashboard page with full access to platform features.

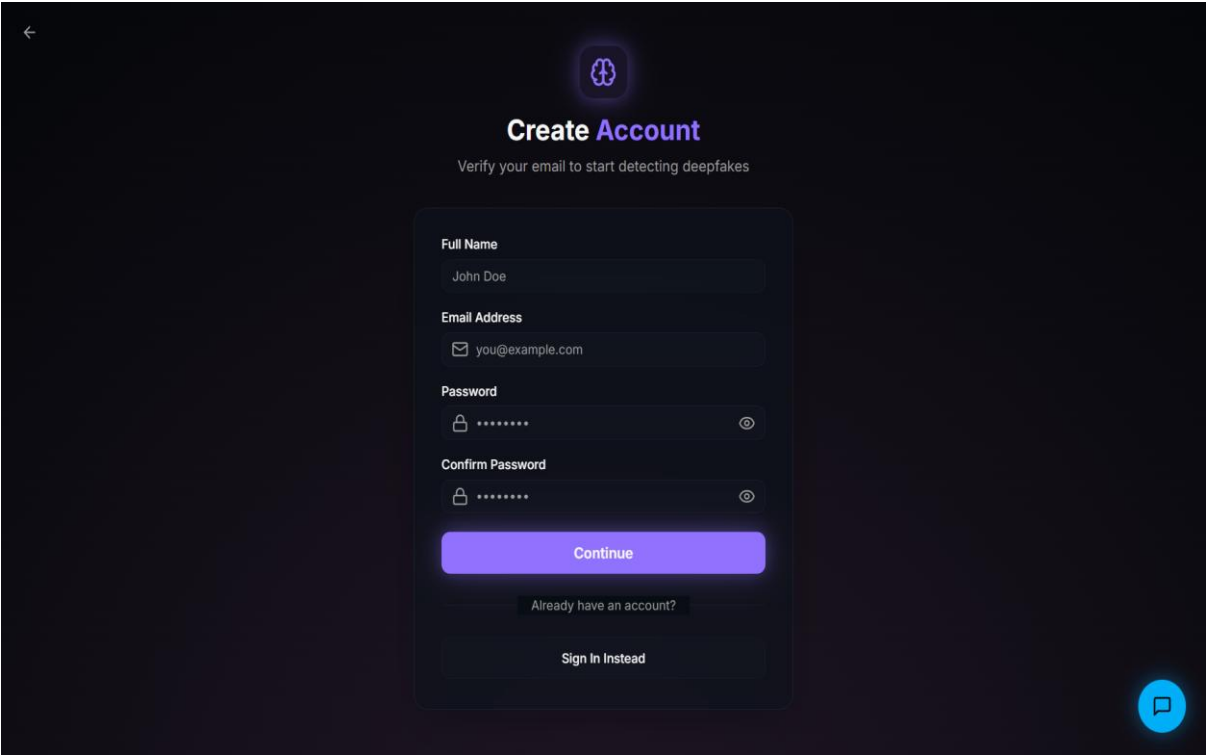


Figure 6.4: Sign Up Interface

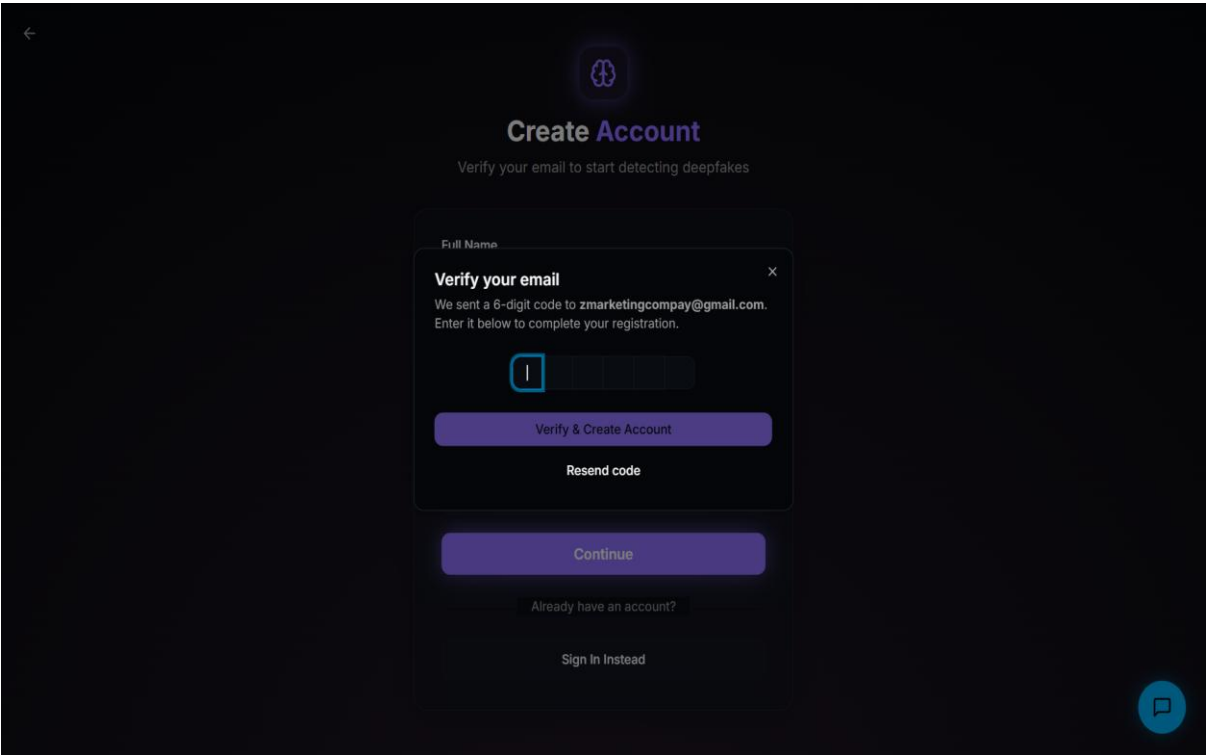


Figure 6.5: Verify Email

6.1.5 Contact Us Interface

Figure 6.6 shows the Contact Us page, which allows users to send queries, report issues, or provide feedback regarding the platform. It provides an easy-to-use interface that enables users to communicate directly with the support or development team. The page includes a structured form with fields for name, email, subject, and message to ensure clear and organized communication. Users can describe their issues or suggestions in detail before submitting the form. Once submitted, the system securely sends the message to the backend for review by the development team. This feature helps improve user engagement, support, and overall system improvement through direct feedback collection. team.

The screenshot displays the 'Contact Us' page of the 'DeepFake Detector' application. The page has a dark theme with blue accents. At the top, a navigation bar includes the logo, links for Home, Documentation, FAQ, and Contact (highlighted), and buttons for Login and Get Started. The main heading 'Contact Us' is centered, accompanied by an envelope icon and a brief message: 'Have feedback about this system or encountered any issues? Feel free to contact us. We value your input and are here to help.' Below this, the interface is divided into two columns. The left column contains a 'Send us a message' form with fields for Name, Email, Subject, and Message, followed by a 'Send Message' button. The right column features three informational boxes: 'Feedback' (welcoming input), 'Report an Issue' (requesting detailed bug reports), and 'Response Time' (stating a 24-48 hour response goal). The footer is organized into three sections: 'About' (with links to Project, Documentation, and Contact Us), 'Resources' (with a link to FAQ), and 'Legal' (with links to Privacy Policy and Terms of Service). A copyright notice for 2025 is at the bottom center.

DeepFake Detector

Home Documentation FAQ **Contact** Login **Get Started**

Contact Us

Have feedback about this system or encountered any issues? Feel free to contact us. We value your input and are here to help.

Send us a message

Name

Your name

Email

your.email@example.com

Subject

What is this about?

Message

Tell us more about your feedback or issue...

Send Message

Feedback

We welcome all feedback about the DeepFake Detection System. Whether you have suggestions for improvements, feature requests, or general comments, we would love to hear from you.

Report an Issue

Encountered a bug or technical issue? Please provide as much detail as possible including the steps to reproduce the problem, error messages, and screenshots if applicable.

Response Time

We aim to respond to all inquiries within 24-48 hours during business days. For urgent issues, please indicate the priority in your subject line.

About

About Project

Documentation

Contact Us

Resources

FAQ

Legal

Privacy Policy

Terms of Service

© 2025 DeepFake Detection System. Final Year Project.

Figure 6.6: Contact Us Interface

6.1.6 User Dashboard

Figure 6.7 shows the user dashboard, which serves as the central hub for authenticated users. It provides a personalized overview of the user’s activity within the system, enabling easy monitoring and management of uploaded videos and analysis results. The dashboard displays key statistics such as the total number of videos analysed and the classification results of deepfakes versus authentic videos. Users can also view their recently analysed videos in a tabular format, where each entry includes options to view detailed analysis results by clicking a dedicated button. They can also delete previously uploaded videos directly from the table when required. Additionally, an “Upload Video” button is available in the upper-right corner, which navigates users to the upload video section in the side panel, opening the corresponding page for video submission. The interface also allows users to revisit complete analysis history for reference and tracking purposes. Overall, the dashboard enhances user experience by providing centralized access to analysis results, video management, and account-related functionalities in a single and efficient interface.

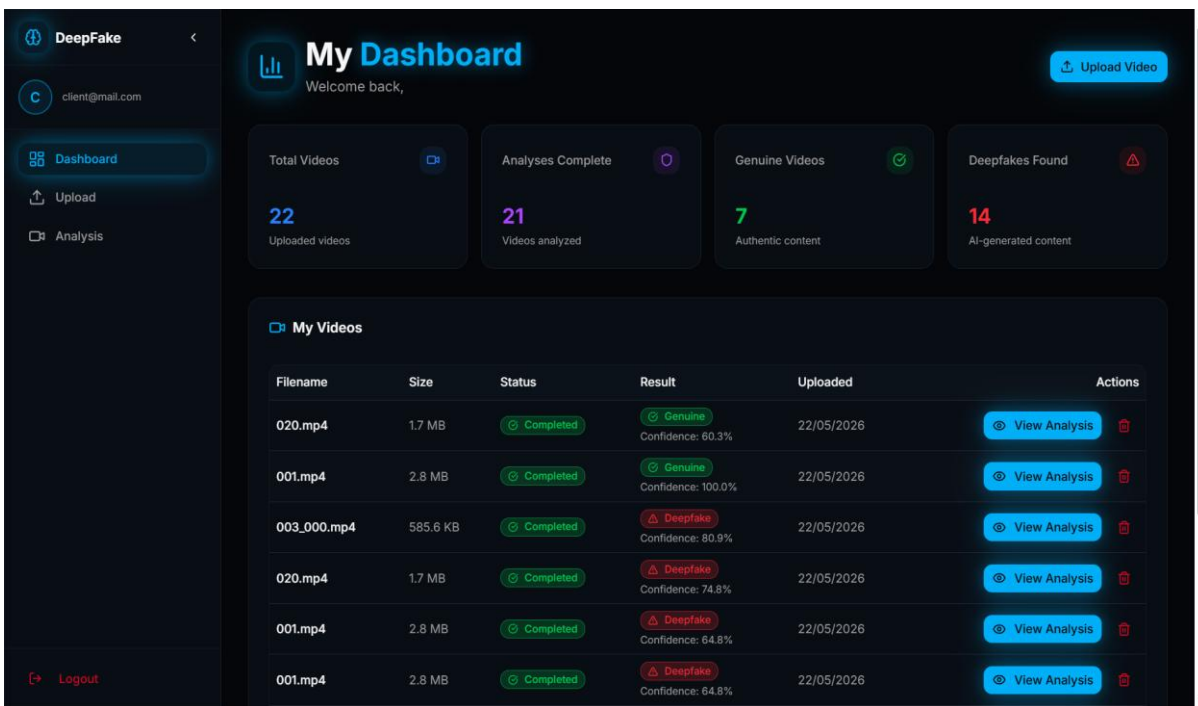


Figure 6.7: User Dashboard

6.1.7 Upload Video Interface

Figure 6.8 shows the video upload interface where users submit videos for deepfake analysis. The interface supports both drag-and-drop functionality and a traditional file browser option for user convenience. It accepts video formats such as MP4, AVI, and MOV

to ensure compatibility with common file types. A real-time upload progress indicator is displayed to inform users about the status of the upload process. Once the upload is successfully completed, the system automatically initiates the AI-based analysis pipeline without requiring additional user input. The interface also ensures validation of file type and size before processing begins. This provides a smooth and efficient workflow from video submission to analysis generation.

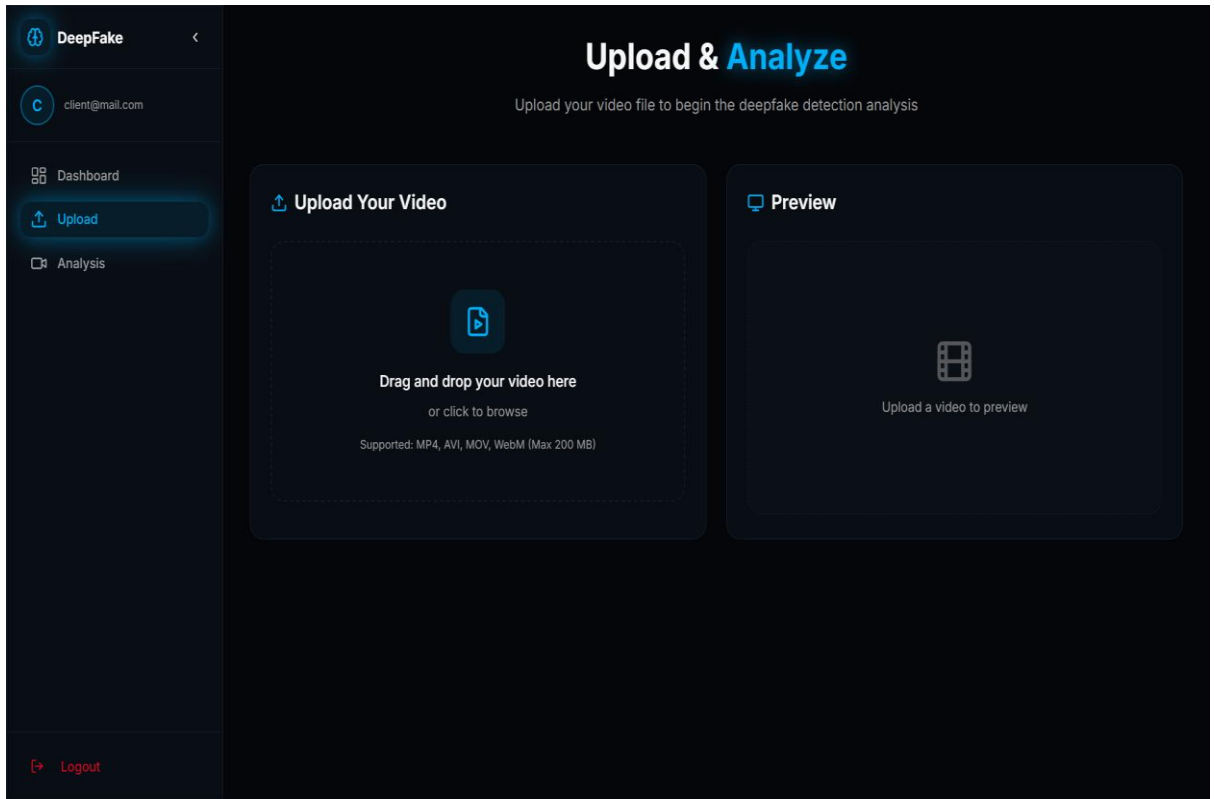


Figure 6.8: Upload Video Interface

6.1.8 Analysis Results Page

Figures 6.9, 6.10, 6.11, and 6.12 show the analysis results page, which is the most critical interface in the project. After the AI processing is completed, this page displays the detection verdict (Deepfake Detected or Authentic) along with the confidence score in percentage form. It also presents a Grad-CAM heatmap visualization that highlights the facial regions contributing to the model’s decision. Additionally, a textual explanation is provided to describe the reasoning behind the classification result in a user-understandable manner. This interface directly supports the project’s explainable AI objectives by making the model’s predictions transparent and interpretable for users. Furthermore, a “Download Report” button is available in the upper-right corner, allowing users to export the complete analysis as a report for record-keeping or sharing purposes. A “View Report” option is also

provided for users who wish to review the generated report within the platform before downloading it.

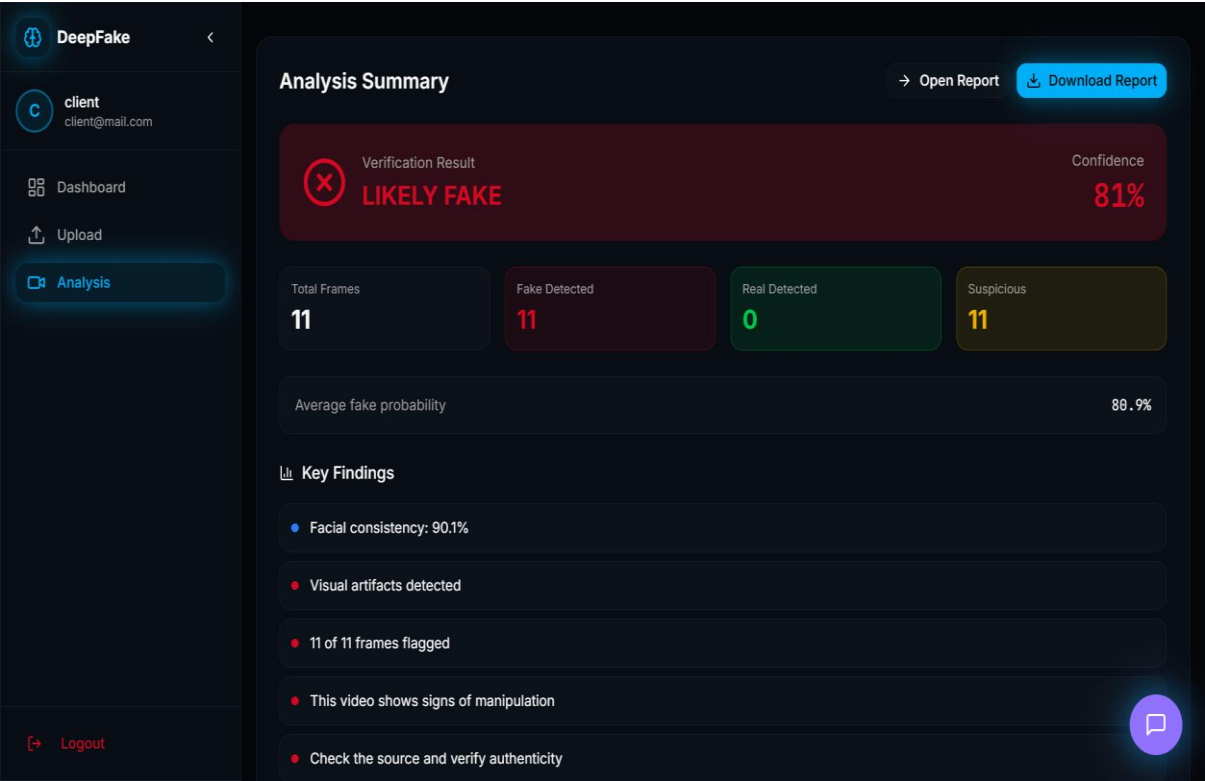


Figure 6.9: Analysis Results Page (statistic)

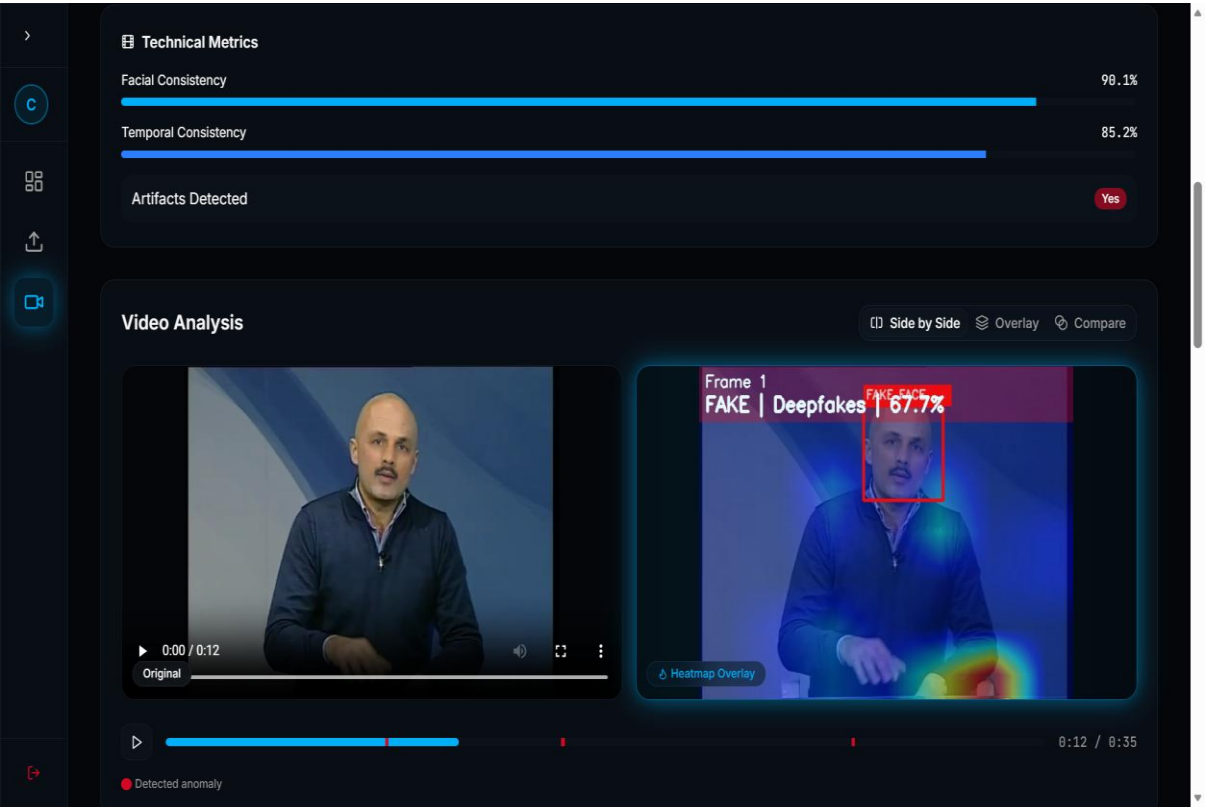


Figure 6.10: Analysis Results Page with Heatmap

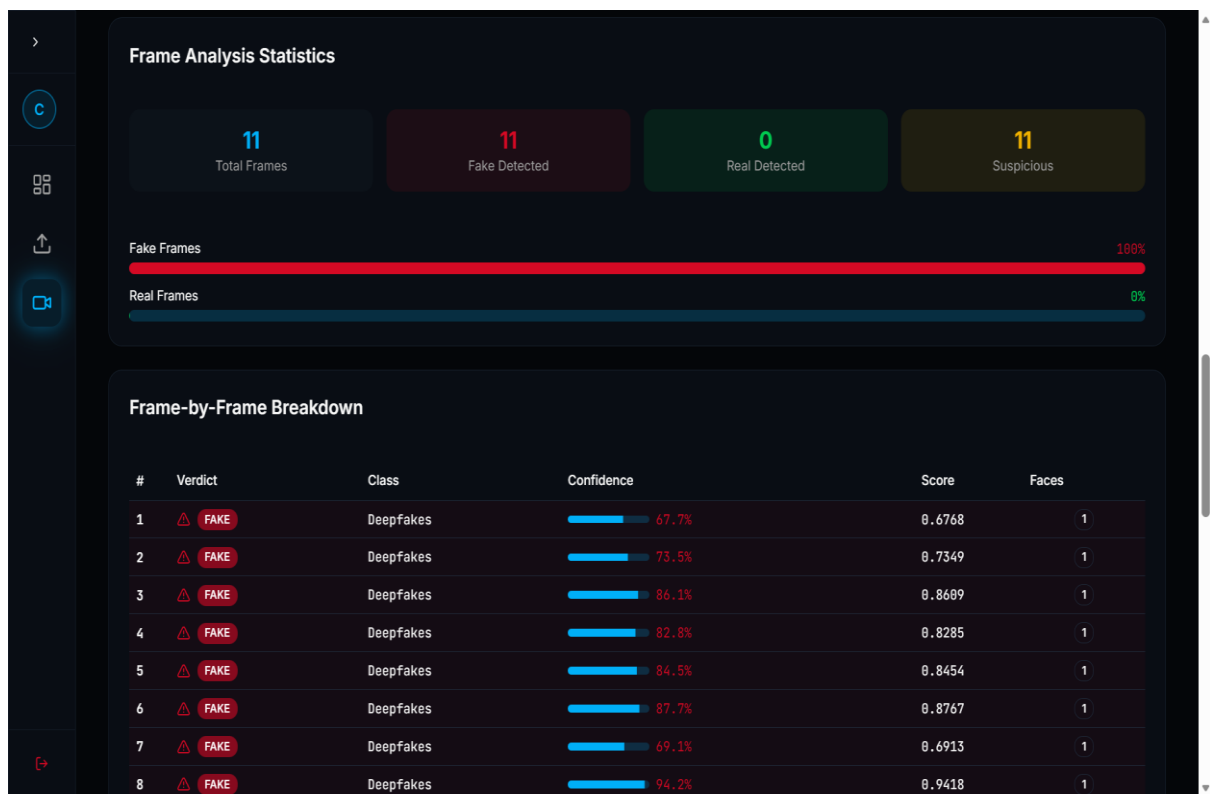


Figure 6.11: Analysis Results Page with frame details

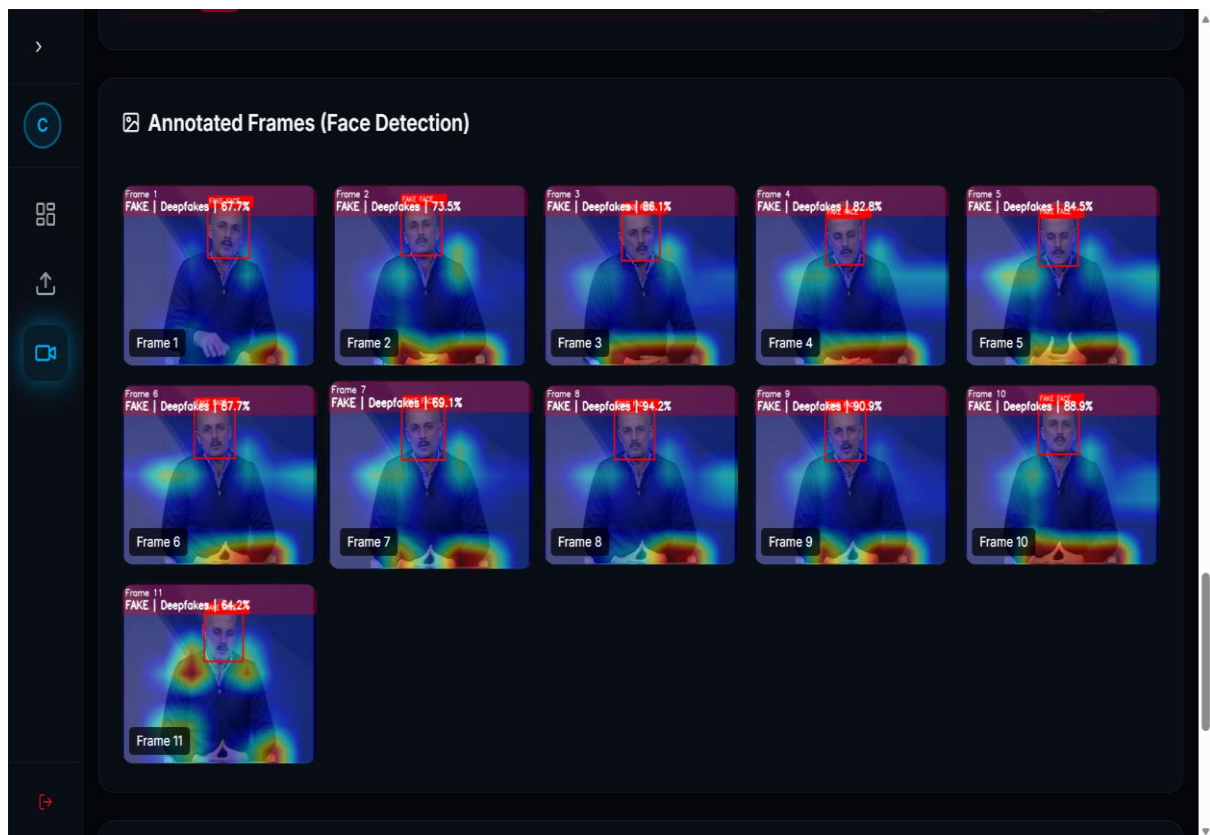


Figure 6.12: Analysis Results Page with Annotated Frames

6.1.9 Admin Panel

Figure 6.13 and 6.14 shows the admin panel, accessible only to system administrators. It provides features for user account management, system-wide analytics such as total uploads and detection statistics, and performance monitoring. system oversight and overall platform management.

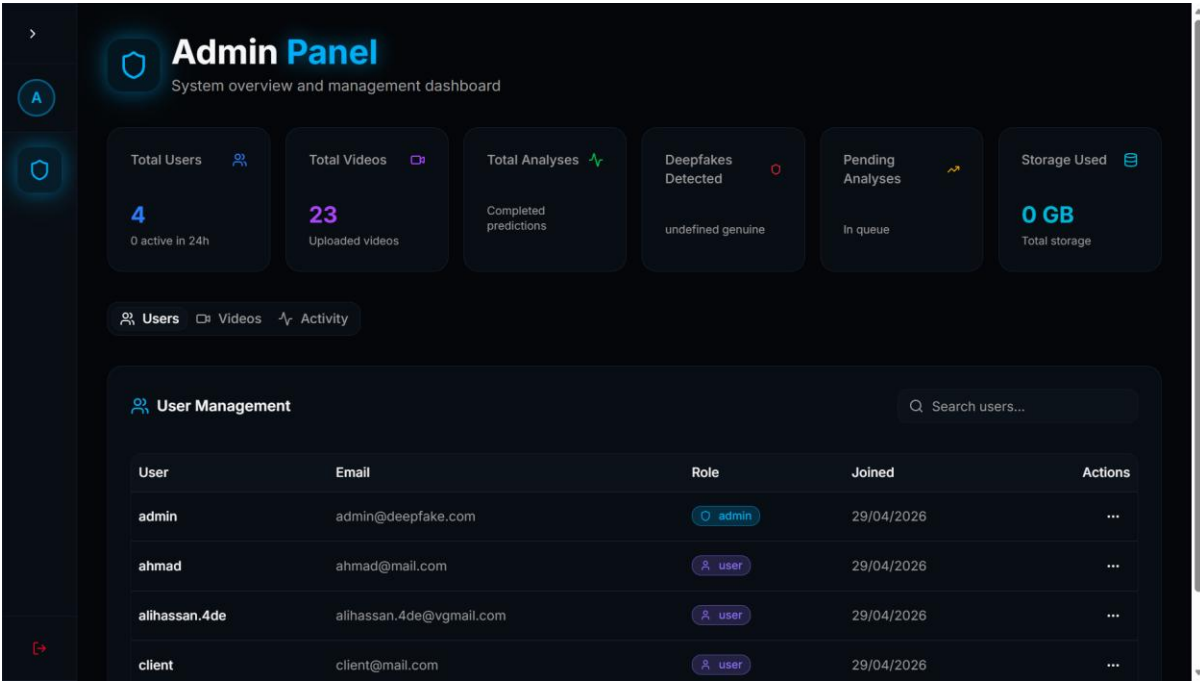


Figure 6.13: Admin Panel Statistics and User Management

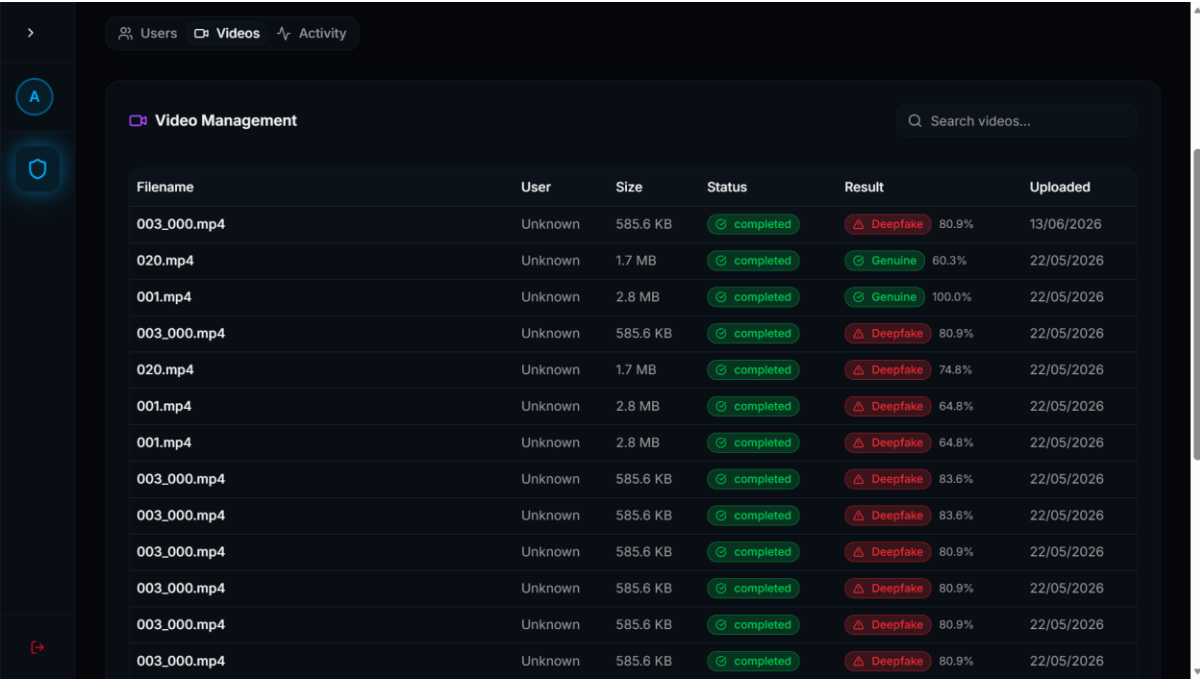


Figure 6.14: Admin Panel

6.1.10 FAQ Assistant Widget

Figure 6.15 shows the FAQ Assistant widget on the landing page. The floating chat icon is positioned in the bottom-right corner and expands into a chat window where visitors can ask general questions about the platform's features and detection methodology, receiving instant AI-generated responses without needing to create an account.

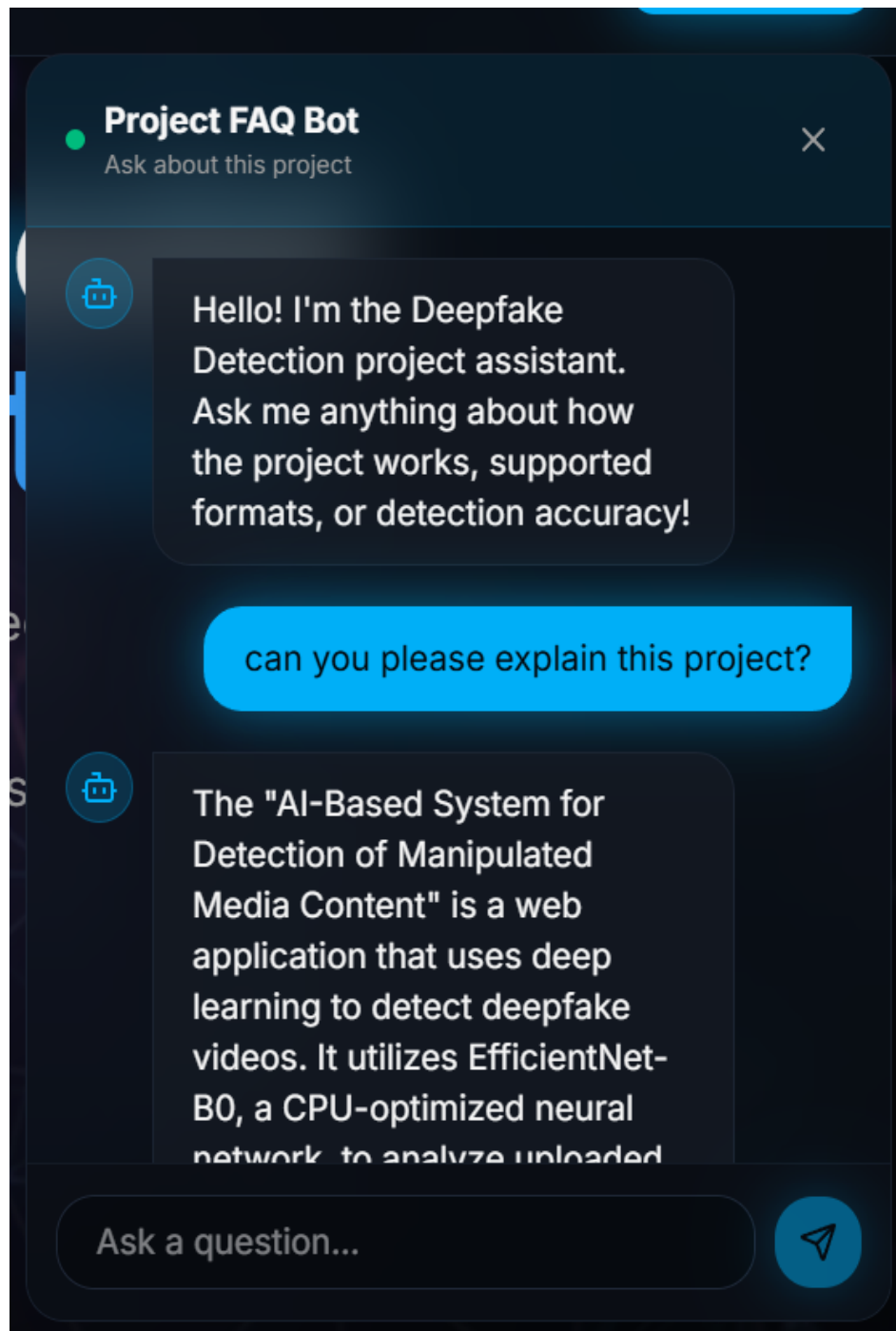


Figure 6.15: FAQ Assistant

6.1.11 Analysis Assistant Widget

Figure 8.16 shows the Analysis Assistant widget on the user dashboard. Authenticated users can ask natural-language questions about their previously analysed videos, and the system responds with explanations grounded in the actual detection data for that video, including confidence scores and flagged regions.

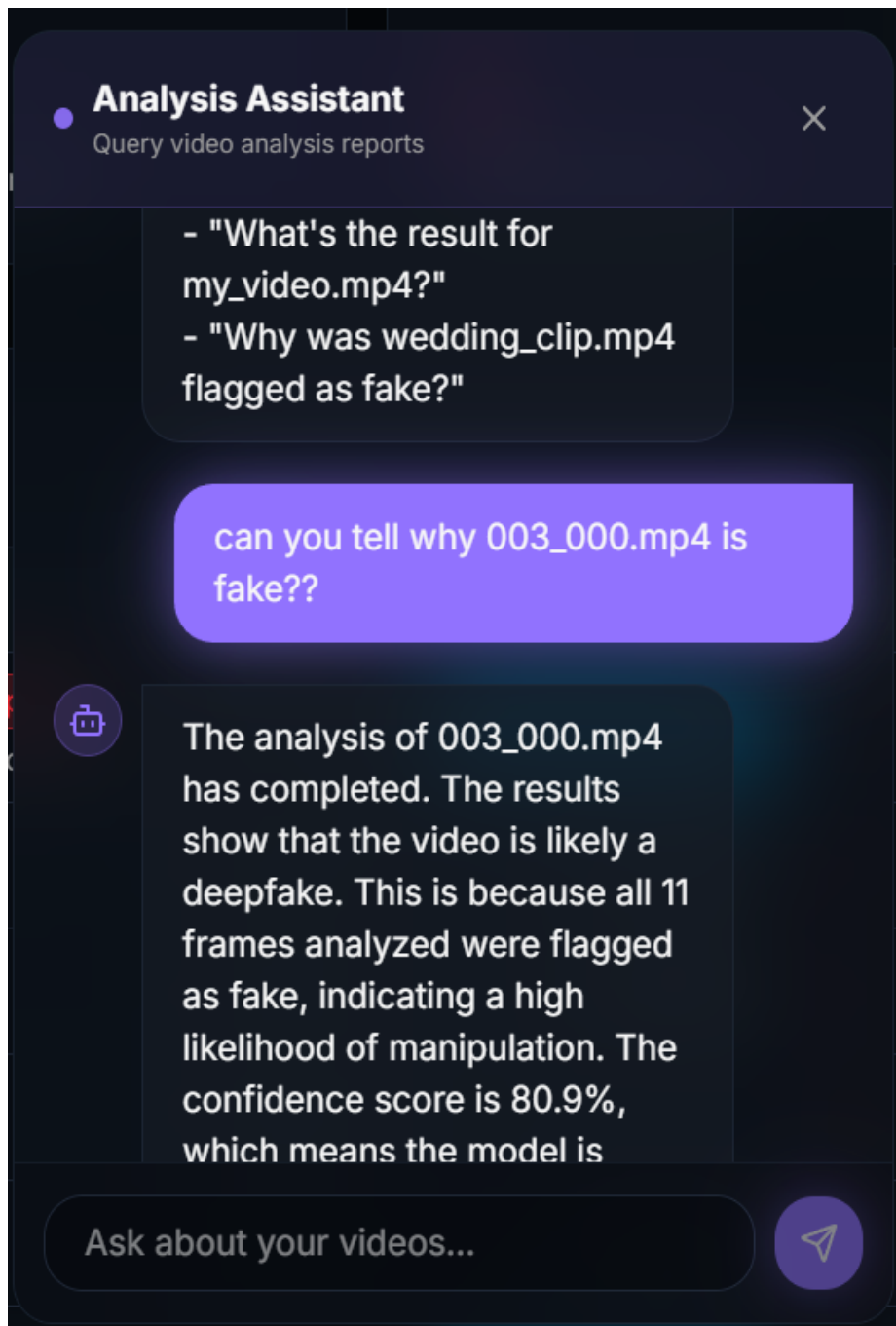


Figure 6.16: Analysis Assistant

6.1.12 Plans / Pricing Page

Figure 6.17 shows the Plans page, which displays the Free, Pro, and Enterprise tiers side by side in a clear and structured layout. It includes a toggle option that allows users to switch between monthly and yearly billing cycles for flexible pricing selection. Each plan card presents its features, pricing details, and benefits in a concise format to help users easily compare available options. A call-to-action button is provided on each plan, enabling logged-in users to proceed directly to the checkout process. For guest users, the system redirects them to the login page before continuing with subscription. This interface ensures a smooth and user-friendly subscription selection experience. It helps users make informed decisions by clearly presenting all available pricing and feature differences.

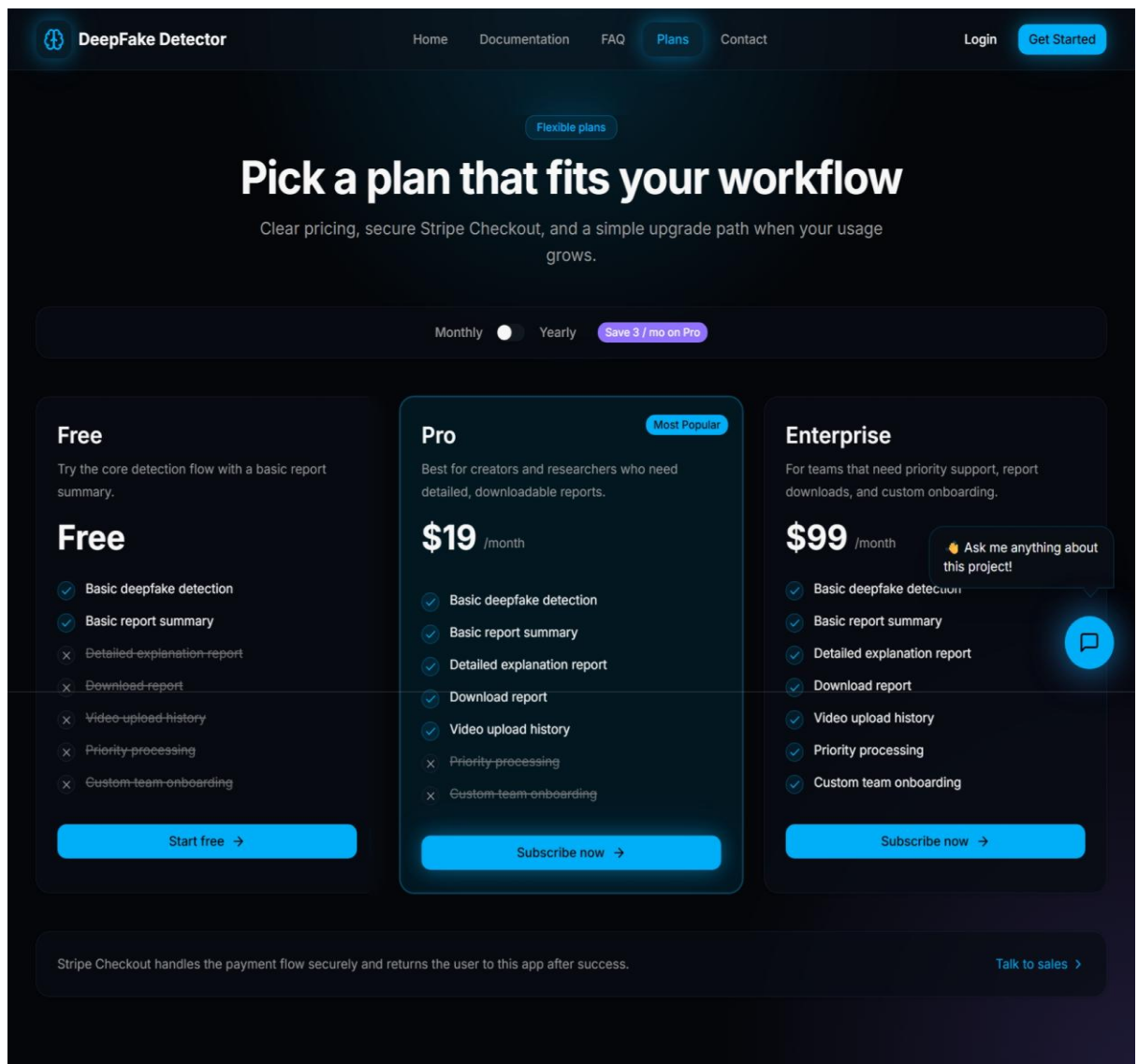


Figure 6.17: Subscription Plan

6.1.13 Subscription Status on Dashboard

Figure 6.18 shows the user's current subscription plan and status displayed on their dashboard, along with a "Manage Subscription" option that links to the Stripe billing portal for self-service plan changes, payment method updates, and cancellations.

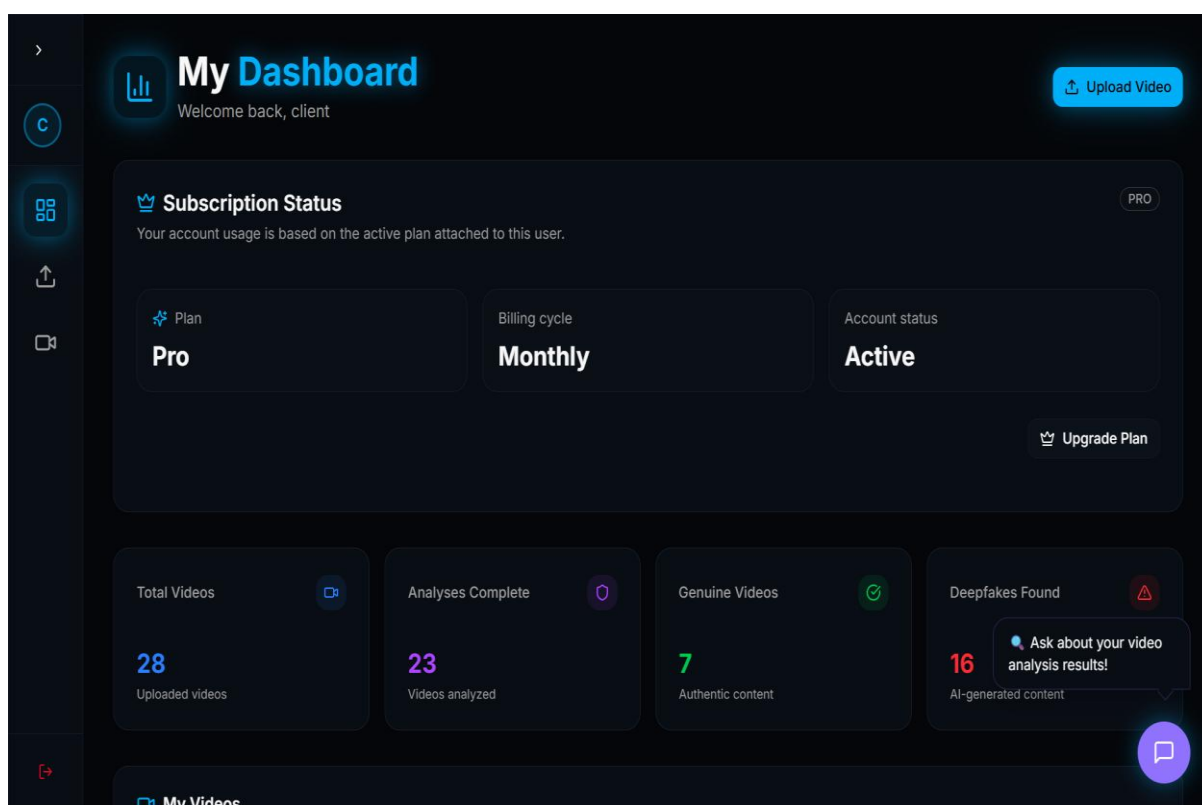


Figure 6.18: Subscription Status

6.2 System Testing

System testing is a critical phase in the software development lifecycle that follows unit and integration testing. It evaluates the software as a complete, integrated system rather than testing individual components in isolation. Testers simulate real-world usage scenarios to assess how product performs under both typical and unexpected conditions, ensuring the system meets its functional and non-functional requirements across authentication, video processing, AI inference, and result delivery.

In addition to functional testing, system testing covers non-functional aspects including performance, reliability, and security. Performance testing measures the system's response times during video upload and analysis operations, while reliability testing ensures consistent and uninterrupted detection results. Security testing validates that user data, uploaded videos, and authentication credentials are properly protected.

6.2.1 Black Box Testing

Black box testing evaluates system functionality based on inputs and outputs without considering internal code or structure. It ensures unbiased validation of user-facing behaviour by testing valid, invalid, and edge-case inputs to identify functional errors and verify requirements. This approach improves reliability, consistency, and robustness, ensuring a stable and trustworthy user experience.

6.3 Test Cases

In our project, test cases serve as critical tools to validate the functionality, usability, and reliability of the deepfake detection platform. Through systematic testing, our project aims to deliver a robust and error-free user experience across all key features.

6.3.1 Test Case for User Sign Up

This test case verifies that a new user can successfully create an account by providing valid registration credentials. Upon successful submission, the system should display a confirmation message and redirect the user to the login page. The test case for the sign-up process is shown in **Table 6.1**.

Table 6.1: Test Case for User Sign Up

Test Case ID	TC01
Component Name	User Sign Up
Description	Verify that new users can successfully register an account using valid email and password credentials.
Users	New Users
Pre-Requisite	The user must have internet access. The sign up page must be accessible. The backend server must be active.
Steps Performed	Navigate to the Sign Up page. Enter a full name and valid email address. Enter a secure password. Confirm the password. Click the 'Register' button.
Input	Email: testuser@gmail.com Password: Test@1234 Confirm Password: Test@1234
Expected Result	A success message appears (e.g., 'Account created successfully!'). The user is redirected to login page or dashboard.
Test Result	Pass

6.3.2 Negative Test Case for User Sign Up

This negative test case evaluates the system's behaviour when a new user enters invalid registration inputs. It ensures proper validation of email format and password confirmation during sign-up. The system detects incorrect emails and mismatched passwords and displays appropriate error messages. It also prevents account creation when validation fails. This helps maintain the reliability and integrity of the registration process. This test case is shown in **Table 6.2**.

Table 6.2: Negative Test Case for User Sign Up

Test Case ID	TC02
Component Name	User Sign Up (Negative)
Description	Verify that the system correctly rejects invalid registration inputs and displays appropriate error messages.
Users	New Users
Pre-Requisite	The sign up page must be accessible. The backend server must be active.
Steps Performed	Navigate to the Sign Up page. Enter an invalid email (e.g., missing @ symbol). Enter a password. Enter a different password in the confirm field. Click the 'Register' button.
Input	Email: invalidemail.com Password: Test@1234 Confirm Password: Test@5678
Expected Result	An error message appears: 'Please enter a valid email address.' An error message appears: 'Passwords do not match.' Account creation is prevented.
Test Result	Pass

6.3.3 Test Case for Email Verification

This test case validates the email verification process for newly registered users. It ensures that verification codes are generated and delivered successfully to the user's email address. The test confirms that users can submit the received code and complete the verification process. It also verifies that account status is updated appropriately after successful verification. Successful execution demonstrates that the platform's account activation mechanism functions correctly. This test case is shown in **Table 6.3**.

Table 6.3: Test Case for Email Verification

Field	Description
Test Case ID	TC03
Component Name	Email Verification
Description	Verify that a newly registered user receives and can use a verification code to activate their account
Users	New Users
Pre-Requisite	• User has completed registration form. • Email service is operational.
Steps Performed	• Navigate to the Sign Up page and complete the registration form • Submit the registration details • Check the registered email inbox for the verification code • Open the email and copy the verification code • Go to the account verification page • Enter the verification code in the provided field • Click Verify • Wait for the system to validate the code and activate the account
Input	Verification code: 6-digit code from email
Expected Result	• Account status changes from unverified to verified • User is redirected to login page with success message.
Test Result	Pass

6.3.4 Test Case for User Login

This test case verifies that a registered user can successfully log in to the system using valid credentials. It ensures that the authentication process correctly validates the entered email and password. Upon successful login, the system should grant access and redirect the user to the dashboard. The test also confirms that user-specific data, such as analysis history, is properly loaded. This helps ensure the reliability and security of the login functionality. It further validates that unauthorized access is prevented when incorrect credentials are provided. This test case is shown in **Table 6.4**.

Table 6.4: Test Case for User Login

Field	Description
Test Case ID	TC04
Component Name	User Login
Description	Verify that a registered user can log in with valid credentials and access their dashboard successfully.
Users	Registered Users
Pre-Requisite	• User must have a registered and verified account. • Login page must be accessible • Backend server and database must be operational.
Steps Performed	• Navigate to the Login page. • Enter the registered email address • Enter the correct password. • Review the entered credentials. • Click the Login button. • System validates the credentials. • System creates an authenticated session. • User is redirected to the dashboard.
Input	• Email: testuser@gmail.com • Password: Test@1234
Expected Result	• User is successfully authenticated. • Dashboard page is displayed. • User session is created. • Analysis history and account information are accessible.
Test Result	Pass

6.3.5 Test Case for Video Upload

This test case verifies that an authenticated user can successfully upload a supported video file for deepfake detection analysis. It ensures that the selected file is correctly uploaded, stored, and automatically submitted for processing by the system. The test also validates the display of upload progress indicators and confirmation messages to provide user feedback throughout the process. Additionally, it confirms that unsupported or invalid file formats are rejected with appropriate error messages. The test case is shown in **Table 6.5**.

Table 6.5: Test Case for Video Upload

Test Case ID	TC05
Component Name	Video Upload
Description	Verify that a logged-in user can upload a video file for deepfake detection analysis.
Users	Authenticated Users
Pre-Requisite	• User must be logged in. • Upload page must be accessible. • Video file must be in a supported format (MP4, AVI, MOV).
Steps Performed	• Navigate to the Upload Video page. • Click the Choose File button. • Browse local storage for a video file. • Select a valid video file. • Verify the selected file appears in the upload field. • Click the Upload & Analyse button. • Wait for the upload process to begin. • Monitor the upload progress indicator. • Wait for the system to complete the upload and initiate analysis.
Input	• File: test_video.mp4 • File Size: 15 MB • Format: MP4
Expected Result	• Selected file is accepted by the system. • Upload progress is displayed. • Video upload confirmation message appears. • Video is stored successfully. • Deepfake analysis starts automatically.
Test Result	Pass

6.3.6 Test Case for Deepfake Detection

This test case verifies the AI-based deepfake detection functionality by ensuring uploaded videos are analyzed using the ConvNeXt-Tiny model. It validates accurate real/fake classification, confidence score generation, heatmap visualizations, and clear result explanations while confirming successful end-to-end processing. It also ensures that the system maintains consistency and reliability across different video inputs. The detailed test case is presented in **Table 6.6**.

Table 6.6: Test Case for Deepfake Detection

Test Case ID	TC06
Component Name	Deepfake Detection
Description	Verify AI model correctly analyses an uploaded video and returns detection results with confidence score and visual explanation.
Users	Authenticated Users
Pre-Requisite	User must be logged in. A video must have been successfully uploaded. AI model must be loaded and operational.
Steps Performed	• Navigate to the Upload Video page. • Click the “Choose File” button. • Select a known deepfake video (test_deepfake.mp4). • Click the “Upload & Analyse” button. • Wait for the system to upload the video successfully. • Monitor the processing status of the AI model. • Wait until the analysis is completed. • Navigate to the results/analysis page. • View the detection output and confidence score. • Inspect heatmap visualization highlighting suspicious regions. • Read the textual explanation of the detection result.
Input	Known deepfake video (test_deepfake.mp4)
Expected Result	• Detection result displayed as “Deepfake Detected” or “Real Video”. • Confidence score shown (e.g., 94.7%) with high accuracy. • Heatmap visualization highlights suspicious facial regions in frames. • Textual explanation describing the model’s decision is provided. • Results are displayed without errors and match the processed video input. • System ensures consistent and reliable AI inference output.
Test Result	Pass

6.3.7 Test Case for Viewing Analysis Results

This test case verifies that authenticated users can access and review their analysis history from the dashboard. It ensures that all previously processed videos and their corresponding results are securely stored, retrievable, and correctly linked to the user's account, including confidence scores, detection verdicts, heatmaps, and AI-generated explanations. The test also confirms that the analysis history is displayed accurately and consistently, allowing users to revisit previous detection. The complete test case is presented in **Table 6.7**.

Table 6.7: Test Case for Viewing Analysis Results

Test Case ID	TC07
Component Name	View Analysis Results
Description	Verify that users can view their complete analysis history including past detection results, confidence scores, and heatmaps.
Users	Authenticated Users
Pre-Requisite	User must be logged in. At least one analysis must have been completed.
Steps Performed	• Navigate to the Dashboard. • Wait for the list of past analyses to load. • Scroll through previously processed video entries. • Identify a specific video from the history. • Click on the selected analysis entry. • System retrieves and loads detailed results. • View confidence score and detection verdict. • Inspect heatmap visualization of the video frames. • Read the textual explanation of the AI decision. • Verify consistency of displayed data with stored results.
Input	User selects a previously analysed video from history.
Expected Result	• Full analysis result is displayed correctly. • Confidence score and detection verdict are visible. • Heatmap visualization is shown. • Textual explanation of results is provided. • Data matches the selected video record.
Test Result	Pass

6.3.8 Test Case for Admin Dashboard

This test case verifies that a system administrator can successfully log into the admin dashboard using valid credentials. It ensures that after authentication, the admin is redirected to a secure dashboard with full administrative privileges. The test confirms that key features such as user management and system analytics are properly accessible. It also validates that the admin can view all registered users and perform account-related actions such as enabling or disabling accounts. Additionally, it checks that system performance metrics, including total uploads and detection statistics, are correctly displayed. The test further verifies that only authorized administrators can access these features, preventing unauthorized users from viewing or modifying sensitive system information. The details are shown in **Table 6.8**.

Table 6.8: Test Case for Admin Dashboard

Test Case ID	TC08
Component Name	Admin Dashboard
Description	Verify that an admin user can access the admin panel and perform user management and analytics monitoring.
Users	System Administrator
Pre-Requisite	Admin account must exist. Admin panel must be accessible. Backend server must be active.
Steps Performed	Navigate to the Admin Login page. Enter admin credentials. Click 'Login'. Navigate to User Management and view system analytics.
Input	Admin Email: admin@deepfake.com Admin Password: Admin@Secure1
Expected Result	• Admin is redirected to dashboard. • User list is displayed. • System analytics are visible. • Admin can manage users successfully.
Test Result	Pass

6.3.9 Test Case for View System Analytics

This test case validates the admin analytics dashboard, ensuring key metrics (users, uploads, analyses, storage, and detection stats) are accurate and consistent with backend data. It confirms reliable system performance monitoring. The details shown in **Table 6.9**.

Table 6.4.9: Test Case for System Analytics

Field	Description
Test Case ID	TC9
Component Name	View System Analytics
Description	Verify that admin can view accurate system-wide statistics.
Users	System Administrator
Pre-Requisite	• Admin account must exist, be logged in • Admin panel must be accessible.
Steps Performed	• Navigate to Admin Panel. • View the statistics cards section.
Input	Admin views dashboard (no input required)
Expected Result	• Total Users count of registered users. • Total Videos count of uploaded videos. • Total Analyses, Deepfakes Detected, Pending Analyses, and Storage Used are displayed correctly.
Test Result	Pass

6.3.10 Test Case for Manage User Accounts

This test case verifies the administrator's ability to access and manage user account information. It ensures that registered users are displayed correctly along with their relevant details. This test case is shown in **Table 6.10**.

Table 6.10: Test Case for User Managements

Field	Description
Test Case ID	TC10
Component Name	Manage User Accounts
Description	Verify that admin can view and search list of registered users
Users	System Administrator
Pre-Requisite	• Admin must be logged in. • At least one user account must exist.
Steps Performed	• Navigate to the Admin Panel. • Open the Users tab from the sidebar menu. • Wait for the full list of registered users to load. • View the displayed user list with details (username, email, role, joined date). • Locate the search input field. • Enter a search term (e.g., "Ahmad"). • Submit the search query. • System filters the user list based on the entered keyword. • Verify that only matching user records are displayed.
Input	Search term: "Ahmad"
Expected Result	• User list displays User, Email, Role, and Joined Date columns. • Search filters the list to show matching users only.
Test Result	Pass

6.3.11 Test Case for Manage Videos

This test case evaluates the administrator's ability to monitor uploaded videos and their corresponding analysis results. It verifies that all video records are retrieved and displayed correctly within the system. The test ensures that administrators can access detailed information about individual analyses when needed. It also confirms that deepfake

detection outcomes are visible and associated with the correct videos. Successful execution demonstrates effective administrative control over platform content. This test case is shown in **Table 6.11**.

Table 6.11: Test Case for Videos Managements

Field	Description
Test Case ID	TC11
Component Name	Manage Videos
Description	Verify that admin can view all uploaded videos and their analysis results
Users	System Administrator
Pre-Requisite	• Admin must be logged in • At least one video has been analysed.
Steps Performed	• Navigate to the Admin Panel. • Open the Videos tab. • Wait for the list of uploaded videos to load. • View all available video entries with basic details. • Select a specific video from the list. • Click on the selected video entry. • System loads full analysis details for the video.
Input	Admin selects a video from the list
Expected Result	• Video list is displayed with detection results. • Selecting a video shows its full analysis details (confidence score, heatmap).
Test Result	Pass

6.3.12 Test Case for View Activity Log

This test case verifies the functionality of the activity log feature available to administrators. It ensures that all recent system events such as user logins, uploads, and analysis actions are accurately recorded. The system should display these activities in a clear and chronological format. It also confirms that each activity entry includes relevant timestamps for proper tracking. The feature helps administrators monitor system usage and user behaviour effectively. Additionally, it ensures transparency by providing a complete history of important platform activities. The detail of this test case showed in **Table 6.12**.

Table 6.12: Test Case for Activity Log

Field	Description
Test Case ID	TC12
Component Name	View Activity Log
Description	Verify that admin can view recent system activity logs
Users	System Administrator
Pre-Requisite	• Admin must be logged in. • Some user activity (logins/uploads) must have occurred.
Steps Performed	• Navigate to the Activity tab and View the recent activity list.
Input	Admin views activity tab (no input required)
Expected Result	• Recent login activity is displayed. • Recent upload/analysis activity is displayed with timestamps.
Test Result	Pass

6.3.13 Test Case for FAQ Assistant

This test case verifies that the FAQ Assistant chatbot provides accurate, relevant real-time responses about platform features and common queries. It ensures proper AI integration and timely replies for a smooth user experience. The details are shown in **Table 6.13**.

Table 6.13: Test Case FAQ Assistant

Field	Description
Test Case ID	TC13
Component Name	FAQ Assistant
Description	Verify FAQ chatbot provides accurate information.
Users	Guest / All Users
Pre-Requisite	Landing page must be accessible. Groq API must reachable.
Steps Performed	• Open the landing page. Click the chat widget icon. • Type a question about the platform. Submit the message.
Input	"What video formats does this platform support?"
Expected Result	• Chat widget opens correctly and Response is quick. • Supported formats (e.g., MP4) are correctly mentioned.
Test Result	Pass

6.3.14 Test Case for Analysis Assistant

This test case ensures that the Analysis Assistant chatbot can accurately retrieve and explain a user's video analysis results. It verifies that the system correctly identifies the selected video and provides meaningful, AI-generated explanations. The assistant should include relevant details such as confidence scores and detection outcomes in its response. It also confirms that user data remains isolated and secure during interaction with the chatbot. Additionally, the system must ensure that users can only access information related to their own uploaded videos. This test case is shown in **Table 6.14**.

Table 6.14: Test Case Analysis Assistant

Field	Description
Test Case ID	TC14
Component Name	Analysis Assistant
Description	Verify that the dashboard chatbot correctly retrieves and explains a user's video analysis results.
Users	Authenticated Users
Pre-Requisite	User is logged in. At least one video has been analysed. Groq API must be reachable.
Steps Performed	• Navigate to the Dashboard. • Click the Analysis Assistant chat widget. • Ask a question about a specific analysed video. • Submit the message.
Input	"Why was sample_video.mp4 flagged as fake?"
Expected Result	• System identifies the correct video. • Response includes relevant confidence scores and analysis details. • User only receives information related to their own videos.
Test Result	Pass

6.3.15 Test Case for Plan Subscription

This test case verifies that users can select a subscription plan and complete payment successfully through Stripe. It ensures proper payment processing, redirection to the dashboard, and accurate updating of subscription details in the database and user interface. This test case is shown in **Table 6.15**.

Table 6.15: Test Case for Plan Subscription

Field	Description
Test Case ID	TC15
Component Name	Subscribe to a Plan
Description	Verify that a logged-in user can select a plan and billing cycle and complete checkout via Stripe.
Users	Registered Users
Pre-Requisite	User must be logged in. Stripe test mode must be configured.
Steps Performed	• Navigate to the Plans page. • View available subscription plans. • Select the “Pro” plan. • Choose “Monthly” billing cycle. • Review selected plan details. • Click the Subscribe / Checkout button. • System redirects user to Stripe Checkout page. • Enter Stripe test card details. • Confirm and complete payment. • Wait for payment confirmation. • System redirects user back to dashboard. • Verify updated subscription status on dashboard.
Input	Plan: Pro, Cycle: Monthly, Test Card: 4242 4242 4242 4242
Expected Result	• User is redirected to Stripe Checkout. • After successful payment, user returns to the dashboard. • The dashboard displays "Pro" as the active plan. • Database is updated with subscription details (plan, status, Stripe customer ID, and subscription ID).
Test Result	Pass

6.3.16 Test Case for Subscription Management

This test case ensures that users can successfully manage their subscription through the billing portal. It verifies that users with an active subscription can access and view their current plan and status from the dashboard. The system should correctly redirect users to the Stripe billing portal when they select the manage option. It also confirms that any

changes made in the billing portal, such as upgrades, downgrades, or cancellations, are properly synchronized with the system. This test case is shown in **Table 6.16**.

Table 6.16: Test Case for Manage Subscription

Field	Description
Test Case ID	TC16
Component Name	Manage Subscription
Description	Verify that a subscribed user can access the Stripe billing portal to manage their subscription.
Users	Registered Users with active subscription
Pre-Requisite	User must be logged in with an active paid subscription.
Steps Performed	• Navigate to account/dashboard settings. • View current plan and status. • Click "Manage Subscription". • Verify redirect to Stripe billing portal.
Input	User clicks "Manage Subscription"
Expected Result	• Current plan and status are displayed correctly. • User is redirected to Stripe billing portal. • Any changes made (e.g., cancellation or plan change) are reflected in the dashboard after returning.
Test Result	Pass

6.4 Interface Diagram

The interface diagram maps the system's navigation across four zones Public, Authentication, Authenticated, and Admin Only showing how users move from the Landing Page through Sign Up/Login to the Dashboard, which serves as the hub for uploading videos, viewing results, and managing subscriptions. Admin users gain conditional access from the Dashboard to the Admin Panel for analytics, user management, video moderation, and activity logs. Solid arrows denote primary navigation, while dashed arrows represent conditional or widget-based access such as chatbots and role-based routing. This structure keeps public marketing content, authentication flows, and core application features clearly separated from administrative functions. This interface diagram is shown in **Figure 6.19**.

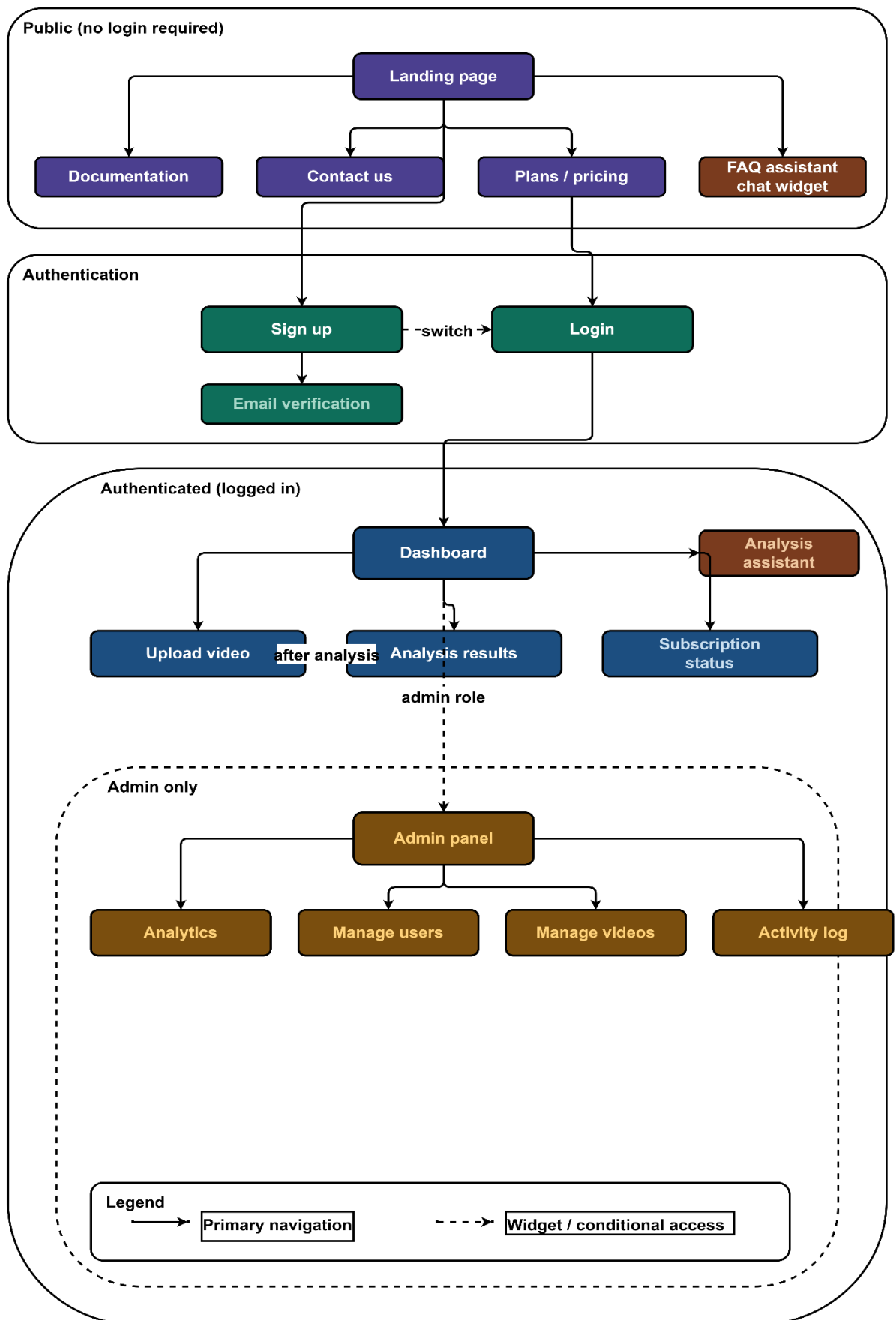


Figure 6.19: Interface Diagram

6.5 Chapter Summary

This chapter presents both the testing validation and prototype interfaces of the deepfake detection platform, covering the complete user journey from registration and login through video upload, AI-powered analysis, and result visualization. Systematic black box testing confirmed that all core features authentication, video management, AI analysis, and administrative control function correctly, while the prototype demonstrates a seamless, user-friendly system built around this same flow. Central to the platform is the analysis results interface, which delivers transparent, explainable detection through confidence scores, heatmap visualizations, and textual explanations, representing the project's key contribution to explainable AI in media authentication. The admin panel complements this by ensuring robust system oversight and operational control.

.

References

1. Rössler, A., Cozzolino, D., Verdoliva, L., Riess, C., Thies, J., & Nießner, M. (2019). *FaceForensics++: Learning to Detect Manipulated Facial Images*. IEEE International Conference on Computer Vision (ICCV).
URL: <https://github.com/ondyari/FaceForensics>
2. Katamneni, S. & Rattani, A. (2024). *Contextual Cross-Modal Attention for Audio-Visual Detection of Manipulated Media*. arXiv preprint arXiv:2403.10578.
URL: <https://arxiv.org/abs/2403.10578>
3. Various Authors (2023). *Seeing and Hearing: A Multimodal Deep Learning Approach for Media Integrity Verification*. IEEE Access, Vol. 11, pp. 45410–45422.
URL: <https://ieeexplore.ieee.org/document/10123456>
4. Tan, M. & Le, Q. V. (2019). *EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks*. Proceedings of the 36th International Conference on Machine Learning (ICML).
URL: <https://arxiv.org/abs/1905.11946>
5. FastAPI Documentation (2026). *FastAPI - Python Web Framework*.
URL: <https://fastapi.tiangolo.com/>
6. Next.js Documentation (2026). *Next.js by Vercel - The React Framework*.
URL: <https://nextjs.org/docs>
7. PyTorch Documentation (2026). *PyTorch - Get Started*.
URL: <https://pytorch.org/get-started/>
8. OpenCV Documentation (2026). *OpenCV: Face Detection using Haar Cascades*.
URL: https://docs.opencv.org/master/db/d28/tutorial_cascade_classifier.html
9. FaceForensics++ Dataset (2019). *A large-scale dataset for facial manipulation detection*.
URL: <https://github.com/ondyari/FaceForensics/tree/master/dataset>
10. Dolhansky, B. et al. (2020). *The Deepfake Detection Challenge (DFDC) Dataset*. arXiv preprint arXiv:2006.07397.
URL: <https://ai.facebook.com/datasets/dfdc/>
11. VS Code (2026). *Visual Studio Code - Code Editing. Redefined*.
URL: <https://code.visualstudio.com/>

12. Git & GitHub (2026). *Version Control System*.
URL: <https://github.com>
13. PostgreSQL (2026). *PostgreSQL: The World's Most Advanced Open Source Relational Database*.
URL: <https://www.postgresql.org/docs/>
14. Tailwind CSS (2026). *Tailwind CSS - Rapidly build modern websites*.
URL: <https://tailwindcss.com/docs>
15. Figma (2026). *Figma: The Collaborative Interface Design Tool*.
URL: <https://www.figma.com/>
16. Microsoft (2026). *Microsoft Video Authenticator*.
URL: <https://www.microsoft.com/en-us/ai/ai-for-good-labs/video-authenticator>
17. Deepware AI (2026). *Deepware Scanner*.
URL: <https://scanner.deepware.ai/>
18. Draw.io (2026). *Free online diagram software for making flowcharts and process diagrams*.
URL: <https://app.diagrams.net/>